

Gestion centralisée de la sécurité avec KeyCloak

David THIBAU – 2026

david.thibau@gmail.com

Keycloak - Identity and Access Management for Modern Applications
Stian Thorgersen | Pedro Igor Silva

Agenda

J1 – Introduction

- Fonctionnalités, installation, UIs Admin et Utilisateur, 1ère application

J1 – Rappels sur les standards

- OAuth2, OpenID Connect, JWT, types de jetons

J1 – Keycloak et OpenID Connect

- Discovery, authentification, ID token, UserInfo, logout

J2 – Autorisation avec OAuth2

- Consentement, limitation des accès, personnalisation et validation du jeton

J2 – Types d'application

- Web, native ou mobile, services back-end

J2 – Intégration Keycloak

- Librairies, Spring Boot, Quarkus, Python, reverse proxy, débogage

J3 – Exploitation courante

- Utilisateurs, mots de passe et MFA, sessions et jetons, événements et audit

J3 – Stratégies d'autorisation

- RBAC, GBAC, scopes OAuth2, Authorization Service

J4 – Adaptation et personnalisation

- Organisation, flows, thèmes, fédération d'identités, SPI, Admin API

J4 – Vers la production

- Configuration, clustering et HA, durcissement

J5 – Supervision et maintenance

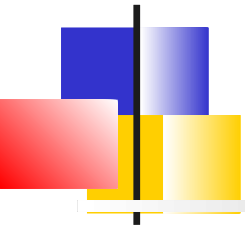
- Métriques, logs, sauvegarde, migrations, incidents

J5 – Automatisation

- Scripts, sauvegardes planifiées, CI/CD et GitOps, audit

Annexes

- FAPI, mTLS et CIBA, RFC 9700, DPoP, FIPS



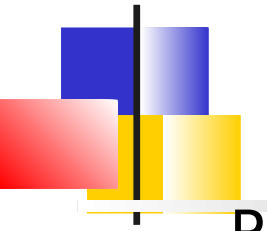
Introduction

Fonctionnalités et installation

Interfaces Admin et utilisateur

Sécuriser une 1ère application

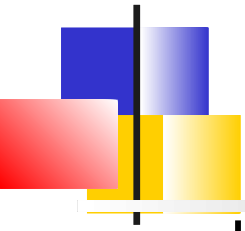
Keycloak



- Produit OpenSource pour la gestion d'identité et des accès (IAM) dédié aux applications modernes (SPA, Mobile, API Web) :
  => Les applications n'ont plus besoin d'implémenter l'authentification ni le stockage de mot de passe
  => On obtient du SSO à l'intérieur de l'entreprise
- Projet démarré en 2014, largement déployé en entreprise depuis
  - Hautement personnalisable, Support pour l'authentification forte, application de stratégies pour les comptes utilisateur
  - S'appuie sur les standards oAuth2.0, OpenIDConnect et SAML
  - S'intègre avec des fournisseurs d'identité tierces : LDAP, OpenID Porvider, ...

Détails fonctionnalités

- ✔ **Authentification** : SSO et logout centralisé, MFA (OTP, WebAuthn / passkeys), flows configurables
- ✔ **Gestion des comptes** : inscription, mot de passe oublié, actions requises (CGU, vérification email, mise à jour du profil), console du compte
- ✔ **Fédération d'identités** : annuaires LDAP / AD, fournisseurs OIDC, SAML et sociaux
- ✔ **Autorisation** : rôles, groupes, scopes, Authorization Services
- ✔ **Administration** : console, API REST, CLI, événements utilisateur et admin
- ✔ **Personnalisation** : thèmes, internationalisation, extensions SPI
- ✔ **Exploitation** : clustering, métriques, déploiement conteneur et Kubernetes
- ✔ Les applications délèguent l'authentification à Keycloak et ne manipulent jamais les mots de passe : elles reçoivent des jetons standards



Protocoles supportés

Keycloak implémente les principaux standards d'identité :

- **OpenID Connect** : couche d'authentification au-dessus d'OAuth 2.0, jetons JWT
- **OAuth 2.0** : délégation d'autorisation, obtention de jetons d'accès
- **SAML 2.0** : assertions XML signées, historique des SSO d'entreprise

Chaque client est déclaré avec un protocole : openid-connect ou saml

Keycloak joue le rôle de fournisseur (OP OIDC ou IdP SAML) et peut aussi fédérer des fournisseurs externes des 2 types

SAML ou OpenID Connect ?

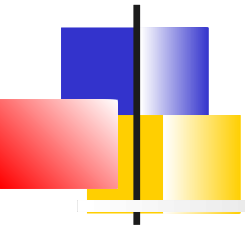
OpenID Connect

- Format JSON / REST, jetons JWT compacts
- Adapté aux SPA, applications mobiles, API et micro-services
- Choix recommandé pour les nouvelles applications

SAML 2.0

- Format XML, échanges via le navigateur (bindings Redirect et POST)
- Pas de notion de jeton d’accès pour appeler des API
- À conserver pour les applications existantes ou progiciels ne supportant que SAML

Stack technologique



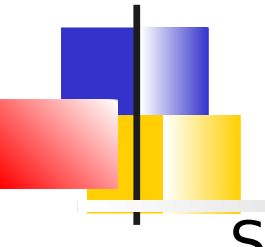
Avant Fin 2020 : Application web déployée sur le serveur Wildfly

Après Fin 2020 : Construit au-dessus du framework Quarkus[1] :

- Temps de démarrage réduit
- Faible empreinte mémoire
- Approche conteneur
- Meilleure expérience de développeur
- Meilleure convivialité

1. Une pile native Kubernetes et Cloud avec les meilleures bibliothèques et standards Java

Architecture et principes de fonctionnement



Serveur Java construit sur Quarkus, déployable en cluster

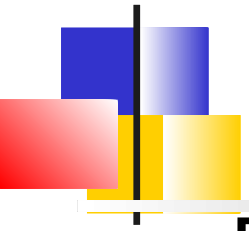
Persistance dans une base relationnelle : PostgreSQL, MySQL, MariaDB, Oracle, SQL Server

- Realms, clients, utilisateurs, rôles, configuration, sessions

Caches Infinispan : données de configuration et sessions, répliqués entre les nœuds

Principe : l'application délègue l'authentification à Keycloak

- L'utilisateur saisit ses identifiants uniquement sur Keycloak
- L'application reçoit des jetons signés par les clés du realm
- Les services vérifient localement la signature des jetons



Installation

- Différentes options pour l’installation :
  - Installation locale via une JVM
  - Démarrage de conteneur *Docker* ou *Podman*
  - Déploiement sur Kubernetes ou OpenShift
    avec en option l’utilisation de *Keycloak Kubernetes Operator* qui facilite l’installation, la configuration et l’exploitation



Répertoires de la distribution

bin/ : scripts *kc.sh / kc.bat* (start, start-dev, build, export, import, show-config), *kcadm.sh, kcreg.sh*

conf/ : *keycloak.conf*, configuration du cache (cache-ispn.xml)

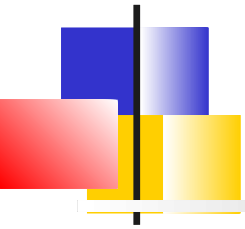
data/ : base H2 du mode dev, fichiers temporaires

- data/import : realms importés au démarrage avec --import-realm

lib/ : librairies du serveur, ne pas modifier

providers/ : extensions (SPI) sous forme de jar

themes/ : thèmes personnalisés



Introduction

Fonctionnalités et installation

Interfaces Admin et utilisateur

Sécuriser une 1ère application

Console d’administration

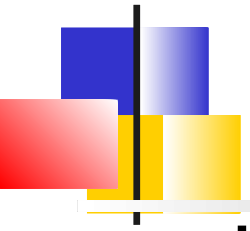
La console d’administration permet de gérer des **realms** :

- Ensemble d’utilisateurs
  - Rôles, groupes, sessions
- Ensemble d’applications clientes
  - Mécanismes d’obtention de jetons : Grant flows
  - Scopes : Accès aux données utilisateur

Un realm est complètement isolé des autres realms

- Par exemple, un realm pour les applications internes et les employés, et un autre pour les applications externes

Le realm *master* détermine la sécurité du serveur KeyCloak



Création de realm

- Le formulaire de création de realm ne demande qu'un nom
- Le nom est utilisé dans les URLs (Pas d'espace ni de caractères spéciaux)
  - Une nom d'affichage peut également être précisé
- Un *realm* peut s'exporter/s'importer via le format JSON

Création d’utilisateur

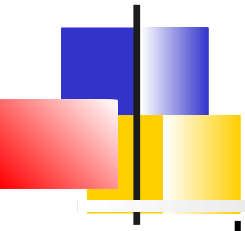
- Après la création de *realm*, on crée des utilisateurs :
  - *username* <=> login
  - email, nom et prénom
  - Actions requises au premier login : Vérification de son profil et de son email par exemple
- On peut ensuite compléter les attributs keycloak par des attributs personnalisés
- Avant que l’utilisateur puisse se logger, il faut définir un mot de passe (temporaire ou non)

Création de groupe

Un **groupe d’utilisateur** permet de mutualiser des attributs ou rôles aux utilisateurs appartenant au groupe.

Un groupe est donc :

- Un nom
- Une liste d’attributs
- Éventuellement une liste de rôles



Création de client

Un client représente une application intégrée à Keycloak.

Lors de la création d'un client, on indique :

- Son nom
- Si il doit être authentifié
- Le flow d'interaction avec Keycloak : *Authorization Code, Client Credentials, Device Flow, CIBA*
- Son usage ou pas du service d'autorisation de keycloak
- Les scopes possibles (accès aux données utilisateurs)
- Les rôles attribuables spécifique de ce client

Création de rôle

Un **rôle** est généralement utilisé par les applications clientes pour déterminer les droits d'accès.

Au niveau de *KeyCloak*, un rôle est :

- Un nom
- Une description

Il existe un espace de noms global pour les rôles (realm roles) et un espace de nom dédié à un client.

Un rôle peut être associé à des utilisateurs ou des groupes

Un rôle peut être composite et donc contenir d'autres rôles.



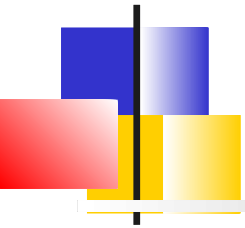
Interface utilisateur

Les utilisateurs accèdent à KeyCloak pour gérer leur propre compte.

L'URL d'accès est :
http://<serveur>/realms/<realm-name>/account/

Par exemple :

- Mettre à jour son profil d'utilisateur
- Mettre à jour son mot de passe
- Activer l'authentification à deux facteurs
- Lister ses applications, les applications auprès desquelles ils se sont authentifiés
- Lister les sessions ouvertes, y compris la déconnexion à distance d'autres sessions



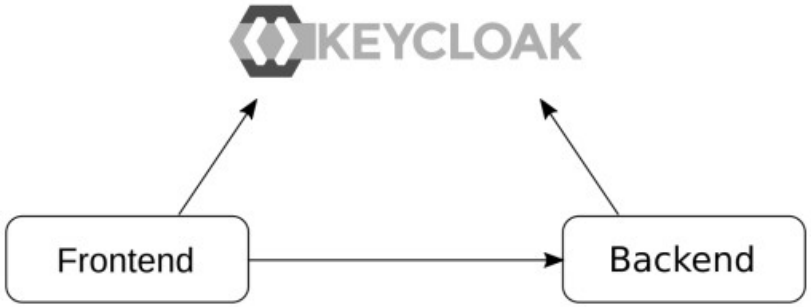
Introduction

Fonctionnalités et installation
Interfaces Admin et utilisateur
Sécuriser une 1ère application

Exemple : SPA + API Rest

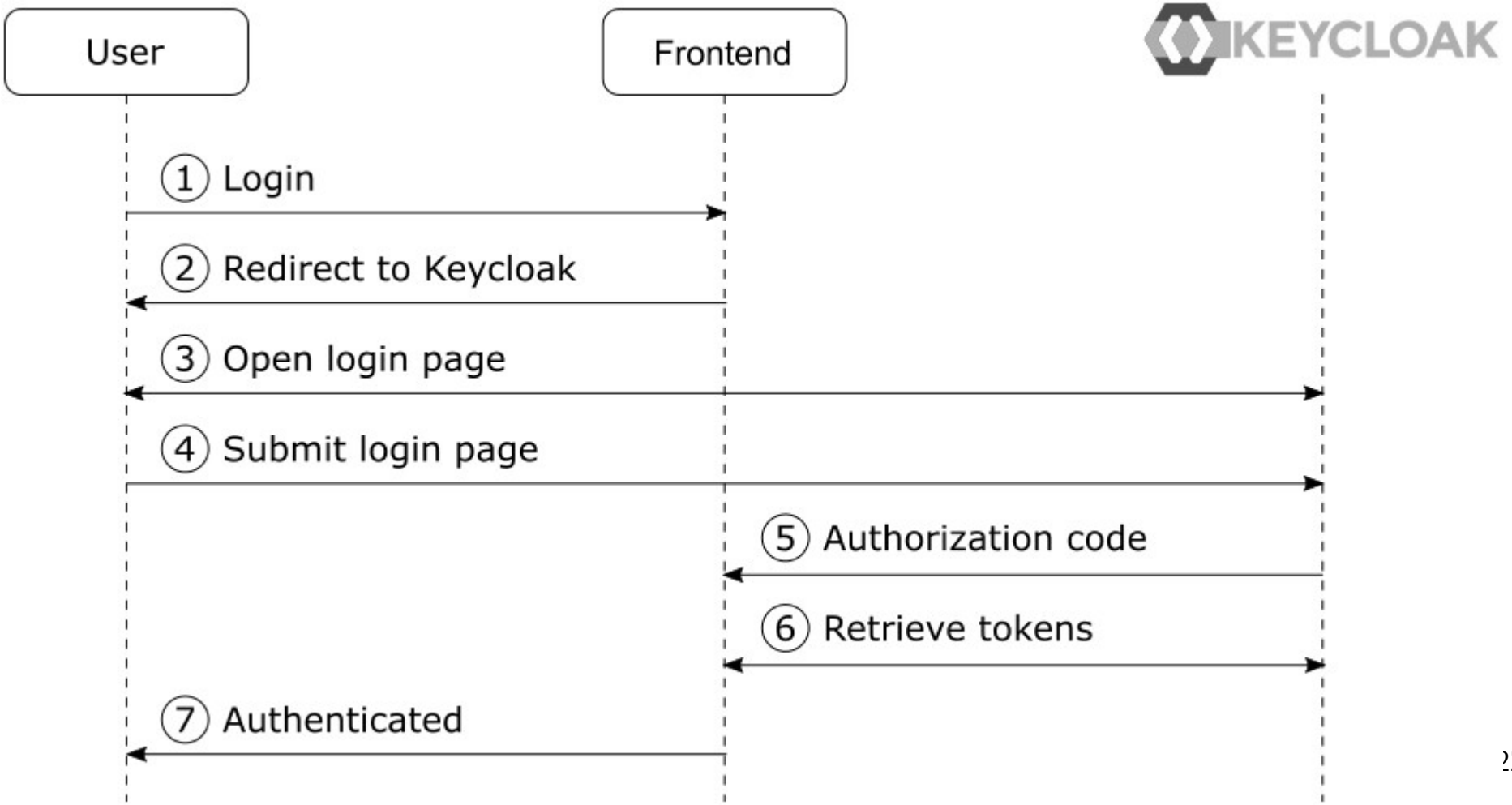
L'application exemple est composée de 2 parties :

- une **front-end** type SPA
  - Authentifiant l'utilisateur via Keycloak afin d'obtenir des jetons le représentant : ID, accès et rafraîchissement
  - Utilisant le jeton d'accès pour invoquer une API Rest
- un **backend** dont les ressources sont protégées par oAuth2



Authentification FRONT-END

OpenID / Authorization code flow



Claims Jetons

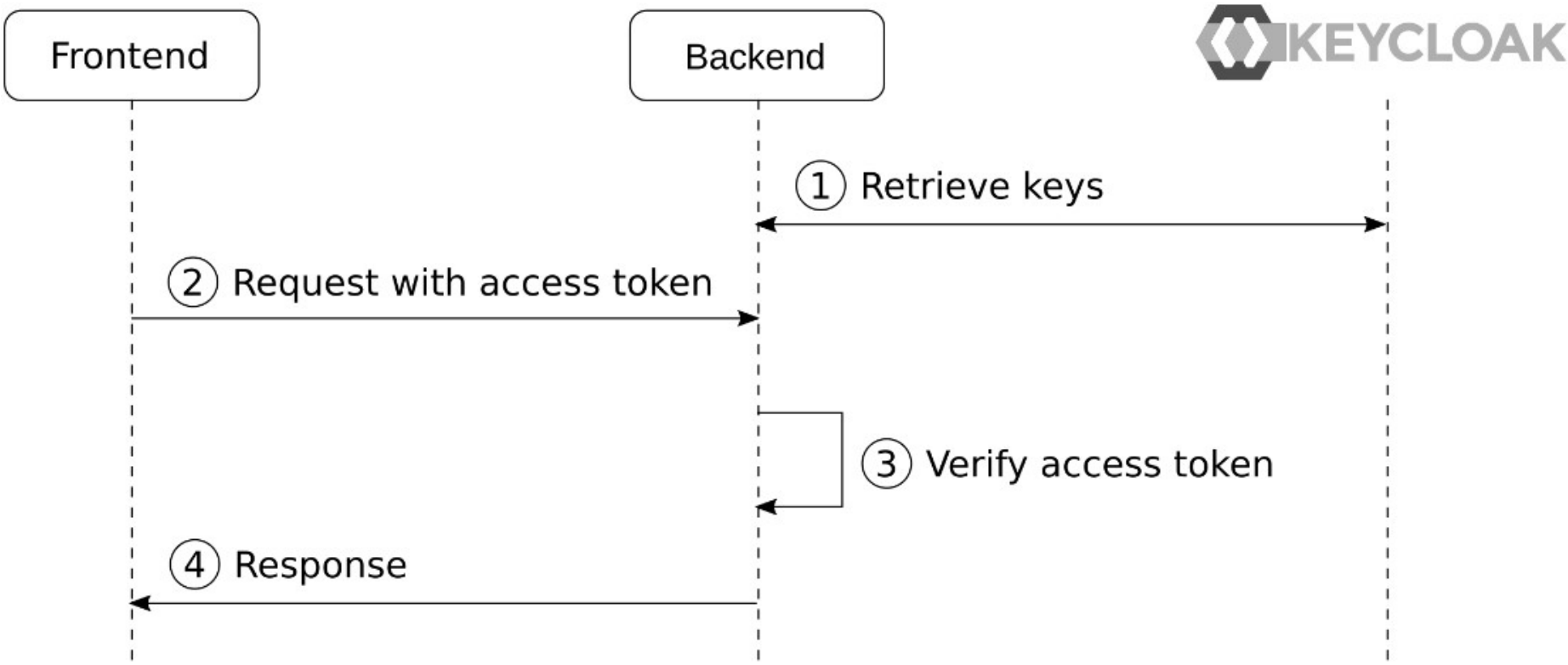
Les jetons obtenus contiennent des ***claims*** dépendant du ***scope*** demandé. Les jetons sont hautement personnalisables :

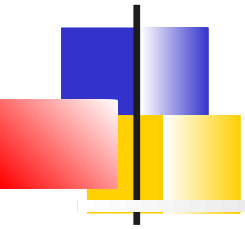
- ***ID_TOKEN*** : Établit l'identité de l'utilisateur
  - ***exp*** : Date d'expiration
  - ***iss*** : Issuer
  - ***sub*** : Identifiant unique de l'utilisateur
  - ***name, preferred_name, …***
- ***ACCESS_TOKEN*** : Détermine les droits d'accès
  - ***allowed-origins*** : Pour le CORS
  - ***realm-access*** : Intersection entre les rôles de l'utilisateur et les rôles accessibles par le client
  - ***resource_access*** : Les rôles du client
  - ***scope*** : les scopes initiaux demandés

TP 1.3

Vérification des accès BACKEND

JSON Web Signature (JWS)





Rappels sur les standards

OAuth2

OpenID Connect

JWT

Transmission et protection des jetons

SAML 2.0

Apports de oAuth2

- Protocole d'autorisation défini pour
  - Le partage de données utilisateur entre applications
  - Le contrôle les données partagées.
- Au départ, orienté applications web grand public, il peut être appliqué aux applications d’entreprise pour contrôler les accès

Rôles définis dans *oAuth2*

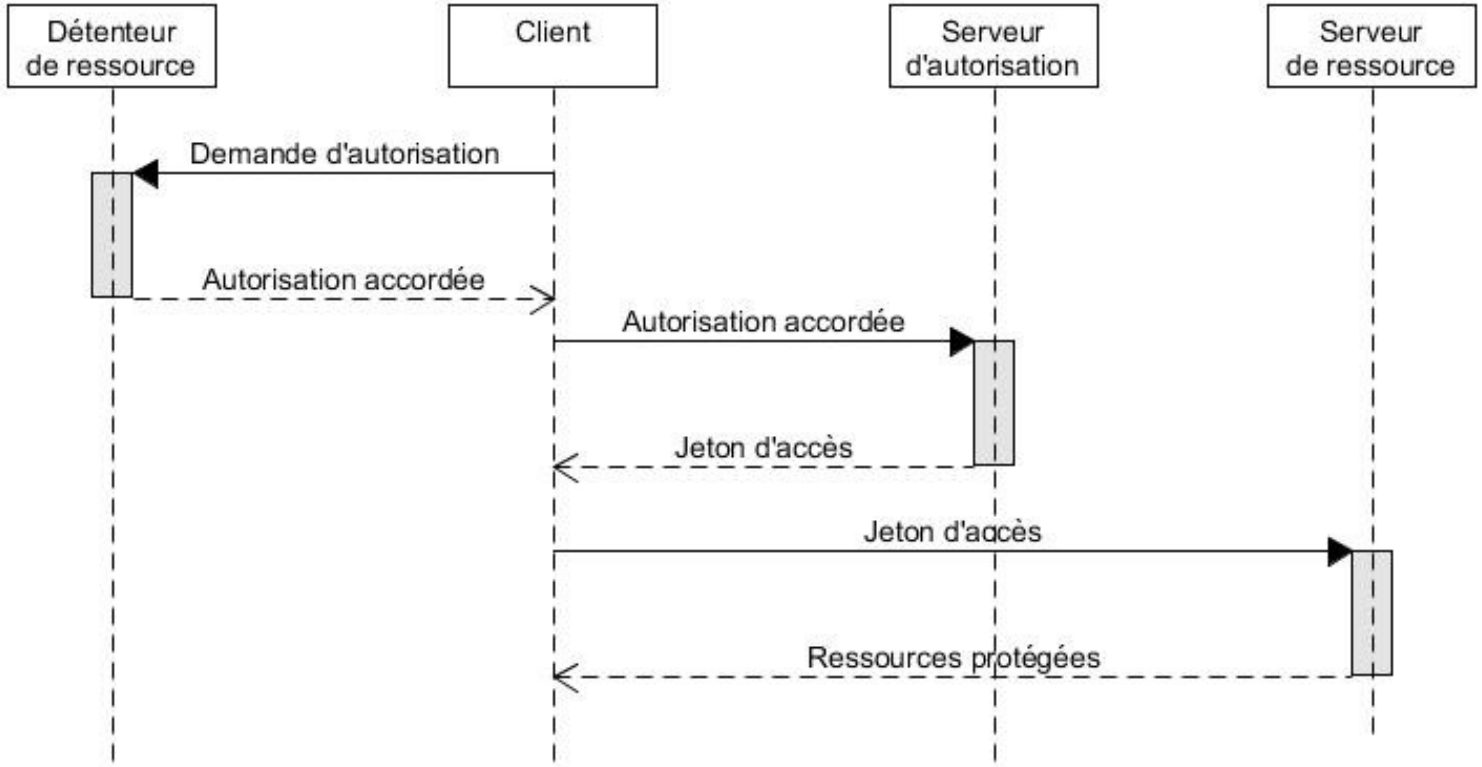
Propriétaire de la ressource : il s'agit généralement de l'utilisateur final qui possède les ressources auxquelles l'application veut accéder.

Serveur de ressources : C'est le service hébergeant les ressources protégées.

Client : Il s'agit de l'application qui souhaite accéder à la ressource

Serveur d'autorisation : C'est le serveur qui délivre l'accès au client. (Keycloak)

Étapes du protocole



Obtention de jetons
les différents flux ou Grant type

Selon le type d'application et le contexte d'usage, différents flows OAuth 2.0 permettent d'obtenir un jeton d'accès. Certains sont recommandés, d'autres à éviter.:

- ***Authorization Code*** : Le plus utilisé, l'utilisateur consent et est authentifié chez le fournisseur de jeton. Un code est alors renvoyé à l'application client qui l'utilise pour obtenir le jeton
  Adapté aux applications web, mobiles, SPA (avec PKCE).
- ***Client Credentials :*** L'application cliente détient un secret qui lui suffit pour obtenir le jeton. L'application accède à la ressource en son propre nom (l'application est le détenteur de la ressource).
  Adapté aux backend consommant des APIs
- ***Device Flow :*** L'application s'exécute sur un appareil sans browser et ne peut pas détenir un secret
  Recommandé pour les cas "sans navigateur" et clients publics
- Le ***Password flow*** implique que l'application client obtienne les mots de passe de l'utilisateur
  Déprécié , à éviter.
- ***L'Implicit flow*** défini dans la spéc est à éviter car il expose le jeton. Identique à Authorization Code Flow mais sans code d'autorisation, retour direct du jeton.
  Déprécié, à éviter.

Types de clients

Les flux sont donc choisis en fonction du type de client.

- Les clients **confidentiels** sont des applications pouvant stocker des crédentiels en toute sécurité.
  EX : Application back-end

- Les clients **publics**, en revanche, sont des applications exécutées côté client qui ne sont pas en mesure de stocker des crédentiels.
  En général, ils s'exécutent dans le navigateur

Protections pour les clients publics

Les clients publics s'exécutant dans un navigateur utilisent ***l'Authorization Code Flow***

2 mesures de protection supplémentaires sont en place :

- ***Valid redirect URI*** : Le serveur d'autorisation n'enverra le code permettant de récupérer un jeton seulement sur une URI prédéfinie.
- ***Proof Key for Code Exchange (PKCE, RFC 7636)*** : Extension d'OAuth 2.0 qui empêche quiconque ayant intercepté un code d'autorisation de l'échanger contre un jeton d'accès.

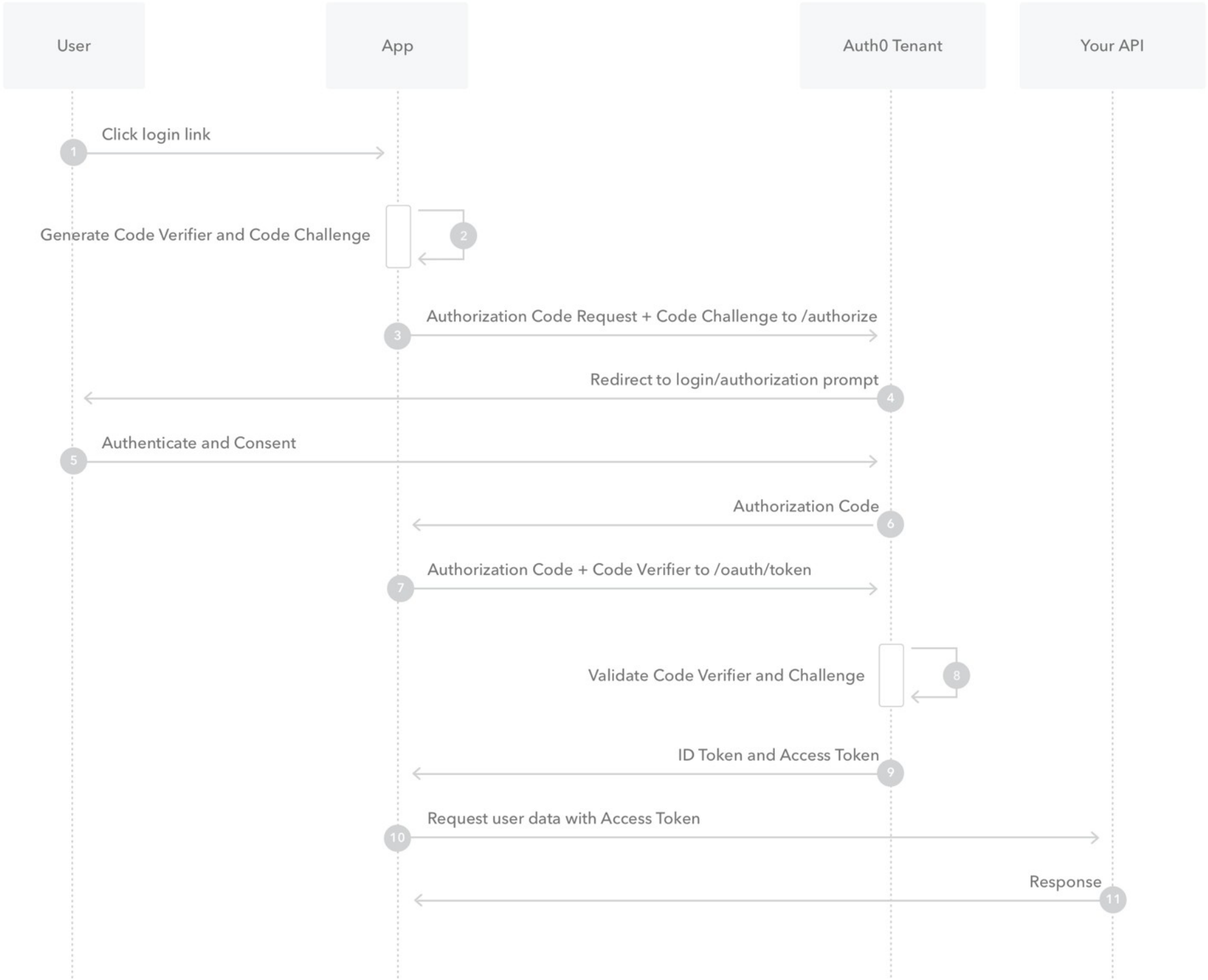
Flux avec PKCE

L’application cliente publique crée une chaîne aléatoire unique : le ***code_verifier***.

Le hash du *code_verifier* est ajouté à la requête de code d’autorisation sous le paramètre ***code_challenge***.

Une fois l’utilisateur authentifié et le code renvoyé, l’application demande les jetons avec le code d’autorisation et le *code_verifier*.

Si les codes correspondent, l’authentification est terminée et un *access_token* est renvoyé.


User
App
Auth0 Tenant
Your API
Click login link
1
Generate Code Verifier and Code Challenge
2
Authorization Code Request + Code Challenge to /authorize
3
Redirect to login/authorization prompt
4
Authenticate and Consent
5
Authorization Code
6
Authorization Code + Code Verifier to /oauth/token
7
Validate Code Verifier and Challenge
8
ID Token and Access Token
9
Request user data with Access Token
10
Response
11

Validation des jeton

Lors de la réception du jeton, le serveur de ressource doit valider l'authenticité du jeton et extraire ses informations différentes techniques sont possibles

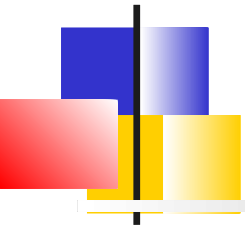
- Jetons opaque et Appel REST vers le serveur d'autorisation
- Jetons JWT et validation via paire de clés privé/publique

Spécifications additionnelles

Bearer tokens (RFC 6750) : Les jetons Bearer sont de loin le type de jetons d'accès le plus couramment utilisé, et ils sont généralement envoyés aux serveurs de ressources via l'en-tête *HTTP Authorization*.

Token Introspection (RFC 7662) : Dans OAuth 2.0, le contenu des jetons d'accès est potentiellement opaque pour les applications. Le endpoint d'introspection permet au client d'obtenir des informations sur le jeton d'accès sans comprendre son format.

Token Revocation (RFC 7009): OAuth 2.0 prend en compte la manière dont les jetons d'accès sont délivrés aux applications, mais pas la manière dont ils sont révoqués.
Ceci est couvert par le endpoint de révocation de jeton.



Rappels sur les standards

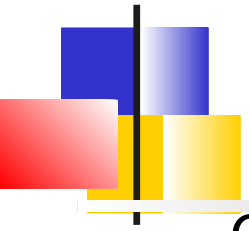
OAuth2

OpenID Connect

JWT

Transmission et protection des jetons

SAML 2.0



Introduction

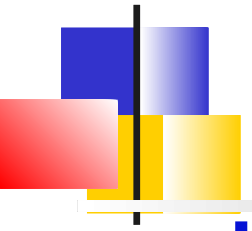
OAuth 2.0 est un protocole d'autorisation : il ne dit pas qui est l'utilisateur

OpenID Connect ajoute une couche d'authentification au-dessus d'OAuth 2.0 :

- Le scope ***openid***, qui active le comportement OpenID
- L'**ID token** : preuve de l'authentification destinée à l'application
- Des **scopes et claims standards** décrivant l'utilisateur
- **L'endpoint UserInfo**

Cas d'usage : SSO d'entreprise, login social, fédération d'identités

L'application ne voit jamais le mot de passe ; la méthode d'authentification (MFA…) relève du fournisseur



Rôles de OpenID

Utilisateur final : équivalent du détenteur de ressource d'OAuth 2.0, l'utilisateur qui s'authentifie

Relying Party (RP) : l'application qui souhaite authentifier l'utilisateur, équivalent du client OAuth 2.0

Fournisseur OpenID (OP) : le fournisseur d'identité qui authentifie l'utilisateur (Keycloak)

L'application demande l'identité de l'utilisateur à l'OP

S'appuyant sur OAuth 2.0, elle peut obtenir en même temps un jeton d'accès

Flux d’OpenID

Code d’autorisation (response_type=code)

- Le code est échangé contre ID token, access token et refresh token
- Avec PKCE : flux recommandé pour tous les clients, y compris SPA et mobiles

Hybride (response_type=code id_token…)

- ID token renvoyé avec le code dès la réponse d’autorisation, usage rare

Implicite (response_type=id_token token)

- Jetons dans l’URL, sans code : retiré d’OAuth 2.1, à proscrire (RFC 9700)

Ces flux nécessitent un navigateur et un utilisateur qui interagit

Flows sans navigateur

Client credentials : pas d'utilisateur, le client agit pour son propre compte

- Pas d'ID token : OAuth 2.0 pur, machine à machine

Device Authorization Grant (RFC 8628) : TV, objets connectés, CLI

- L'utilisateur s'authentifie sur un autre appareil en saisissant un code

CIBA : authentification déclenchée par un backend (centre d'appels…), validée sur un autre canal

- Détail en annexe

Password grant (ROPC) : l'application collecte le mot de passe, à proscrire

Scopes et claims standards

- Le scope ***openid*** est obligatoire ; les autres scopes demandent des claims :
  - ***profile*** : name, given_name, family_name, preferred_username, locale…
  - ***email*** : email, email_verified
  - ***address*** : address
  - ***phone*** : phone_number, phone_number_verified
- ***offline_access*** : demande d'un refresh token utilisable hors session
- Les claims sont livrés dans l'ID token et/ou par UserInfo
- Dans Keycloak : client scopes du même nom, avec leurs mappers

Spécifications additionnelles OpenID

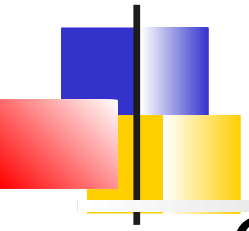
Discovery : découverte dynamique de la configuration du fournisseur

Dynamic Client Registration : enregistrement d'un client par API (Keycloak : client registration, kcreg)

Logout :

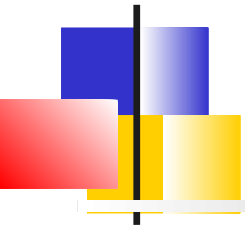
- **RP-Initiated Logout** : déconnexion initiée par l'application auprès du fournisseur
- **Back-Channel Logout** : notification de déconnexion de serveur à serveur
- **Front-Channel Logout** : notification par iframes dans le navigateur

Session Management : surveillance de la session par iframe pollant keycloak, compromise par le blocage des cookies tiers



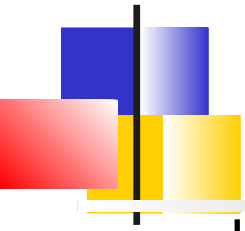
ID token et UserInfo

- OpenID Connect impose le format JWT pour l'ID token
  - Le client lit les claims après validation de la signature, de iss, aud, exp et nonce
- L'endpoint UserInfo, appelé avec l'access token, retourne des claims sur l'utilisateur
  - Configurable : pas nécessairement les mêmes informations que l'ID token
  - Pas de claims techniques (iss, aud, nonce…)



Rappels sur les standards

OAuth2
OpenID Connect
JWT
Transmission et protection des jetons
SAML 2.0



Introduction

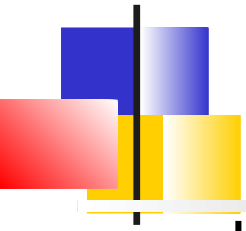
Keycloak utilise **JWT** comme format par défaut, y compris pour les jetons d'accès

Avantages :

- Format JSON facilement exploitable par les librairies
- Les serveurs de ressources valident le jeton localement, sans appel au serveur d'origine

Contreparties :

- Signé n'est pas chiffré : contenu lisible par tous, pas de données sensibles
- Non révocable avant expiration s'il est validé localement : durées de vie courtes
- Taille croissante avec les rôles et claims (en-têtes HTTP, cookies)



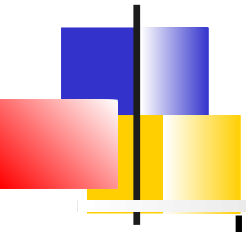
JOSE

JWT est issu de la famille de spécifications JOSE :

- **JWT** (RFC 7519) : format des revendications (claims)
- **JWS** (RFC 7515) : jeton signé, 3 parties en Base64URL : en-tête.charge utile.signature
- **JWE** (RFC 7516) : jeton chiffré, 5 parties
- **JWA** (RFC 7518) : algorithmes, asymétriques (RS256, ES256, PS256) ou à secret partagé (HS256)
- **JWK** (RFC 7517) : représentation JSON d'une clé ; JWKS : ensemble de clés

En-tête : alg (algorithme), kid (identifiant de la clé), typ (type de jeton)

Utilisation de JOSE dans OpenID Connect



- JWT + JWS : format de l'ID token, obligatoirement signé
- JWA : identifiants des algorithmes (RS256, ES256, PS256…)
- JWK / JWKS : publication des clés publiques sur jwks_uri
- JWE : chiffrement optionnel de l'ID token avec la clé publique du client. Réservé aux contextes réglementés (FAPI)
- Autres JWT signés : réponse UserInfo signée, logout token, private_key_jwt, preuve DPoP

Validation d’un JWT

Le discovery fournit l’URL ***jwks_uri*** : clés publiques du realm, mises en cache par le serveur de ressources

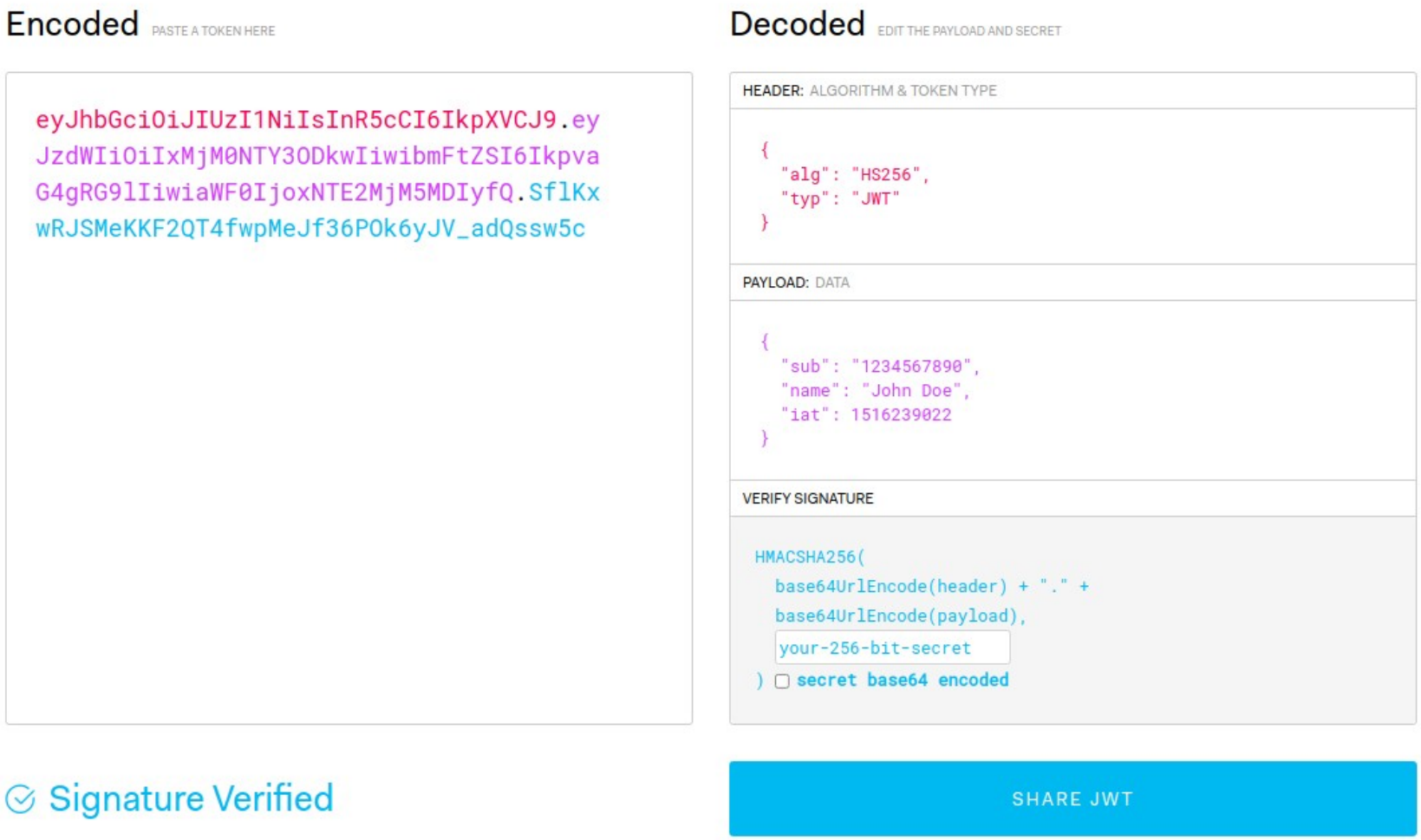
Étapes de validation :

- Algorithme dans une liste autorisée : refuser none, éviter la confusion RS256 / HS256
- Sélection de la clé par le *kid*, vérification de la signature
- Contrôle des claims : iss, aud, exp, nbf

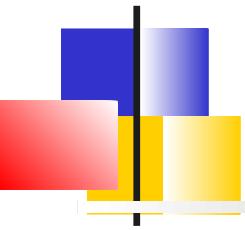
Rotation des clés : un *kid* inconnu doit déclencher le rafraîchissement du cache JWKS

En pratique : utiliser une librairie éprouvée plutôt que du code maison

jwt.io

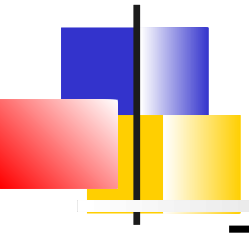


Attention : ne pas coller de jetons de production dans un site en ligne.



Rappels sur les standards

OAuth2
OpenID Connect
JWT
Transmission et protection des jetons
SAML 2.0



Introduction

Tous les jetons ne se valent pas en termes de sécurité.

Certains sont "présentables" (Bearer), d’autres nécessitent une preuve de possession (DPoP, mTLS), d’autres protège le contenu (Phantom, Split).

Keycloak prend en charge plusieurs de ces approches selon les cas d’usage.

Bearer token

Jeton auto-suffisant, envoyé dans l'en-tête HTTP :

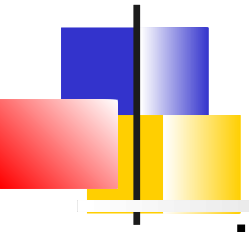
Authorization: Bearer <token>

Le serveur de ressource accepte le jeton sans preuve supplémentaire.

- Supporté nativement par Keycloak.
- Risque : toute entité possédant le jeton peut l'utiliser (token replay).

Transmission :

- Toujours dans l'entête
- Toujours en https
- Le stockage en LocalStorage expose au XSS



DPoP

Le client prouve qu'il détient une clé privée associée au jeton.

Envoie un header DPoP contenant une preuve cryptographique signée.

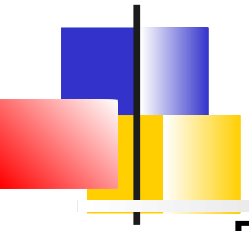
- Empêche l'usage d'un jeton volé par un tiers.
- Support dans les versions récentes de Keycloak

mTLS

Utilise une **authentification TLS mutuelle** (le client présente un certificat).

Le jeton est lié à l'empreinte du certificat.

- Très robuste, adapté aux cas réglementés (FAPI, banque).
- Supporté par Keycloak, mais nécessite :
  - Activation dans le client
  - Infrastructure compatible mTLS



Phantom token et Split token

Deux patterns d'architecture, non standardisés, mis en œuvre par une API gateway

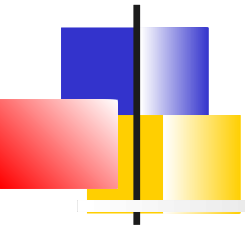
Objectif : ne pas exposer le contenu du jeton en dehors du système d'information

Phantom token : le client reçoit un jeton opaque

- La gateway l'échange contre un JWT par introspection (RFC 7662)
- Les API internes reçoivent le JWT

Split token : le client ne reçoit que la signature du JWT

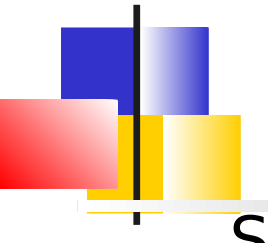
- La gateway conserve le reste du jeton et le reconstitue



Rappels sur les standards

OAuth2
OpenID Connect
JWT
Transmission et protection des jetons
SAML 2.0

SAML 2.0 : acteurs et assertion



Standard OASIS (2005) basé sur XML

Identity Provider (IdP) : authentifie l'utilisateur et émet les assertions

Service Provider (SP) : l'application qui consomme l'assertion

Assertion : document XML signé par l'IdP

- **Subject** : identifiant de l'utilisateur (NameID)
- **Attributs** : email, nom, groupes…
- **Conditions** : période de validité, audience (SP destinataire)

L'assertion joue le rôle de l'ID token d'OpenID Connect

Flux SSO initié par le SP

L’utilisateur accède à l’application (SP)

Le SP redirige le navigateur vers l’IdP avec une AuthnRequest (binding HTTP-Redirect)

L’IdP authentifie l’utilisateur ou réutilise sa session SSO

L’IdP renvoie une page qui poste la réponse au SP (binding HTTP-POST)

- Vers l’Assertion Consumer Service (ACS) du SP

Le SP vérifie signature, audience et validité, puis ouvre sa session

Tous les échanges transitent par le navigateur

Variante initiée par l’IdP (portail d’applications), Single Logout (SLO)

Confiance et métadonnées

La confiance IdP / SP repose sur l'échange de métadonnées XML

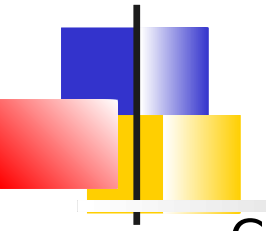
- EntityID : identifiant unique de chaque partie
- Endpoints : SSO, ACS, Single Logout
- Certificats de signature et de chiffrement

NameID : format de l'identifiant (persistent, transient, email)

Mapping des attributs entre IdP et SP

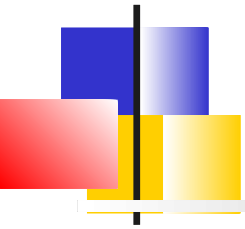
Dans Keycloak :

- IdP SAML : client de type saml, métadonnées sur /realms/{realm}/protocol/saml/descriptor
- SP : fédération d'un IdP SAML externe (ADFS, Entra ID…)



SAML 2.0 et OpenID Connect

- Concepts équivalents :
  - IdP ↔ OpenID Provider, SP ↔ Relying Party
  - Assertion ↔ ID token
  - Métadonnées ↔ discovery
  - ACS ↔ redirect_uri
  - Single Logout ↔ RP-Initiated et Back-Channel Logout
- SAML 2.0 : XML, échanges uniquement via le navigateur, pas de jeton d'accès pour appeler des API
  - À conserver pour les applications existantes et les progiciels ne supportant que SAML
- OpenID Connect : JSON et JWT, adapté aux SPA, au mobile et aux API
  - Choix recommandé pour les nouvelles applications



Keycloak et OpenID

Discovery endpoint
Authentification
Logout

Discovery Endpoint

Le point de découverte de Keycloak est accessible à :

```
https://<server>/realms/<realm-name>/.well-known/openid-configuration
```

Il permet à un client de découvrir toutes les informations de configuration utiles :

- Endpoints et URIs accessibles
- Types de grants et de réponses supportés
- Méthodes PKCE supportées (code_challenge_methods_supported)
- Mécanismes de logout supportés
- Algorithmes de signature et de chiffrement, scopes et claims supportés

Les librairies clientes s’auto-configurent à partir de ce document



Endpoints

```
"issuer":
  "https://<server>/realms/<realm-name>",

"authorization_endpoint":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/auth",

"token_endpoint":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/token",

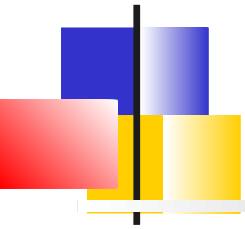
"introspection_endpoint":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/token/introspect",

"userinfo_endpoint":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/userinfo",

"end_session_endpoint":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/logout",

"jwks_uri":
  "https://<server>/realms/<realm-name>/protocol/openid-connect/certs",
```

Authentification avec OpenID

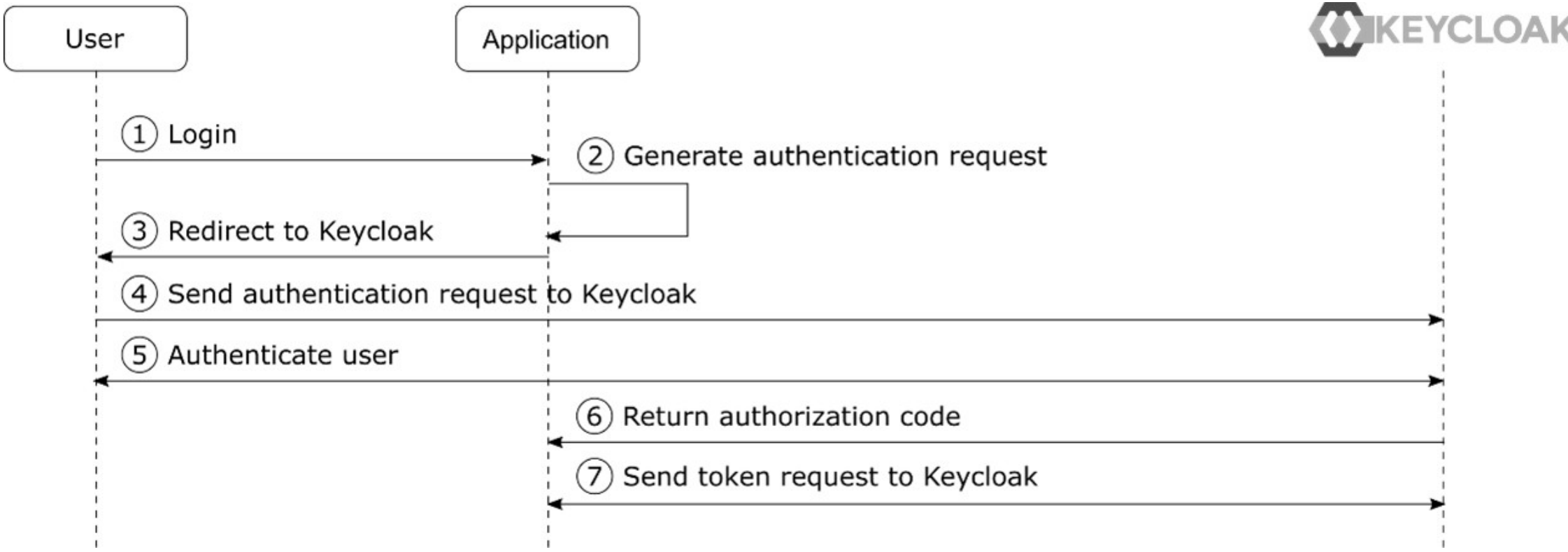


Discovery endpoint

Authentification

Logout

Flow


User
Application
KEYCLOAK
1 Login
2 Generate authentication request
3 Redirect to Keycloak
4 Send authentication request to Keycloak
5 Authenticate user
6 Return authorization code
7 Send token request to Keycloak

Requête d’autorisation

Paramètres obligatoires :

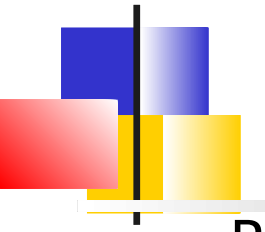
- client_id, redirect_uri (déclarée dans le client), response_type=code
- scope : doit contenir openid, d’autres scopes peuvent être ajoutés

Paramètres de sécurité :

- state : valeur aléatoire vérifiée au retour (protection CSRF)
- nonce : recopié dans l’ID token (protection contre le rejeu)
- PKCE : code_challenge et code_challenge_method=S256, obligatoire selon le RFC 9700

Paramètres optionnels : prompt (none, login, consent), max_age, login_hint, kc_idp_hint

La réponse est un code d’autorisation à usage unique, valable 1 min par défaut



Échange du code

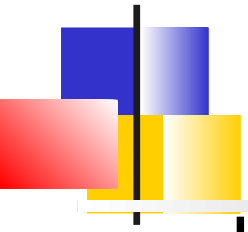
Requête POST sur le token endpoint :

- grant_type=authorization_code
- code : le code reçu
- redirect_uri : identique à celle de la requête d'autorisation (vérification)
- code_verifier : secret PKCE dont le code_challenge est dérivé

Authentification d'un client confidentiel :

- client_id et client_secret (en Basic ou dans le corps)
- Ou JWT signé (private_key_jwt), certificat mTLS

Un client public n'envoie que son client_id : PKCE est alors indispensable

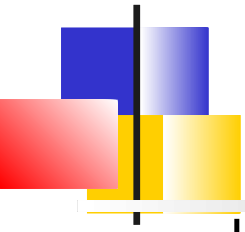


Réponse jetons

La réponse JSON contient :

- access_token et expires_in : durée de validité en secondes
- refresh_token et refresh_expires_in
- id_token : jeton d'identification
- token_type : Bearer, ou DPoP si le jeton est lié à une clé du client
- scope : scopes effectivement accordés, ils peuvent différer de la demande
- session_state : identifiant de la session Keycloak

Rafraîchissement



L’access token a une courte durée de vie pour limiter l’impact d’une fuite

- L’application n’a pas à ré-authentifier l’utilisateur

Le refresh token, de durée plus longue, permet d’obtenir de nouveaux jetons :

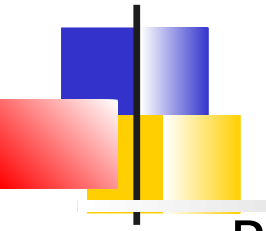
- grant_type=refresh_token sur le token endpoint
- Nouvel access token, nouvel ID token et nouveau refresh token

Par défaut, l’ancien refresh token reste réutilisable

- Option « Revoke Refresh Token » du realm pour un usage unique

Sa validité est bornée par la session SSO (SSO Session Idle et Max)

Offline token (scope offline_access) : survit à la fin de la session SSO



Jeton d’identification

Preuve de l’authentification, destinée au client (l’application)

- Ne doit pas servir à appeler une API : c’est le rôle de l’access token

JWT signé par la clé du realm :

- <En-tête>.<Charge utile>.<Signature>, encodés en Base64URL

Le client doit le valider :

- Signature, avec les clés publiées sur jwks_uri
- iss, aud, exp
- nonce identique à celui de la requête

Revendications

iss : émetteur, URL du realm

sub : identifiant unique de l'utilisateur

aud : client_id de l'application ; **azp** : partie à laquelle le jeton a été émis

exp, **iat**, **auth_time** : expiration, émission, dernière authentification

nonce : valeur transmise dans la requête d'autorisation

acr : niveau d'authentification ; **sid** : identifiant de session

at_hash : empreinte de l'access token

Les autres claims décrivent l'utilisateur et se configurent via les client scopes et mappers



UserInfo endpoint

Retourne les informations de l'utilisateur authentifié

```
GET https://<server>/realms/<realm-name>/protocol/openid-connect/userinfo
Authorization: Bearer <access_token>
```

L'access token doit avoir été obtenu avec le scope openid

Configuration :

- Option « Add to userinfo » de chaque mapper
- Client → Advanced → User info signed response algorithm : réponse JWT signée

Intérêt : garder un ID token léger et obtenir des informations à jour



User Profile

Depuis la version 24, le profil utilisateur est déclaratif et toujours activé

Realm settings → User profile

- Liste des attributs gérés et groupes d’attributs
- Éditeur de formulaire ou JSON

Un même schéma pilote :

- La console d’administration et la console du compte
- Les pages d’inscription et de mise à jour du profil
- L’API d’administration

Attributs non déclarés : Unmanaged attributes (Disabled, Enabled, Admin can view, Admin can edit)



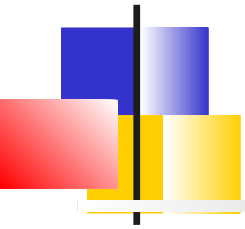
Définition d’un attribut

Propriétés d’un attribut :

- Nom, libellé (internationalisable), mono ou multi-valué
- Obligatoire pour les utilisateurs et/ou les admins, éventuellement selon les scopes demandés
- Permissions de lecture et d’écriture : user, admin
- Validations : length, email, pattern, options, integer, uri…
- Annotations : type de champ du formulaire (select, textarea…)

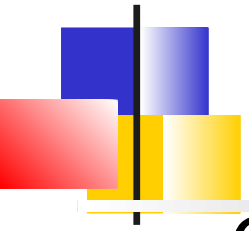
Exposition dans les jetons :

- Mapper « User Attribute » dans un client scope dédié
- Nom du claim et jetons cibles : ID token, access token, UserInfo



Authentification avec OpenID

Discovery endpoint
Authentification
Logout



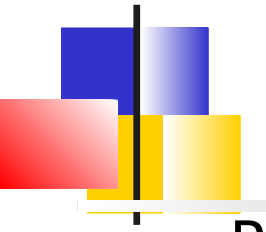
Introduction

- Gérer la déconnexion dans une expérience SSO est délicat, surtout si l’on veut une déconnexion instantanée de toutes les applications
- Un logout peut être initié par l’expiration de la session ou par une action de l’utilisateur
- Selon le type de client, la déconnexion est répercutée de façon différente
- La déconnexion doit aussi rendre inutilisables les jetons de l’utilisateur

Mécanisme du logout

- L’utilisateur clique sur Déconnexion dans une application
- L’application nettoie son contexte et redirige vers l’end_session_endpoint avec :
  - id_token_hint ou client_id : identification du client
  - post_logout_redirect_uri : URI de retour, déclarée dans le client
  - state, ui_locales (langue de l’écran de déconnexion)
- Keycloak ferme la session SSO : les refresh tokens sont invalidés
  - Un access token JWT reste valide jusqu’à son expiration pour une API qui le valide localement (sauf introspection)
- Si configuré, Keycloak notifie les autres clients (front ou back-channel)
- L’utilisateur est redirigé vers post_logout_redirect_uri

Découverte de la déconnexion



Plusieurs mécanismes peuvent être mis en œuvre, en fonction du type de client et du niveau de sécurité attendu :

- Expiration naturelle ou échec de rafraîchissement des tokens
- OpenID Connect Session Management
- OIDC Back-Channel Logout
- OIDC Front-Channel Logout

Les méthodes 2 à 4 permettent une propagation active de la déconnexion SSO à plusieurs applications.

Détection par expiration

L’application se rend compte de la déconnexion quand un token n’est plus utilisable.

- Les tokens d’accès et ID tokens ont une durée de vie courte (ex. 5 min).

L’application essaie de rafraîchir le token via le refresh_token.

Si ce dernier est révoqué ou expiré, l’utilisateur doit se reconnecter.

Avantages : simple, ne nécessite aucune configuration spéciale.

Limite : la déconnexion n’est pas immédiate pour les autres clients.

OIDC Session Management

L’application surveille l’état de la session SSO via une iframe

- Iframe chargeant /realms/{realm}/protocol/openid-connect/login-status-iframe.html
- Polling régulier, logout local si la session est fermée

Avantage : pas de traitement côté serveur

Limite majeure : l’iframe est cross-site et dépend des cookies tiers

- Bloqués par Safari et Firefox, de plus en plus restreints par Chrome
- keycloak-js désactive cette vérification quand les cookies tiers sont indisponibles

Mécanisme à considérer comme legacy

OIDC Back-Channel Logout

Keycloak notifie directement le backend des applications

- Chaque client déclare une Backchannel logout URL
- Keycloak envoie un POST contenant un logout token (JWT signé)
- L’application valide le jeton et invalide la session correspondante (sid ou sub)

Option « Backchannel logout session required » : inclut le sid

Mécanisme recommandé : fiable, indépendant du navigateur, fonctionne même navigateur fermé

Limites :

- Keycloak doit pouvoir joindre le backend (réseau, Docker, proxy)
- L’application doit retrouver sa session à partir du sid



OIDC Front-Channel Logout

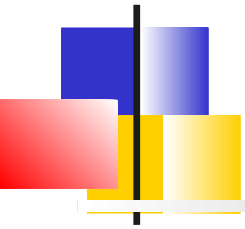
Propagation du logout via des iframes client dans la page de logout Keycloak.

- Chaque client enregistre une URI de logout front-channel.
- Lors du logout, Keycloak affiche une page contenant une iframe par client.
- Ces iframes appellent la page de logout du client pour qu'il nettoie la session.

Avantages : pas de backend requis.

Limites :

- Iframes cross-site : dépend des cookies tiers, de plus en plus bloqués par les navigateurs
- Fragile, nécessite une configuration soignée



Autoriser les accès avec oAuth2

Jeton d’accès et consentement

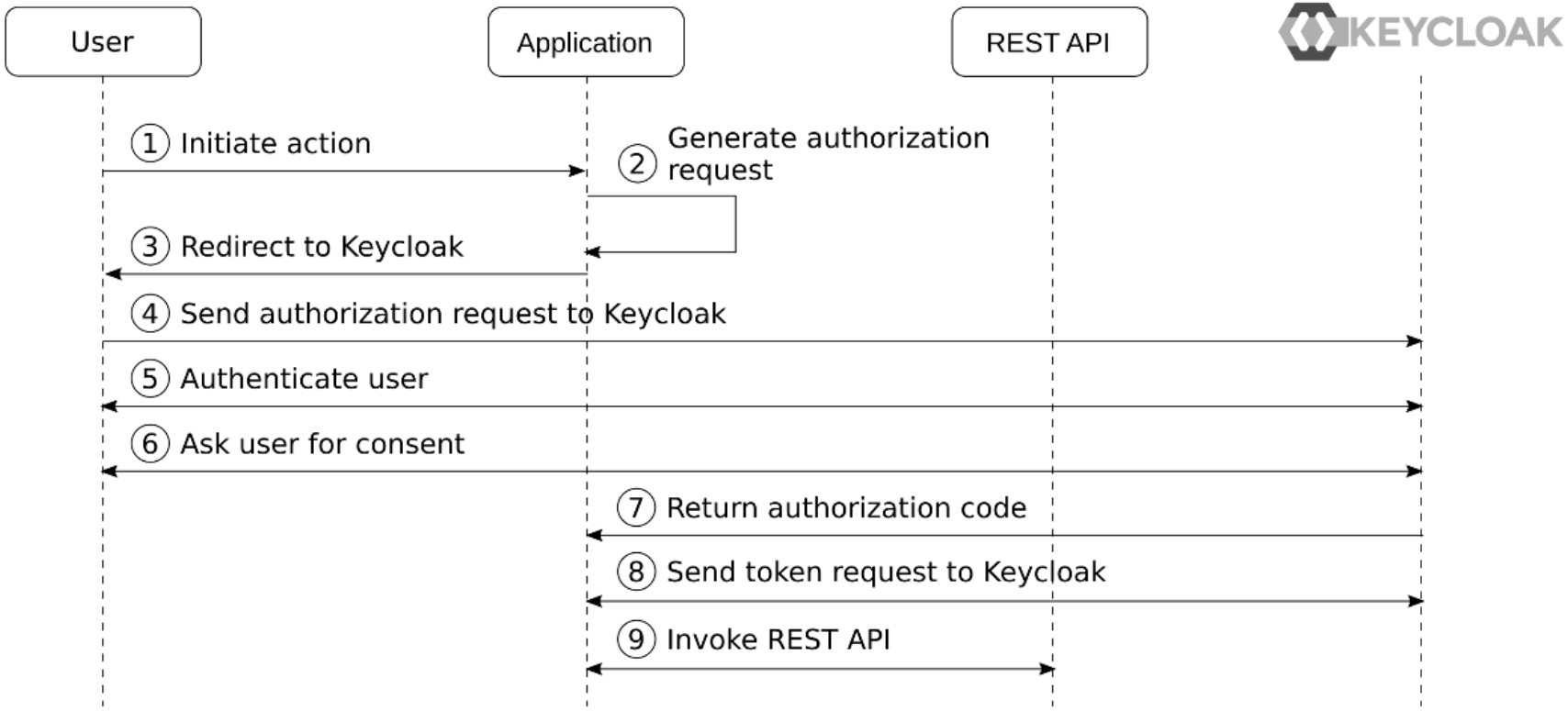
Limitations des accès

Personnalisation du jeton

Validation du jeton

Obtenir un jeton d’accès

L’***Authorization-flow*** est le moyen le plus courant d'obtenir un jeton d'accès. Ce flux inclut normalement le consentement de l’utilisateur



Contenu du jeton d’accès

Les attributs les plus importants du jeton d’accès :

- ***aud*** (audience) : Liste de services auxquels ce jeton est destiné à être envoyé.
- ***realm_access*** : Liste de rôles realm auxquels le jeton donne accès :
  intersection des rôles attribués à un utilisateur et des rôles autorisés auxquels une application est autorisée d’accéder.
- ***resource_access*** : Liste de rôles client auxquels le jeton donne accès.
- ***scope*** : Scope inclus dans le jeton d'accès.

Consentement de l’utilisateur

Avec Keycloak, les applications peuvent être configurées pour exiger ou ne pas exiger le consentement de l’utilisateur.

Le type de privilèges d'accès demandé par l'application est contrôlé par les scopes demandés par l'application.

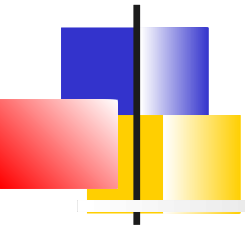
- Les scopes peuvent être demandés *progressivement* par l’application

Les consentements de l’utilisateur sont conservés par Keycloak. Un utilisateur peut supprimer ses consentements en accédant à Keycloak

Application interne / externe

Pour une application interne, il n'est pas nécessaire d’obtenir le consentement de l'utilisateur

- Cette application « corporate » de confiance doit être accessibles pour les utilisateurs de l’entreprise
- => C’est la valeur par défaut car Keycloak est dédié à l’entreprise



Autoriser les accès avec oAuth2

Jeton d’accès et consentement

Limitations des accès

Personnalisation du jeton

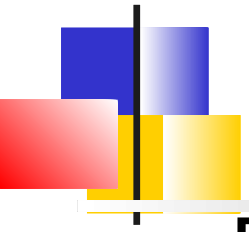
Validation du jeton

Limiter les accès du jeton

Les accès du jeton peuvent être limités selon 3 axes principaux :

- **Audience** : permet de lister les fournisseurs de ressources qui doivent accepter un jeton d'accès.
- **Rôles** : Les ACLs sur les ressources sont exprimées en fonction des rôles utilisateur.
- **Scopes**[1] : Les ACLs sur les ressources sont exprimées en fonction des scopes clients.

1. Mécanismes par défaut de oAuth2, les scopes oAuth2 sont mappés sur les client scopes.



Configuration des audiences

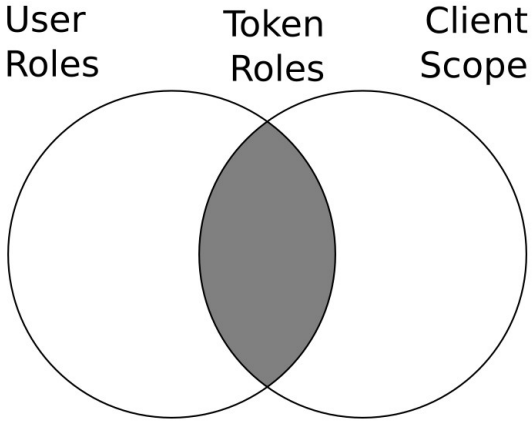
- Dans Keycloak, il existe 2 manières différentes d'inclure un client dans l'audience.
  - Manuellement en ajoutant un client spécifique à l'audience à l’aide d’un mapper.
    On peut par exemple utiliser le scope client dédié au client toujours présent dans le jeton
  - Automatiquement, si le client a une scope sur un rôle client d'un autre client.

Configuration des rôles

Un utilisateur est affecté à un certain nombre de rôles.

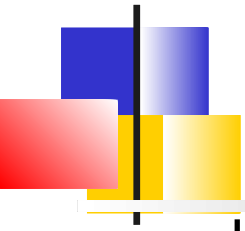
Un client, n'a pas de rôles directement assignés, mais a un *scope* sur un ensemble de rôles, qui contrôle quels rôles peuvent être inclus dans le jetons envoyés au client

=> les rôles inclus dans les jetons sont l'intersection entre les rôles d'un utilisateur et les rôles qu'un client est autorisé à utiliser



Configuration des scopes du client

- ✔ Un client a une *scope* sur les rôles. Les rôles autorisés pour ce client. Par défaut « Full scope » :
  Ceci est configuré via l'onglet
  Client → Client Scopes → Client Dedicated → Scope du client.
- ✔ Un client peut accéder à un ou plusieurs scopes client.
  Ceci est configuré via l'onglet ***Client → Client Scopes*** du client.
  - ✔ Les client Scopes sont par défaut ou optionnel
- ✔ Un « scope client » peut également avoir un scope sur les rôles.
  Lorsqu'un client a accès à ce « scope client », il obtient le scope sur les rôles correspondant.
  Client Scopes → Scope du client scope.



ACLs sur les scopes

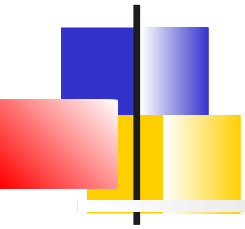
Le mécanisme par défaut dans OAuth 2.0 pour limiter les autorisations pour un jeton d'accès passe directement par les scopes.

Dans Keycloak, un scope de OAuth 2.0 est mappé à une scope client.

- Si l’on souhaite uniquement disposer d'une scope pour fixer des ACLs sur un serveur de ressource, il suffit de définir un client scope avec aucun mapper ni aucun rôle associé .

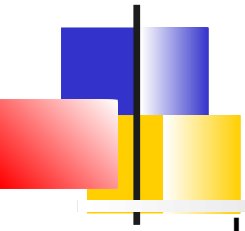
Ce type de scopes sont généralement dédiés à une application de l’entreprise. Leur nom peut être préfixé par le nom ou l’URL du service

- albums:view, https://api.acme.org/api/albums.view, https://www.googleapis.com/auth/calendar.events, ...



Autoriser les accès avec oAuth2

Jeton d’accès et consentement
Limitations des accès
Personnalisation du jeton
Validation du jeton



Introduction

Les différents clients d'une application n'ont pas nécessairement besoin des mêmes informations dans les jetons.

Dans OAuth2 / OIDC, les *scopes* permettent de définir les droits ou informations qu'un client peut demander (ex. openid, email, profile).

Keycloak implémente ce mécanisme via 2 composants clés :

- ***Client Scopes*** : ensembles configurables regroupant des paramètres (notamment des mappers) associés à un ou plusieurs scopes OAuth2/OIDC.
  Ils peuvent être automatiquement attachés ou activés explicitement lors de l'authentification.
- ***Protocol Mappers*** : définissent les informations (claims) ajoutées dans les tokens.

En combinant client scopes et mappers, Keycloak adapte dynamiquement les jetons en fonction des scopes OAuth2 demandés.

Protocol Mappers

Les ***Mappers*** permettent de personnaliser les revendications des jetons fournies à un client:

- Informations statiques,
- Attributs d'utilisateur
- Rôles.

Ils s'appliquent aux tokens ID, tokens d'accès, et à l'endpoint *userinfo* selon la configuration

Keycloak fournit des mappers prédéfinis.

Les mappers peuvent être définis au niveau d'un client ou d'un client scope

Client Scopes

Les ***client scopes*** permettent de définir des configurations client qui pourront être mutualisées entre différents clients.

Cela englobe principalement

- Les mappers : Résolution des claims
- Le *Scope* ou *Role Scope mapping* : les rôles de l'utilisateur auxquels le client a accès

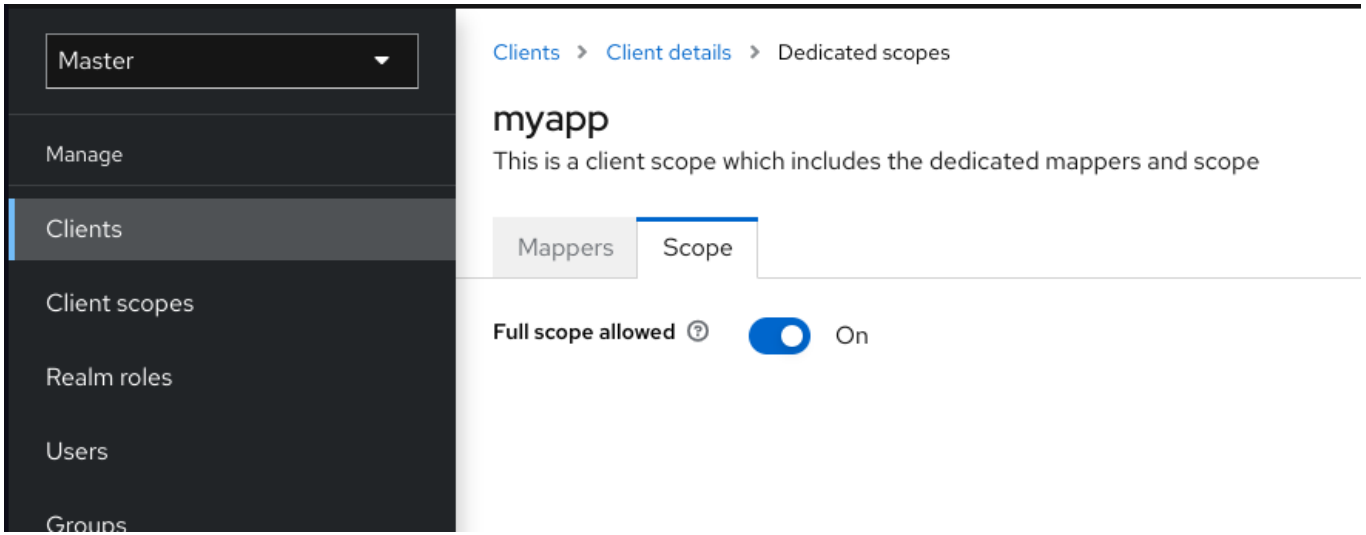
Les *client scopes* prennent en charge le paramètre *scope* d'OAuth 2.
C'est donc le moyen d'adapter les revendications et en particulier les rôles en fonction des exigences de l'application.

Role scope mapping

Role scope Mapping permet de limiter les rôles indiqués dans un jeton d'accès.

Le jeton d'accès reçu par un client ne contient que les rôles explicitement spécifiés pour le scope du client.

Si non défini, chaque client obtient tous les rôles de l'utilisateur.



Client Scopes

Keycloak propose des client scopes prédéfinis pour OpenID :

- ***roles*** : Pas défini par OpenID Connect et pas ajoutée automatiquement.
  Il comporte des mappeurs utilisés pour ajouter les rôles de l'utilisateur au jeton d'accès et ajouter des audiences pour les clients qui ont au moins un rôle client.
- ***web-origins*** : Pas défini dans OpenID Connect et pas ajoutée automatiquement.
  Utilisé pour ajouter des origines Web autorisées dans le jeton d'accès. (allowed-origins)
- ***microprofile-jwt*** : Gère les revendications définies dans la spécification MicroProfile/JWT.
- ***offline_access*** : Spécification OpenID Connect. Permet d'obtenir un *offline refresh* token. Permet aux applis de rester inactive longtemps
- ***profile***, ***email***, ***address***, ***phone*** : Défini dans OpenID Connect : des mappers mais pas de rôle mapping

Liens entre Client et Client Scopes

L’association entre client et Client scopes s’effectue dans l'onglet

Client → Client Scopes

2 types de liaisons :

- ***Default Client Scopes*** : Le client hérite des mappeurs et des mapping de rôles dans tous les cas.
- ***Optional Client Scopes*** : Le client hérite des mappeurs et des mapping de rôles seulement si le paramètre scope de la requête OpenID a mentionné ce scope.

De plus, chaque client a un client-scope dédié automatiquement appliqué quelque soit le scope oauth2 demandé.

Settings | Keys | Credentials | Roles | Client scopes | Sessions | Advanced

Setup | Evaluate

Name | Search by name | Add client scope | Change type to | Refresh

1 - 10

Assigned client scope	Assigned type	Description
ciba-client-dedicated	none	Dedicated scope and mappers for this client
acr	Default	OpenID Connect scope for add acr (authentication context class reference) to the token
address	Optional	OpenID Connect built-in scope: address
email	Default	OpenID Connect built-in scope: email
microprofile-jwt	Optional	Microprofile - JWT built-in scope
offline_access	Optional	OpenID Connect built-in scope: offline_access
phone	Optional	OpenID Connect built-in scope: phone

Évaluer les client scopes

Il est possible d'évaluer les mappeurs effectifs et les mapping de rôle qui seront générés.

Client Scopes → Evaluate

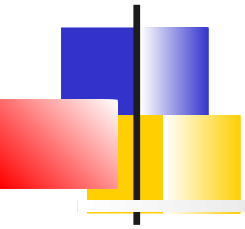
Choisir ensuite les scopes optionnels éventuels



Permissions des client scopes

A l’émission d’un jeton, les client scopes ne sont appliqués que si l’utilisateur y est autorisé.

- Lorsqu’aucun rôle mapping n’est défini, Tous les utilisateurs peuvent utiliser tous les scopes.
- Quand un client scope contient des mappings de rôles, seuls les utilisateurs possédant au moins un de ces rôles peuvent utiliser ce scope.
  => Il doit y avoir une intersection entre les rôles d'utilisateur et les rôles du scope client.



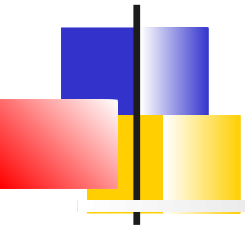
Autoriser les accès avec oAuth2

Jeton d’accès et consentement
Limitations des accès
Personnalisation du jeton
Validation du jeton



Validation du jeton

- 2 choix pour valider un jeton d'accès :
  - Appeler le endpoint d'introspection
    Respect pur du standard oAuth2 (les jetons sont supposés opaques)
    Mais ajoute une requête supplémentaire
  - Vérifier directement le jeton grâce à JWT
    JWT permet de parser les informations directement
    La signature permet de s’assurer de l’authenticité du jeton via une clé publique



Sécurisation des différents types d’application

Application web

Application native ou mobile

REST APIs et services

Types d’application web

La sécurisation d’une application web dépend de l'architecture de l'application, on peut distinguer :

- Traditionnelle : L'application Web s'exécute entièrement à l'intérieur d'un serveur « backend ».
- SPA avec API dédiée : L'application s'exécute dans le navigateur et appelle uniquement une API dédiée sous le même domaine.
- SPA avec API intermédiaire : L'application SPA appelle des API externes via une API intermédiaire hébergée sous le même domaine
- SPA avec API externe : L'application SPA appelle des API hébergées sous un domaine différent[1].

Quelque soit l’architecture le meilleur mécanisme d’authentification d’un utilisateur reste l’*Authorization Code Flow*

1. Ex : SPA déployé sous node.js et backend déployé sous un autre serveur

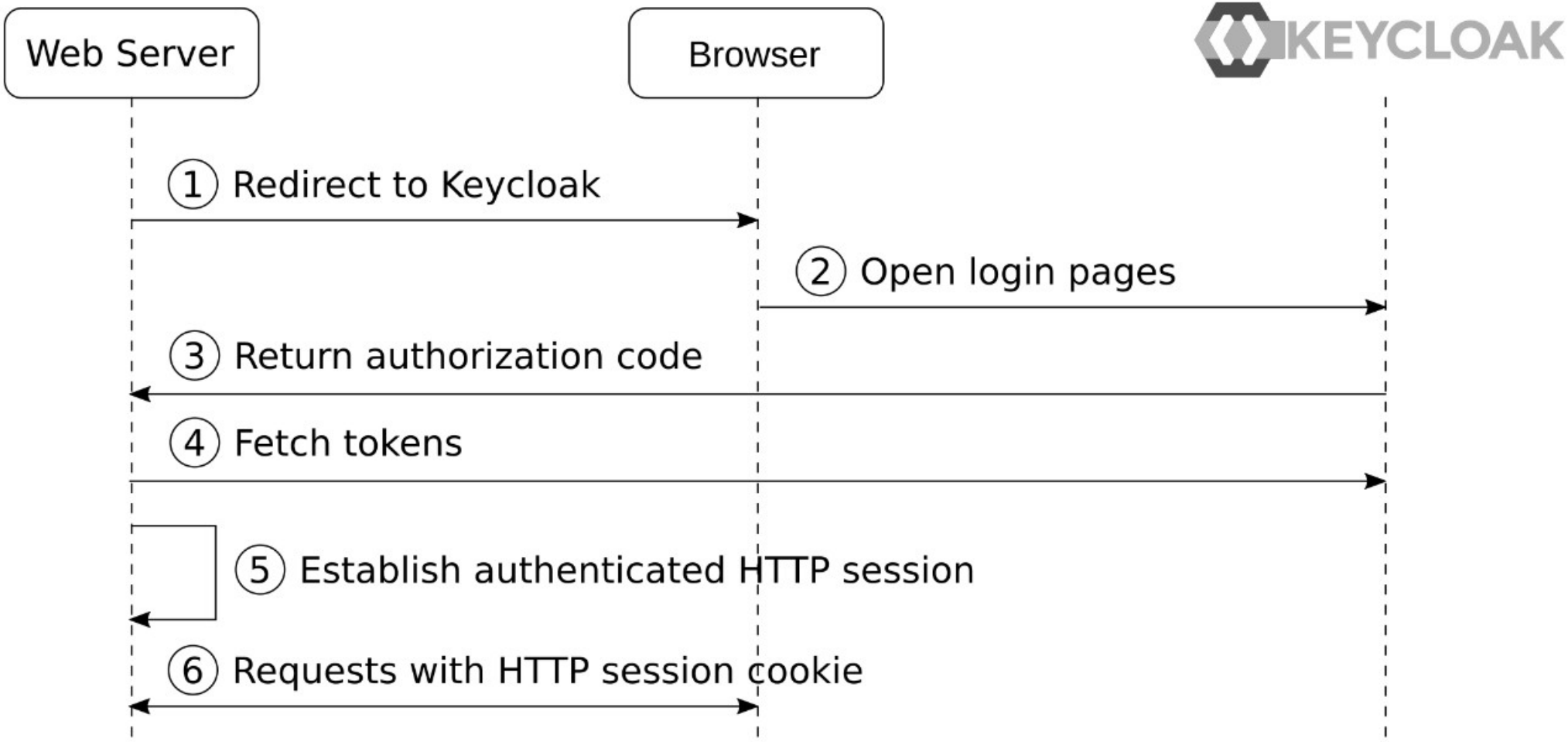
Application traditionnelle

Avec ce type d’application, seul le jeton d'identification est utilisé pour établir une session HTTP, il contient les informations nécessaires pour les ACLs de l’application.

Recommandations :

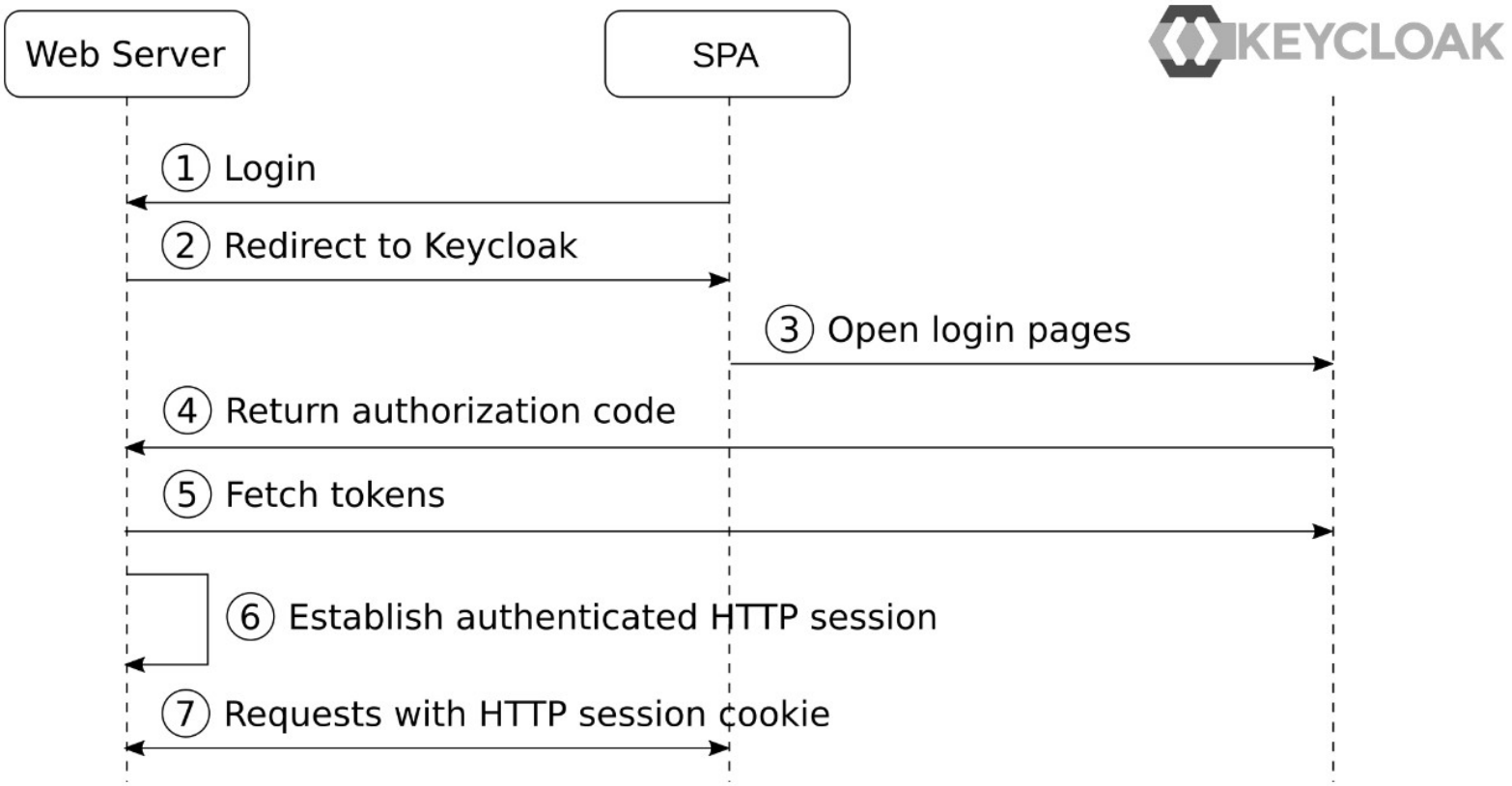
- Enregistrer un client confidentiel
  Pour obtenir le jeton, il faut alors le code + le secret du client stocké dans le back-end
- On configure généralement *PKCE* pour plus de sécurité
- Donner des URLs de redirection strictes

Application traditionnelle

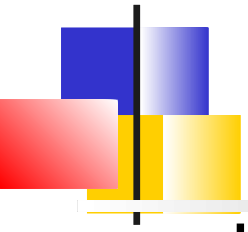


SPA avec API dédiée

Identique au cas d'une appli web traditionnelle. Le secret est conservé côté backend



SPA avec API intermédiaire dédiée



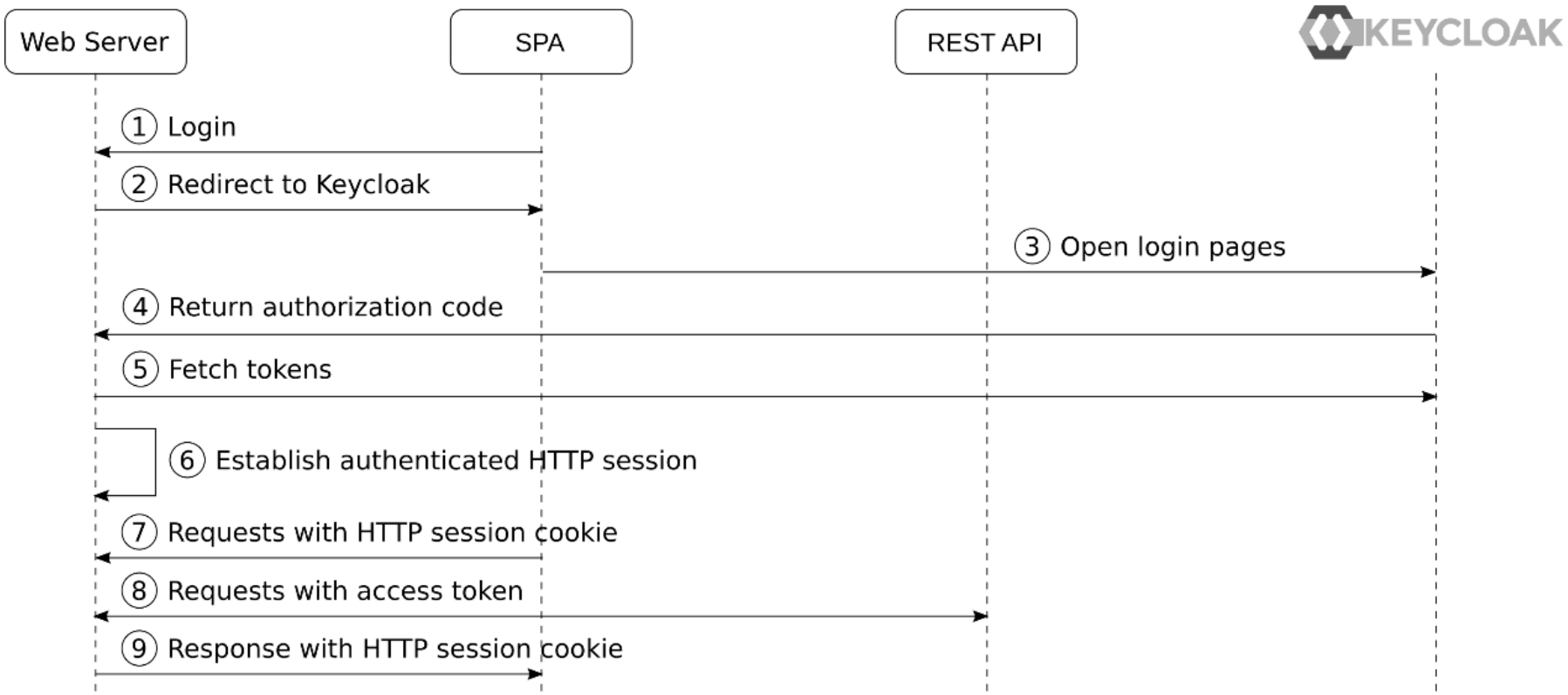
Le moyen le plus sûr d'invoquer des API externes à partir d'un SPA consiste à utiliser une API intermédiaire hébergée sur le même domaine que le SPA[1].

- Possibilité d’utiliser un client confidentiel et les jetons ne sont pas directement accessibles dans le navigateur.
- Pas besoin de mettre en place du CORS

Les appels vers les APIs externes utilisent le jeton d’accès

1. Design patterns : Gateway, Backend for frontend

SPA avec API intermédiaire dédiée



SPA avec API externe

Le moyen le plus simple est d'effectuer le flux *authorization-code* depuis le SPA avec un client public enregistré dans Keycloak.

- L'approche est moins sécurisée car les jetons, y compris le jeton d'actualisation, sont exposés directement au navigateur.

=> Avoir une courte expiration pour le jeton d'actualisation

=> Révoquer les jetons d'actualisation après leur 1ère utilisation.

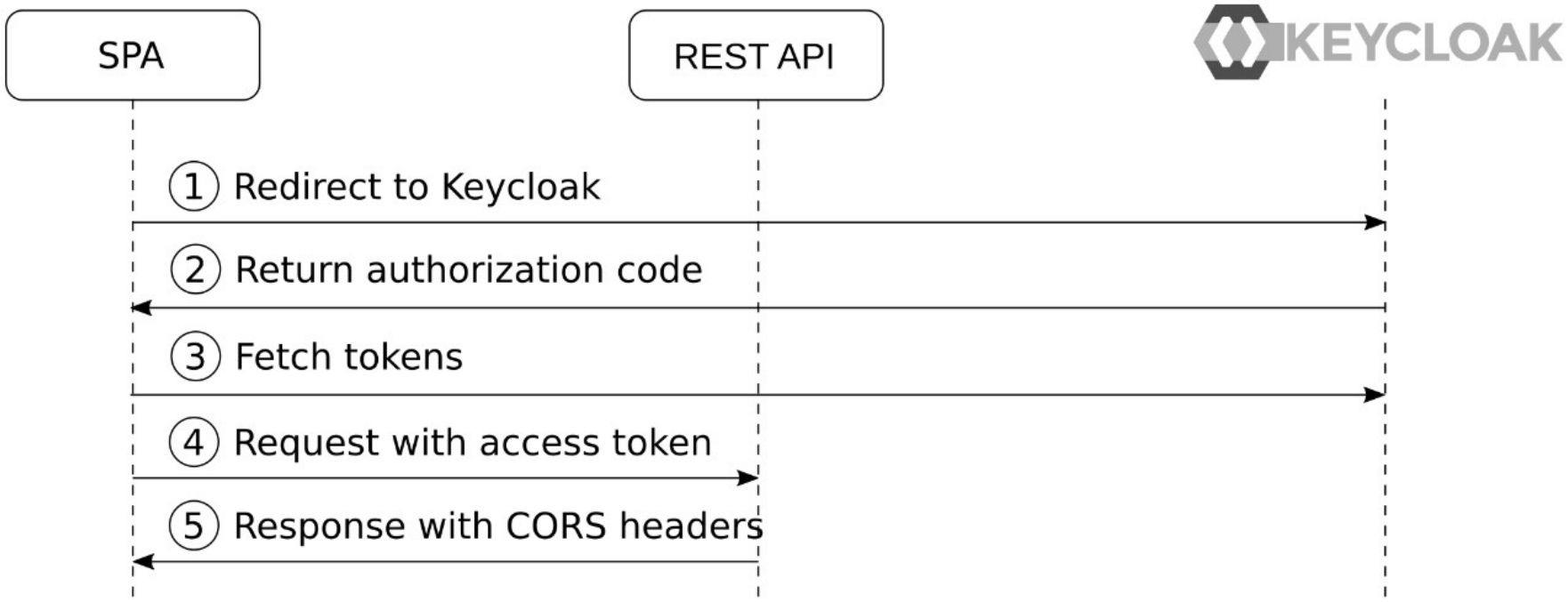
=> Utiliser l'extension PKCE

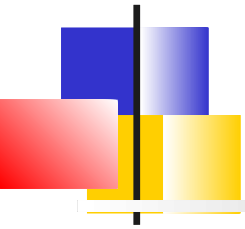
=> Stocker les jetons dans l'état de la fenêtre ou la session de stockage HTML5

=> Protéger le SPA contre le Cross Script Scripting (XSS)[1]

1. Injection de script malicieux

SPA avec API externe



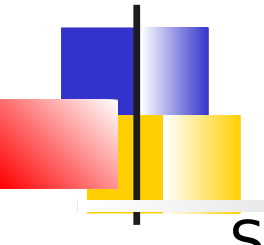


Sécurisation des différents types d’application

Application web

Application native ou mobile

REST APIs et services



Introduction

Sécuriser une application web avec Keycloak est plus simple que de sécuriser une application native ou mobile.

- Les pages de connexion étant affichées dans le navigateur.

On pourrait être tenté d'implémenter une page de connexion dans l'application et utiliser le *password grant flow*, mais mauvaise idée :

- L 'application ne doit pas avoir accès aux mots de passe
- Certaines fonctionnalités de keycloak ne seraient plus disponibles comme le MFA, le SSO ou la gestion de session

=> Le flow recommandé est donc *Authorization Code flow* avec l'extension PKCE

Authentification

Dans une application mobile ou native, l'utilisateur doit toujours s'authentifier via un navigateur :

- Utilisez une vue Web intégrée dans l'application (à éviter)
  - Facile à intégrer, mais peu sécurisée
  - Risque d'interception des credentials
  - Pas de SSO possible (cookies non partagés entre apps)
- Navigateur embarqué sécurisé (in-app browser)
  - Exemple : Custom Tabs (Android), SFSafariViewController (iOS)
  - Bénéficie du SSO du navigateur système
  - Recommandé par les bonnes pratiques Oauth/OIDC
- Navigateur externe (user-agent système[1]) :
  - Très sûr (authentification hors de l'app)
  - Mais moins fluide : bascule visible vers le navigateur

1. Navigateur par défaut du système

Authorization Code avec PKCE

Authorization Code avec PKCE est aujourd'hui le standard recommandé pour les applications mobiles ou desktop.

1) L'utilisateur clique sur "Se connecter" dans l'application.
2) L'application ouvre un navigateur système ou sécurisé (ex : Custom Tabs sur Android).
3) Keycloak affiche la page de connexion.
4) L'utilisateur s'authentifie.
5) Keycloak redirige vers l'application via un mécanisme compatible (URI personnalisée, loopback...).
6) L'application reçoit un code d'autorisation.
7) Elle échange ce code contre des tokens, en ajoutant un code PKCE pour sécuriser la requête.
8) Keycloak valide la requête et renvoie les tokens.

Renvoi du code d’autorisation

Pour renvoyer le code d'autorisation à l'application, 4 approches basées sur des URI de redirection définies par OAuth 2.0 :

- **Claimed HTTPS scheme** : App Links (Android) / Universal Links (iOS) : l’app revendique un domaine HTTPS et reçoit la redirection automatiquement via l’OS. => L’OS ouvre l'URI dans l'application au lieu du navigateur système.
- **Schéma d'URI personnalisé** : Un schéma d'URI personnalisé est enregistré avec l'application. Lorsque Keycloak redirige vers ce schéma d'URI, la requête est envoyée à l'application.
  Le schéma d'URI correspond à l'inverse d'un domaine du fournisseur de l'application. Par exemple :
  org.acme.app://oauth2/provider-name
  Plus simple à configurer mais moins sécurisé
- **Interface de loopback** : l'application ouvre un serveur Web temporaire sur l'interface de loopback, l'URI de redirection enregistré est
  http://127.0.0.1/oauth2/provider-name
  Recommandé pour les applications desktop
- **URI de redirection spéciale**: En utilisant *urn:ietf:wg:oauth:2.0:oob*, le code d’autorisation est affiché par Keycloak, l’utilisateur peut alors le copier/coller dans l’application.
  Déconseillé : méthode obsolète et non recommandée par les spécifications récentes d’OAuth.

Device Code Flow

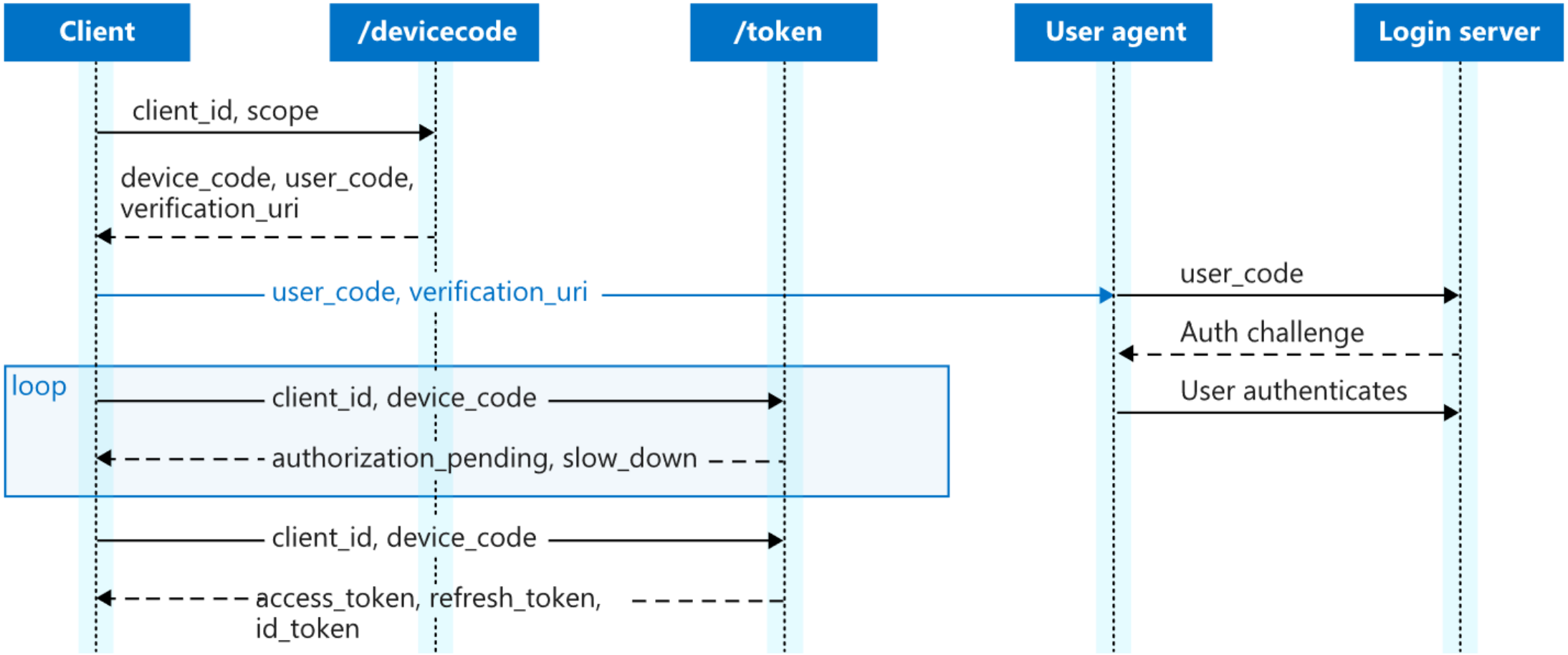
Le ***Device Code Flow*** permet à une application sans navigateur intégré (ex. Smart TV, CLI) d'obtenir des jetons via un second appareil avec navigateur.

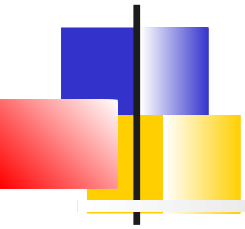
1) L'application effectue un POST sur le *device_authorization_endpoint*.
   Keycloak répond avec un ***code utilisateur*** et un ***code device.***
2) L'utilisateur saisit le code utilisateur sur un endpoint de keycloak (*verification_uri*) en utilisant un autre device avec navigateur
3) Après avoir entré le code, l'utilisateur s'authentifie et consent
4) L'application est ensuite capable de récupérer les jetons en utilisant son device code

Ce flow n'utilise pas de redirection et ne supporte pas le SSO.

TP 4.1

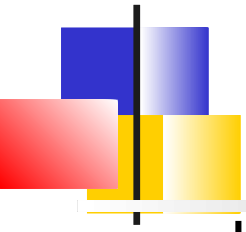
Device Code Flow





Sécurisation des différents types d’application

Application web
Application native ou mobile
REST APIs et services



Introduction

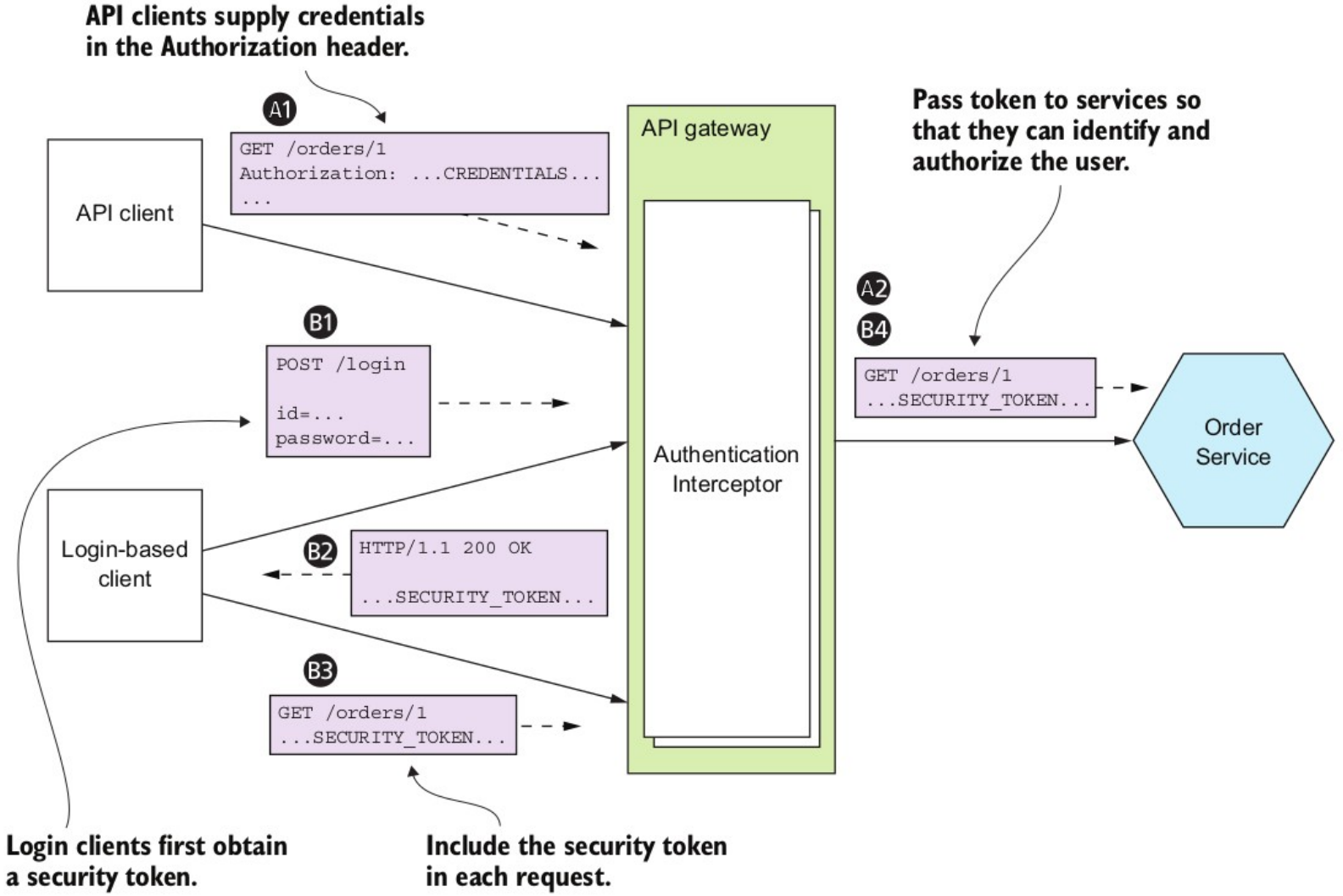
Lorsqu'une application souhaite invoquer une API protégée par oAuth2, elle obtient d'abord un jeton d'accès de Keycloak, puis inclut le jeton d'accès dans l'en-tête d'autorisation.

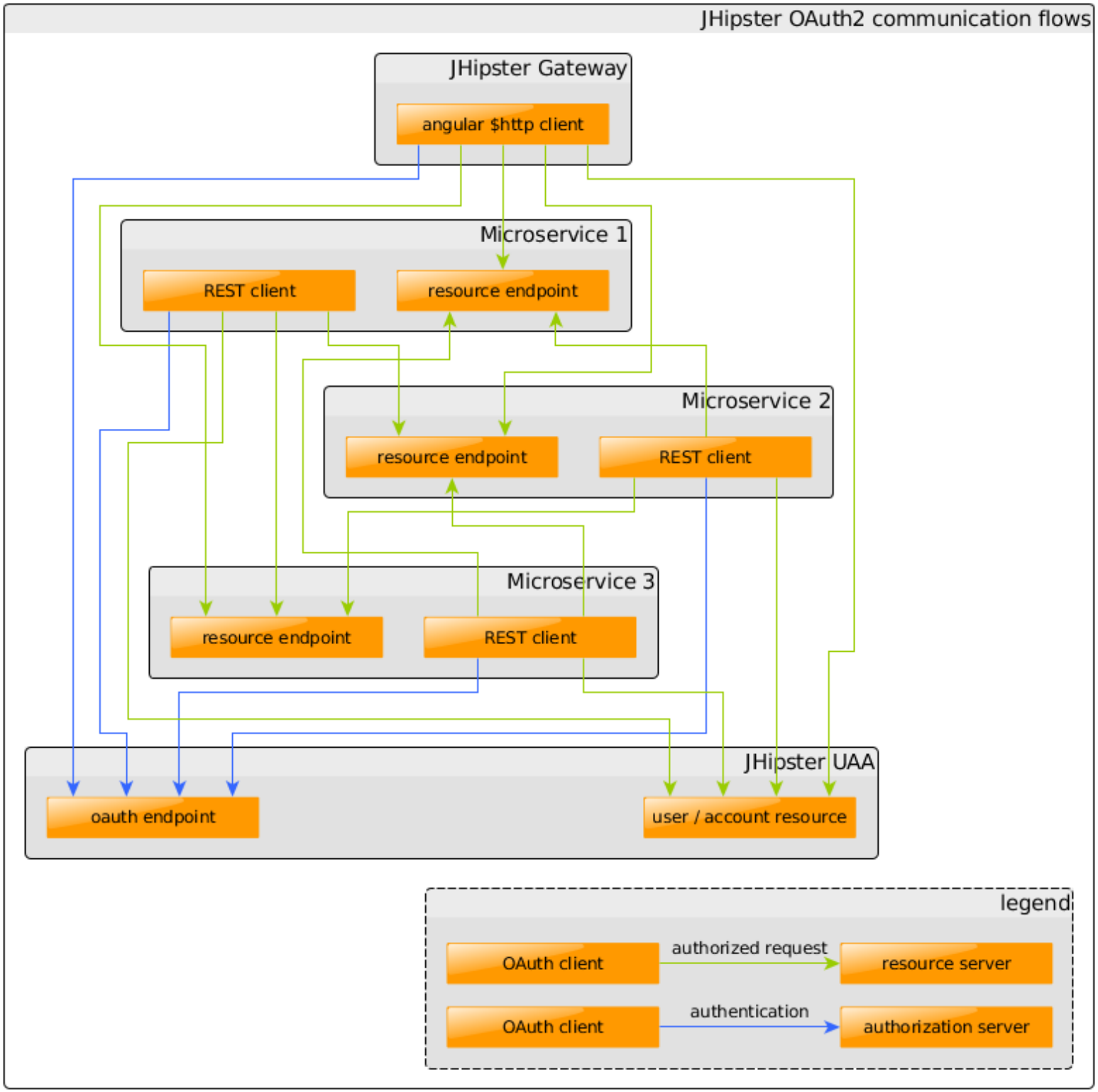
Lors des architecture micro-services, 2 alternatives sont possibles :

- Propagation de jeton, le même jeton est propagé lors des appels inter-services. Tous les micro-services partagent le même contexte de sécurité
- Chaque micro-service utilise son propre jeton obtenu avec un *Client Credentials grant*. Chaque interaction a alors son propre contexte de sécurité

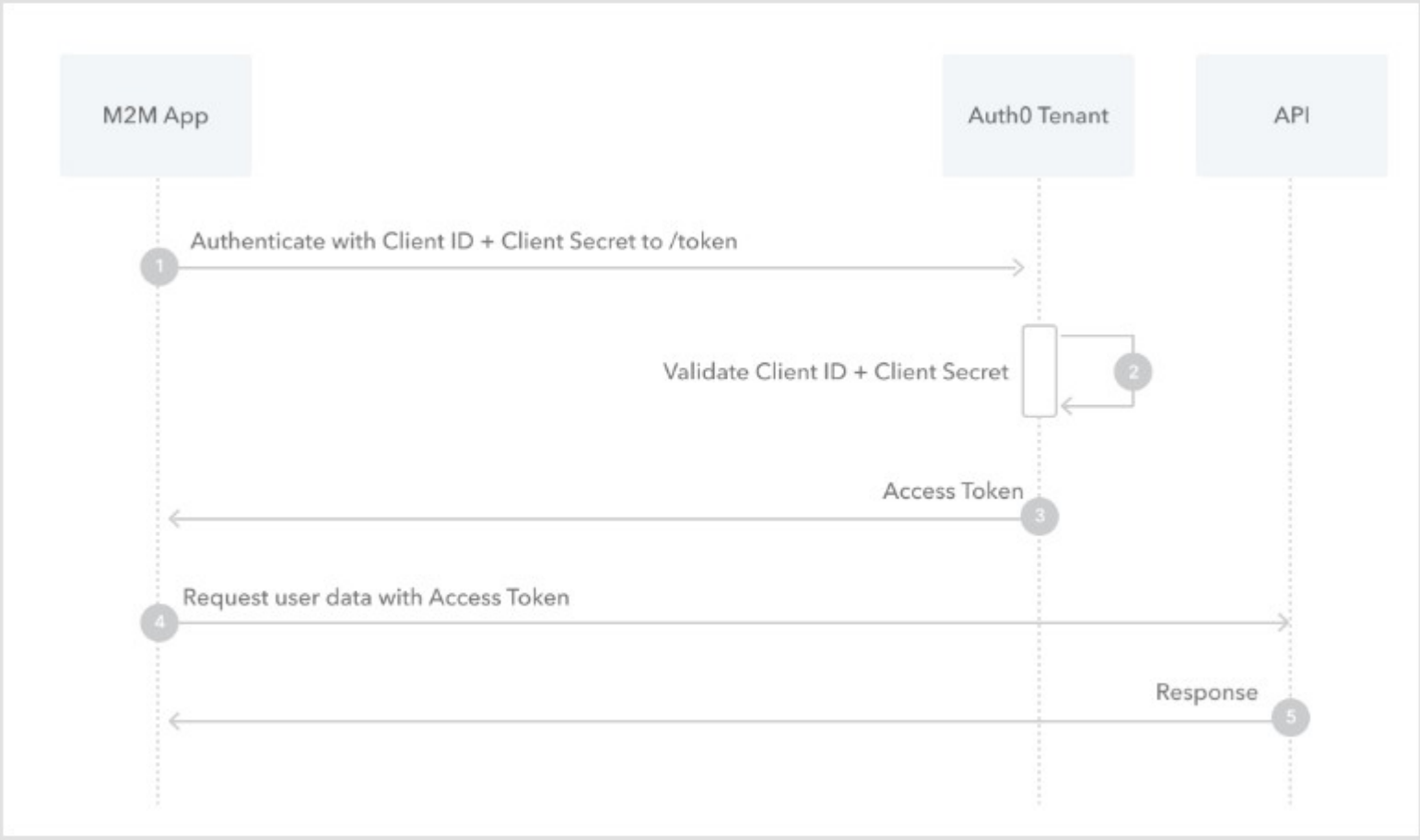
Access Token Pattern

La gateway relaie le jeton d'accès

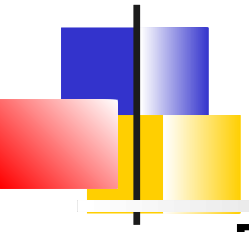



JHipster OAuth2 communication flows
JHipster Gateway
angular $http client
Microservice 1
REST client
resource endpoint
Microservice 2
resource endpoint
REST client
Microservice 3
resource endpoint
REST client
JHipster UAA
oauth endpoint
user / account resource
legend
OAuth client
authorized request
resource server
OAuth client
authentication
authorization server

Client Credentials Flow avec secret



Client credentials avec certificat X509



Dans ce cas, une application backend obtient sont jeton via le flow clent credentials

Pour s'authentifier au lieu d'utiliser un secret il utilise son certificat X509

Configuration Keycloak

- Déclarer un client avec Client authentification et Service Account Roles
- Dans l'onglet credentials, définir X509 et indiquer le Subject DN complet du certificat

Client credentials avec JWT-signé

Permet à un client de s'authentifier en utilisant un JWT signé avec une clé privée

Génération du JWT

- Le client génère un JWT contenant des revendications (claims) spécifiques. généralement client_id, iss, sub, aud et iat.
- Le JWT est signé avec une clé privée détenue par le client. Le serveur vérifie l'intégrité et l'authenticité avec la clé publique correspondante.

Envoi du JWT :

- Le client envoie le JWT signé dans le cadre d'une requête d'authentification en utilisant généralement le champ client_assertion dans une requête de token.

Validation du JWT :

- Le serveur valide le JWT en vérifiant la signature avec la clé publique du client.
- Il vérifie également les revendications pour s'assurer que le JWT est valide et n'a pas expiré.
- Si le JWT est valide, le serveur d'autorisation procède à l'authentification du client et peut émettre un token d'accès.



Mise en œuvre dans Keycloak

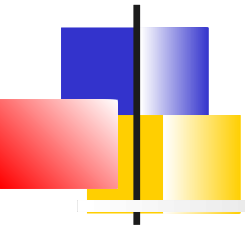
Dans l'onglet *Credentials*, sélectionner ***Signed JWT*** ou *Signed JWT with secret*

Dans l'onglet *Keys*, ou peut générer ou importer une paire clé privé/clé publique.

Dans l'application cliente, générer un JWT et le signer avec la clé privée puis :

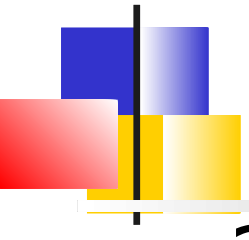
```
POST /realms/{realm-name}/protocol/openid-connect/token HTTP/1.1
Host: keycloak.example.com
Content-Type: application/x-www-form-urlencoded


grant_type=client_credentials
&client_id={client-id}
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion={signed-jwt}
```



Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage

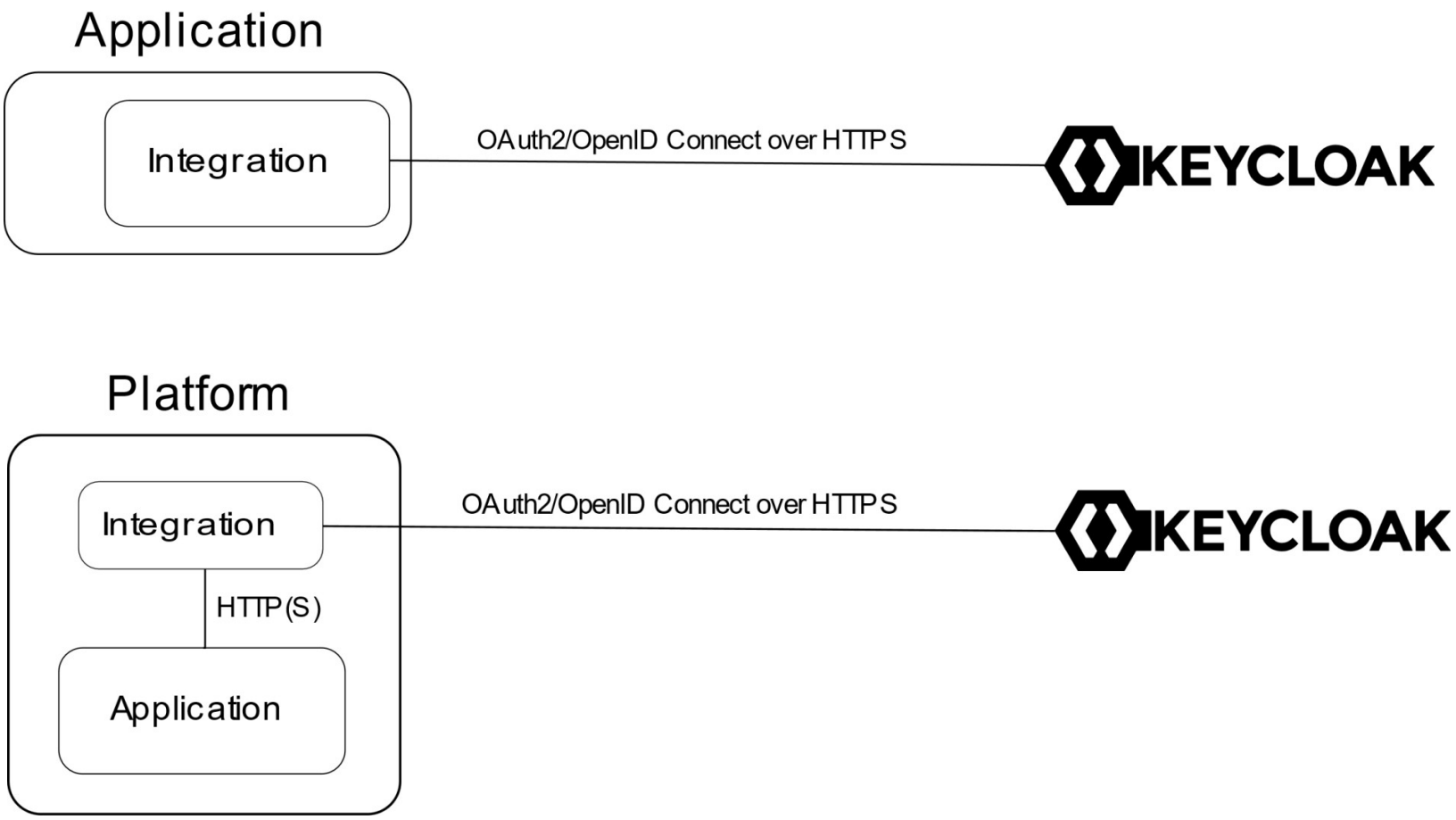


Introduction

2 principaux styles d'intégration, selon l'emplacement du code d'intégration et de la configuration

- Embarqué : Le code embarque des librairies du langage ou du framework choisi
- Proxy : Utiliser un reverse proxy devant son application.
  Code legacy, Gateway micro-service

Embarqué ou Proxy

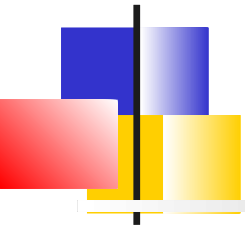


Librairies OpenID/oAuth

De nombreuses librairies bas niveau OpenID sont disponibles :

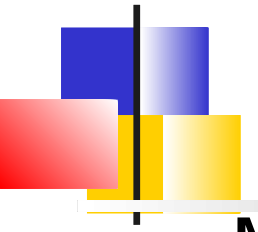
- C : *mod_auth_openidc*
- C# : *IdentityModel.OidcClient*
- Java : *GKIDP Broker*
- Golang : *OpenID Connect SDK*
- Javascript : *node openid-client, oidc-client-js, oauth4webapi*
- PHP : *phpOIDC 2016 Winter*
- Python : *OidcRP, pyoidc*
- Typescript : *angular-auth-oidc-client*

Elles sont référencées et certifiées par OpenIdConnect :
https://openid.net/developers/certified/



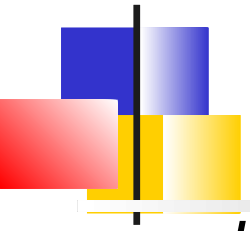
Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage



Introduction

- Même si il sont toujours documentés, les adaptateurs Keycloak sont en cours de dépréciation.
- Ils ont vocation à être remplacés par les frameworks de développement qui progressivement offre du support pour OIDC/oAuth2 donc Keycloak.
- Seul l’adaptateur Javascript sera maintenu.



Adaptateurs Keycloak

Keycloak distribue des adaptateurs pour différentes plates-formes :

- Java : *JBoss EAP, WildFly, Tomcat, Jetty, Filtres Servlet, Spring Boot, Spring Security (dépréciés)*
- Javascript : Client-side
- Node.js (deprécié) : Server side
- Apache HTTP Server : *mod_auth_openidc*

Ces adaptateurs sont configurables via un simple fichier ***keycloak.json*** qui peut être généré via la console d'admin

Exemple adaptateur Javascript

1) Charger la librairie via l'URL :

```
KC_URL/js/keycloak.js Ou npm install keycloak-js
```

2) Créer un objet *Keycloak* avec les informations du client et l'initialiser lorsque la fenêtre est chargée.

```
<head><script>
    function initKeycloak() {
        const keycloak = new Keycloak(); // Configuration avec keycloak.json
        keycloak.init({
          onLoad: 'login-required'  // Nécessite le login lors du chargement de page
        }).then(function(authenticated) {
            alert(authenticated ? 'authenticated' : 'not authenticated');
        }).catch(function() {
            alert('failed to initialize');
        });
    }
</script></head><body onload="initKeycloak()">
```

3) Après l'authentification, le token est présent dans ***keycloak.token***

```
req.setRequestHeader('Authorization', 'Bearer ' + keycloak.token);
```



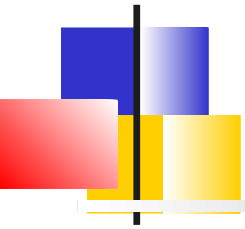
Configuration de l'objet keycloak

Instanciation : ***url***, ***realm*** et ***clientid***

```
 const keycloak = new Keycloak({
url: 'http://keycloak-server${kc_base_path}',
realm: 'myrealm',
clientId: 'myapp'
});
```

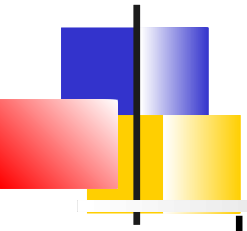
Paramètres de la fonction init() :

- ***login-required*** : Affiche la page de login si l'utilisateur n'est pas déjà authentifié
- ***check-sso*** : Succès seulement si l'utilisateur est déjà authentifié
- ***checkLoginIframe*** : Autorise une iframe qui détecte les logout sso
- ***flow*** : Le flow utilisé. Supporte également *implicit* et *hybrid*
- ***pkceMethod*** *:* Si on veut activer PKCE, seule valeur possible S256
- ***enableLogging*** : true pour activer les traces
- ***scope*** : Scopes demandés
- ...



Intégration KeyCloak

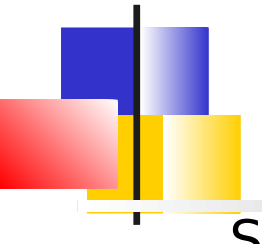
Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage



Apports de SpringBoot

Le support de *oAuth2* via Spring comprend 3 starters :

- ***OAuth2 Client*** : Intégration pour s'authentifier avec OpenIDConnect avec Google, Github, Facebook, Keycloak ...
- ***OAuth2 Resource server*** : Extraction du jeton et vérification des ACLs par rapport aux scopes client et/ou aux rôles contenu dans le jeton d'accès
- ***OAuth2 Authorization server :*** Un keycloak en Spring Boot (réapparu depuis peu)
- ***Okta*** : Pour travailler avec le fournisseur oAuth Okta (Pas disponible dans toutes les versions)



OpenIDConnect avec SpringBoot

Starter ***oauht2-client***

```
@Bean
public SecurityWebFilterChain
securityWebFilterChain(ServerHttpSecurity http) {
    return http.authorizeExchange()
      .anyExchange().authenticated()
      .and().oauth2Login()
      .and().csrf().disable()
      .build();
}
```

Configuration Keycloak

```
spring:
  security:
    oauth2:
      client:
        provider:
          keycloak:      # Ou tout simplement issuer-uri
            token-uri:
   http://localhost:8089/realms/<realm-name>/protocol/openid-
   connect/token
            authorization-uri: http://localhost:8089/realms/<realm-
   name>/protocol/openid-connect/auth
            user-info-uri: http://localhost:8089/auth/realms/<realm-
   name>/protocol/openid-connect/userinfo
            user-name-attribute: preferred_username
        registration: # Configuration client
          spring-app:
            provider: keycloak
            client-id: spring-app
            client-secret: 57abb4f6-5130-4c73-9545-6d377dd947cf
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/keycloak"
            scope :openid
```

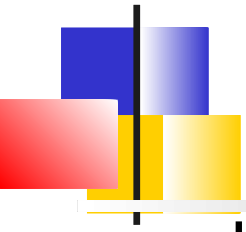
Accès à l’utilisateur loggé

```
@GetMapping("/oidc-principal")
public OidcUser getOidcUserPrincipal(
  @AuthenticationPrincipal OidcUser principal) {
    return principal;
}

…

Authentication authentication =
   SecurityContextHolder.getContext().getAuthentication();
if (authentication.getPrincipal() instanceof OidcUser) {
    OidcUser principal = ((OidcUser)
   authentication.getPrincipal());

    // ...
}
```

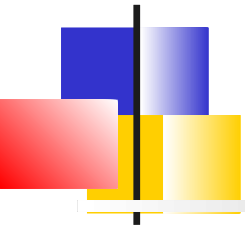


Logout

L’application Spring lors d’un logout doit se déconnecter sur Keycloak en utilisant le endpoint ***/protocol/openid-connect/logout*** et en fournissant dans le paramètre ***id_token_hint*** le jeton.

Cela est configuré via un ***LogoutHandler*** de Spring :

```
httpSecurity.logout(logout -> logout.logoutUrl("/logout").
   addLogoutHandler(keycloakLogoutHandler).
   invalidateHttpSession(true).
   logoutSuccessUrl("/"))
```



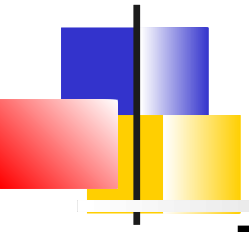
LogoutHandler

```
private void logoutFromKeycloak(OidcUser user) {
    String endSessionEndpoint = user.getIssuer() + "/protocol/openid-connect/logout";

    UriComponentsBuilder builder = UriComponentsBuilder
      .fromUriString(endSessionEndpoint)
      .queryParam("id_token_hint", user.getIdToken().getTokenValue());



    ResponseEntity<String> logoutResponse = restTemplate.getForEntity(
    builder.toUriString(), String.class);
    logger.info("Redirecting to " + builder.toUriString());
    if (logoutResponse.getStatusCode().is2xxSuccessful()) {
        logger.info("Successfulley logged out from Keycloak");
    } else {
        logger.error("Could not propagate logout to Keycloak");
    }
}
```



SSO-logout

- Pour gérer le sso (logout initié par un autre application), il faut utiliser un backchannel logout

```
@PostMapping("/sso-logout")
public void ssoLogout(@RequestParam("logout_token")
  String logoutToken, HttpSession session) {

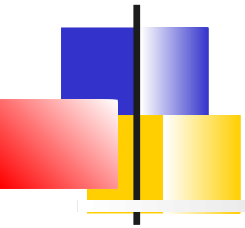

    session.invalidate();
}
```

Serveur de ressources

Dépendance : **`oauth2-resource-server`**

Le serveur de ressources doit vérifier la signature du jeton pour s'assurer que les données n'ont pas été modifiées.

- ***jwk-set-uri*** contient la clé publique que le serveur peut utiliser pour la vérification
- ***issuer-uri*** pointe vers l'URI de Keycloak. Utilisé pour la découverte de *jwk-set-uri*



Exemple *application.yml*

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          # issuer-uri: http://keycloak:8083/realms/<realm-name>
          jwk-set-uri: http://keycloak:8083/realms/<realm-name>/protocol/openid-connect/certs
```

Scope vs Spring Authority

Spring ajoute des autorités[1] au principal en fonction des *scopes* présents dans le jeton

Les autorités correspondant aux scopes sont préfixées par "SCOPE_".
Par exemple, la portée *openid* devient une autorité accordée SCOPE_openid.

Il ajoute également l'autorité *OIDC_USER*

Cela peut être adapté via un bean ***JwtAuthenticationConverter***

- Positionner les Authorities à partir d'autre(s) Claim
- Changer le préfixe utilisés

1. Classe Authority de spring-security

Exemple pour Keycloak

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverterForKeycloak() {

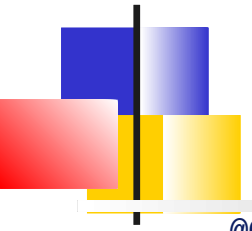
   Converter<Jwt, Collection<GrantedAuthority>> jwtGrantedAuthoritiesConverter = jwt -> {
        Map<String, Collection<String>> realmAccess = jwt.getClaim("realm_access");
        Collection<String> roles = realmAccess.get("roles");
        return roles.stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
            .collect(Collectors.toList());
    };

    var jwtAuthenticationConverter = new JwtAuthenticationConverter();

    jwtAuthenticationConverter.setJwtGrantedAuthoritiesConverter(jwtGrantedAuthoritiesConver
    ter);

    return jwtAuthenticationConverter;
}
```

Configuration typique *SpringBoot*



```
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.cors()
            .and()
            .authorizeRequests()
              .antMatchers(HttpMethod.GET, "/user/info", "/api/foos/**")
                .hasAuthority("SCOPE_read")
              .antMatchers(HttpMethod.POST, "/api/foos")
                .hasAuthority("SCOPE_write")
              .anyRequest()
                .authenticated()
            .and()
              .oauth2ResourceServer()
                .jwt();
    }
}
```



Adaptateur SpringBoot

Keycloak a un adaptateur pour SpringBoot mais il est déprécié

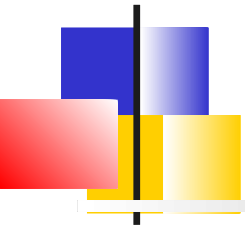
`org.keycloak:keycloak-spring-boot-starter`

Support pour *Tomcat, Undertow, Jetty*

La configuration s'effectue via le fichier de configuration SpringBoot

Les ACLS peuvent facilement s'appuyer sur les rôles

```
keycloak.realm = demorealm
keycloak.auth-server-url = http://127.0.0.1:8080
keycloak.ssl-required = external
keycloak.resource = demoapp
keycloak.credentials.secret = 11111111-1111-1111-1111-111111111111
keycloak.use-resource-role-mappings = true
keycloak.securityConstraints[1].authRoles[0] = admin
keycloak.securityConstraints[1].securityCollections[0].name = admin stuff
keycloak.securityConstraints[1].securityCollections[0].patterns[0] = /admin
```



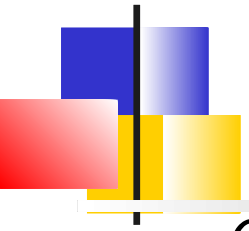
Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage

Extensions *oidc*

Différentes extensions Quarkus peuvent être utilisées en fonction des usages :

- ***quarkus-oidc*** : fournit un adaptateur OpenID supportant l'extraction du jeton et l' Authorization Code Flow
- ***quarkus-oidc-client*** : Permet d'obtenir des tokens d'accès et de rafraîchissement auprès de fournisseur supportant les grant type : *client-credentials, password* et *refresh_token*
- ***quarkus-oidc-token-propagation*** : Dépend de *quarkus-oidc* Permet de propager un jeton d'accès aux services en aval
- ***quarkus-oidc-client-filter*** : Dépend de *quarkus-oidc-client* Positionne le jeton obtenu par *oidc-client* dans l'entête d'Authorization pour faire des appels REST vers des services aval



DevServices et *KeyCloak*

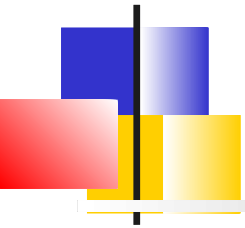
Quarkus propose une auto-configuration pour *KeyCloak*,
=> Pas besoin d'installer Keycloak sur son poste

Activé si

- *quarkus-oidc*
- dev et test mode
- La propriété *quarkus.oidc.auth-server-url* n'est pas configuré

Le service démarre un container *Keycloak* et l'initialise en important ou créant un realm.
Le realm peut-être spécifié par :
`quarkus.keycloak.devservices.realm-path=quarkus-realm.json`

De plus, la Dev UI (*/q/dev*) permet d'acquérir les jetons de Keycloak et de tester l'application Quarkus.



microprofile-jwt

Quarkus demande le profil **microprofile-jwt**

Ce profil génère les rôles utilisateur de Keycloak dans l'attribut ***groups*** du jeton.

Quarkus mappe l'attribut groups à sa propre notion de rôle

Exemple Service REST

```
@Path("/api/users")
public class UsersResource {

    @Inject
    SecurityIdentity securityIdentity;

    @GET
    @Path("/me")
    @RolesAllowed("user")
    @NoCache
    public User me() {
        return new User(securityIdentity);
    }

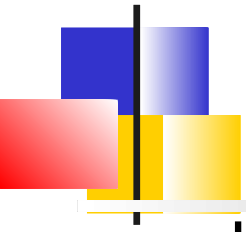
    public static class User {

        private final String userName;

        User(SecurityIdentity securityIdentity) {
            this.userName = securityIdentity.getPrincipal().getName();
        }

        public String getUserName() {
            return userName;
        }
    }
}
```

Client REST utilisant les jetons oAuth2



Un client REST voulant accéder à une ressource protégée par oAuth2 doit utiliser un jeton.

2 scénarios

- Un client obtient son propre jeton et l'utilise dans ses appels rest :
  oidc-client +
  oidc-client-reactive-filter ou oidc-client-filter
  un micro-service accédant à un autre micro-service
- Le token courant (obtenu par oidc) est propagé
  oidc-token-propagation ou oidc-token-propagation-reactive
  Cas de la gateway par exemple

Configuration oidc-client

```
# Découverte automatique
quarkus.oidc-client.auth-server-url=
   http://localhost:8180/auth/realms/quarkus

# Découverte manuelle
quarkus.oidc-client.discovery-enabled=false
quarkus.oidc-client.token-path=
   http://localhost:8180/auth/realms/quarkus/protocol/openid-c
   onnect/tokens

# grant type client_credentials
quarkus.oidc-client.auth-server-url=http://localhost:8180/realms/quarkus/
quarkus.oidc-client.client-id=quarkus-app
quarkus.oidc-client.credentials.secret=secret
```

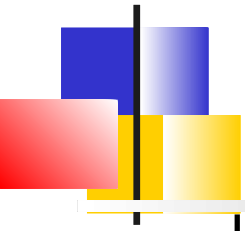
Utilisation dans le RestClient

Extensions : ***quarkus-oidc-client-reactive-filter*** ou ***quarkus-oidc-client-filter***

Les extensions fournissent ***OidcClientRequestReactiveFilter*** ou ***OidcClientRequestFilter*** qui peuvent être appliqués à une interface RestClient

```
@RegisterRestClient
@RegisterProvider(OidcClientRequestReactiveFilter.class)
@Path("/")
public interface ProtectedResourceService {

    @GET
    Uni<String> getUserName();
}
```



Propagation de jeton

L'extension ***quarkus-oidc-token-propagation*** fournit 2 implémentations de *ClientRequestFilter* qui simplifient la propagation des informations d'authentification

- ***AccessTokenRequestFilter*** propage le jeton Bearer présent dans la requête en cours
- ***JsonWebTokenRequestFilter*** fournit la même fonctionnalité, mais fournit en plus la prise en charge des jetons JWT

Cette extension est typiquement utilisée :

- Pour propager le jeton venant d'être obtenu par l'Authorization Code Flow.
  Ex : requête initiale Gateway
- Pour propager le jeton présent dans le Bearer
  le même jeton circule dans toute l'architecture micro-services



Enregistrement du filtre *AccessToken*

```
@RegisterRestClient
@AccessToken
@Path("/")
public interface ProtectedResourceService {

    @GET
    String getUserName();
}
```

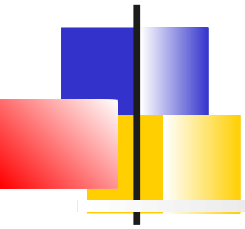
Ou

```
@RegisterRestClient
@RegisterProvider(AccessTokenRequestFilter.class)
@Path("/")
public interface ProtectedResourceService {

    @GET
    String getUserName();
}
```

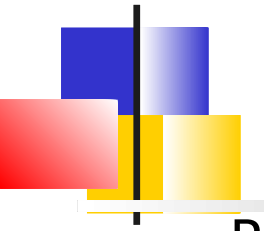
Ou automatiquement si configuration suivante :

```
quarkus.oidc-token-propagation.register-filter=true
quarkus.oidc-token-propagation.json-web-token=false
```



Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage



Écosystème Python

Pas d'adaptateur officiel : on utilise des librairies OpenID / OAuth2 standards

Authlib : client OAuth2 / OIDC, intégrations Flask, Django, Starlette / FastAPI

PyJWT : décodage et validation de JWT, client JWKS intégré

mozilla-django-oidc : authentification OIDC pour Django

python-keycloak : accès aux endpoints OIDC et à l'API d'administration

Mêmes principes qu'en Java :

- Application web : Authorization Code Flow, session côté serveur
- API : validation du jeton Bearer (signature, iss, aud, exp)

Application web Flask avec Authlib

```
oauth = OAuth(app)
oauth.register("keycloak",
  client_id="flask-app", client_secret="...",
  server_metadata_url="https://kc/realms/myrealm/.well-known/openid-configuration",
  client_kwargs={"scope": "openid profile email"})

@app.route("/login")
def login():
  return oauth.keycloak.authorize_redirect(url_for("auth", _external=True))

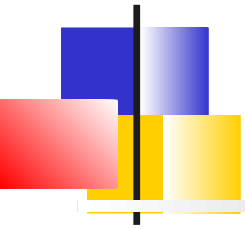
@app.route("/auth")
def auth():
  token = oauth.keycloak.authorize_access_token()
  session["user"] = token["userinfo"]
  return redirect("/")
```

API FastAPI : validation du jeton

```
import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

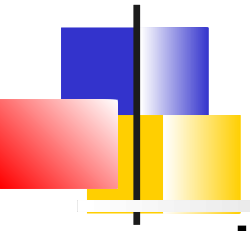
ISSUER = "https://kc/realms/myrealm"
jwks = jwt.PyJWKClient(ISSUER + "/protocol/openid-connect/certs")

def current_user(cred = Depends(HTTPBearer())):
  try:
    key = jwks.get_signing_key_from_jwt(cred.credentials)
    return jwt.decode(cred.credentials, key.key, algorithms=["RS256"],
                      audience="my-api", issuer=ISSUER)
  except jwt.PyJWTError:
    raise HTTPException(status_code=401)
```



Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage



Introduction

- La prise en charge d'OpenID Connect et d'OAuth2 est une fonctionnalité obligatoire pour les proxys.
  - Apache HTTP Server et Nginx, fournissent les extensions nécessaires pour ces protocoles

Exemple Apache

Installation du module ***mod_auth_oidc***[1]

Puis configuration

```
LoadModule auth_openidc_module modules/mod_auth_openidc.so
ServerName localhost
<VirtualHost *:80>
  ProxyPass / http://localhost:8000/
  ProxyPassReverse / http://localhost:8000/
  OIDCCryptoPassphrase CHANGE_ME
  OIDCProviderMetadataURL http://keycloak/realms/<realm-name>/.well-known/openid-configuration
  OIDCClientID mywebapp
  OIDCClientSecret CLIENT_SECRET
  OIDCRedirectURI http://localhost/callback
  OIDCCookieDomain localhost
  OIDCCookiePath /
  OIDCCookieSameSite On
  <Location />
    AuthType openid-connect
    Require valid-user
  </Location>
</VirtualHost>
```


1. *https://github.com/zmartzone/mod_auth_openidc*

Nginx

- Pas de module OIDC dans Nginx open source (module natif réservé à NGINX Plus)
- Solution courante : auth_request + oauth2-proxy

```
location /oauth2/ { proxy_pass http://oauth2-proxy:4180; }
location = /oauth2/auth {
  proxy_pass http://oauth2-proxy:4180;
  proxy_pass_request_body off;
  proxy_set_header Content-Length "";
}
location / {
  auth_request /oauth2/auth;
  error_page 401 =403 /oauth2/sign_in;
  auth_request_set $user $upstream_http_x_auth_request_user;
  proxy_set_header X-User $user;
  proxy_pass http://backend:8080;
}
```

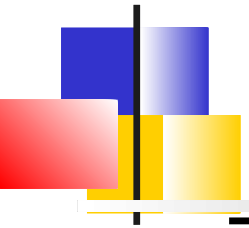
oauth2-proxy

Reverse proxy d'authentification OAuth2 / OIDC, provider dédié Keycloak

```
oauth2-proxy --provider=keycloak-oidc \
  --oidc-issuer-url=https://kc/realms/myrealm \
  --client-id=web-proxy --client-secret=... \
  --cookie-secret=<32 octets aléatoires> \
  --email-domain=* --allowed-group=/admins \
  --set-xauthrequest --reverse-proxy
```

Côté Keycloak :

- Client confidentiel, redirect URI https://app/oauth2/callback
- Mapper Audience avec le client-id pour que le jeton soit accepté
- Mapper Group Membership si filtrage par groupe

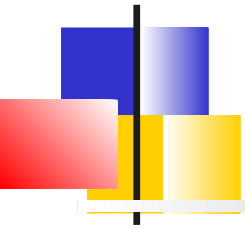


Traefik

- Traefik Proxy n’intègre pas OIDC : délégation via le middleware ForwardAuth

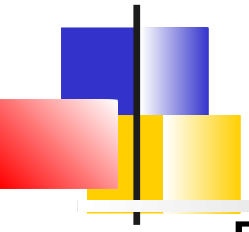
```
labels:
  - traefik.http.middlewares.oidc.forwardauth.address=
      http://oauth2-proxy:4180/oauth2/auth
  - traefik.http.middlewares.oidc.forwardauth.authResponseHeaders=
      X-Auth-Request-User,X-Auth-Request-Email
  - traefik.http.middlewares.oidc-err.errors.status=401
  - traefik.http.middlewares.oidc-err.errors.service=oauth2-proxy
  - traefik.http.middlewares.oidc-err.errors.query=/oauth2/sign_in
  - traefik.http.routers.app.middlewares=oidc-err,oidc
```

- Alternatives : plugins OIDC communautaires, middleware OIDC de Traefik Hub



Intégration KeyCloak

Introduction
Librairies et adaptateurs
SpringBoot
Quarkus
Python
Reverse Proxy
Débogage



Outils de diagnostic

Décoder les jetons : jwt.io ou en ligne de commande

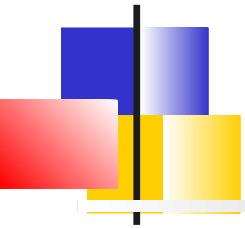
Client scopes → Evaluate : aperçu de l'access token, de l'ID token et de UserInfo générés pour un utilisateur

Events de login : Realm settings → Events → User events

- Type d'événement, client, IP et code d'erreur

Outils de développement du navigateur : paramètres de la redirection, code, state, cookies

curl sur les endpoints token, userinfo et introspect



Logs d’intégration

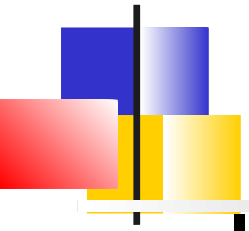
Côté Keycloak :

```
kc.sh start --log-level="INFO,org.keycloak.events:debug"
kc.sh start --log-level="INFO,org.keycloak.authentication:debug"
```

Le listener jboss-logging trace les erreurs de login en WARN et les succès en DEBUG

Côté applications :

```
# Spring Boot
logging.level.org.springframework.security=DEBUG
# Quarkus
quarkus.log.category."io.quarkus.oidc".level=DEBUG
# Python
logging.getLogger("authlib").setLevel(logging.DEBUG)
```



Erreurs fréquentes

Invalid parameter: redirect_uri : URI non déclarée dans le client (correspondance exacte)

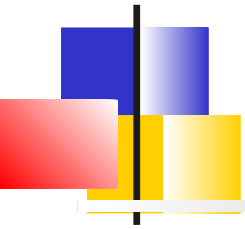
invalid_client / unauthorized_client : secret erroné, client public ou confidentiel, flow non activé

401 sur l'API :

- Claim aud absent : ajouter un mapper Audience
- iss différent : hostname vu par le navigateur ≠ hostname vu par le back-end (localhost vs nom de conteneur)
- Clé inconnue (kid) après rotation : cache JWKS à rafraîchir
- Horloges désynchronisées : exp, iat, nbf

Erreurs CORS : Web origins du client

Boucle de redirection : cookies bloqués (SameSite, cookies tiers)



Exploitation courante

Gestion des utilisateurs

Mots de passe et MFA

Sessions et jetons

Événements et audit

Utilisateurs locaux et externes

Keycloak gère une base utilisateur dans sa base de données. Les opérations de gestion concerne :

- La création d’utilisateur
- La gestion de leurs différents crédentiels
- La définition des actions requises
- L’activation éventuelle de la self-registration
- La personnalisation des attributs

D’autre part, Keycloak peut s’intégrer avec des systèmes externes comme LDAP ou d’autres fournisseurs d’identité : "fédération d'utilisateurs"

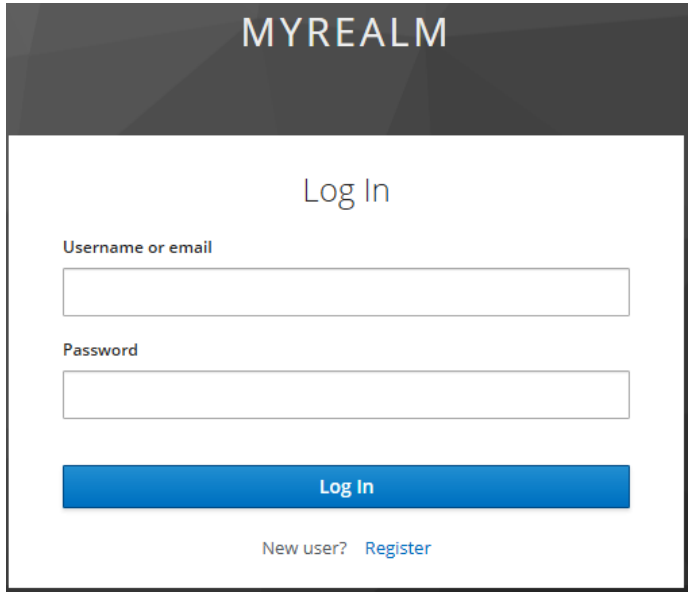
Required User Actions

- Keycloak permet d'interagir avec les utilisateurs pendant le processus d'authentification à l'aide d'une fonctionnalité appelée ***Required User Actions***
  - ***Verify Email*** : envoyer un e-mail à l'utilisateur pour confirmer qu'il appartient à cet utilisateur.
  - ***Update Password***: demandez à l'utilisateur de mettre à jour son mot de passe.
  - ***Update Profile***: demandez à l'utilisateur de mettre à jour son profil en fournissant son prénom, son nom et son adresse e-mail.

Self-registration

Keycloak permet aux utilisateurs de s'enregistrer par eux même

Realm Settings → Login → User registration



Compte utilisateurs

Empêcher un attaquant d'accéder à un compte utilisateur consiste principalement à activer une authentification forte (pas seulement un mot de passe comme moyen d'authentification).

La protection des mots de passe nécessitent :

- Algorithme de hachage de mot de passe fort
  Realm Settings → Authentication → Password Policy
- Une bonne politique pour les mots de passe (combinaison de caractères spéciaux, digits, majuscule, minuscule, longueur)
  Realm Settings → Authentication → Password Policy
- Une protection contre les attaques force-brute
  Realm Settings → Security Defense → Brute Force Protection
- Éduquer les utilisateurs afin qu'il utilise différents mots de passe entre service

Récupération de compte

Mot de passe oublié : Realm settings → Login → Forgot password

- Nécessite un serveur SMTP : Realm settings → Email
- Parcours défini par le flow « reset credentials »
- Lien envoyé par mail à durée limitée : Realm settings → Tokens → action tokens

Perte du second facteur :

- Codes de récupération (Recovery Authentication Codes) générés à l'enrôlement
- Ou suppression de l'OTP par un administrateur puis ré-enrôlement

Verrouillage après échecs répétés : Realm settings → Security defenses → Brute force detection

Actions de l’administrateur

Onglet Credentials d’un utilisateur :

- Réinitialiser le mot de passe, éventuellement temporaire
- Supprimer un credential (OTP, WebAuthn)

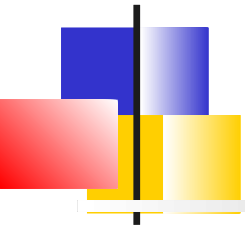
Envoi d’un mail d’actions : Credentials → Credential reset

- UPDATE_PASSWORD, CONFIGURE_TOTP, VERIFY_EMAIL, UPDATE_PROFILE

Required actions positionnées manuellement, exécutées au prochain login

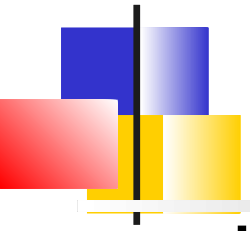
Déverrouiller un compte bloqué par la détection brute force

Désactiver un compte (Enabled = off) sans le supprimer



Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit



Introduction

Keycloak permet de configurer l'authentification des utilisateurs :

- Politique sur les mots de passe
- Utilisation de OTP (One Time Password)
- Utilisation de WebAuthn (Web Authentication)
- Configuration des séquences de l'authentification

Politique des mots de passe

Keycloak permet de définir des contraintes sur les mots de passe

- Algorithme de hachage
  Par défaut : PBKDF2
- Itérations de Hash
  Impact sur les performances
- Digits, LowerCase, UpperCase, Caractères spéciaux
- NotUsername, Not Email
- Expression régulière
- Date d’expiration
- Liste noire : Les mots de passe interdits

Authentication → Policies → Password Policy



OTP

En plus de fournir un mot de passe, les utilisateurs doivent fournir une 2ème preuve de leur identité

- Un code fourni par les applications FreeOTP ou Google Authenticator disponible sur mobile
- Soit l'utilisateur, soit l'administrateur peut décider de l'utilisation de OTP

Différentes stratégies peuvent être configurées pour l'OTP :

- Time-Based One-Time Password (TOTP) : Le code est valable pendant une certaine période (30 secondes par défaut)
- HMAC-Based One-Time Password (HOTP) : Le code est valable tant qu'il n'a pas été utilisé

Authentication → Policies → OTP Policy

Mise en place OTP

Les utilisateurs peuvent activer par eux même OTP via la console de gestion de leur compte

Signing In → Set up Authenticator Application

=> QR code représentant la clé partagée qui servira à générer les codes

Pour forcer pour tous les utilisateurs, il faut définir un autre flow d'authentification



WebAuthn

WebAuthn (Web Authentication API) est une norme du W3C conçue pour remplacer les mots de passe par des méthodes d'authentification[1] plus sécurisées, comme :

- Des clés de sécurité matérielles (type YubiKey)
- L'empreinte digitale
- La reconnaissance faciale
- Le code PIN d'un appareil sécurisé

Il s'appuie sur la cryptographie à clé publique.

WebAuthn est plus sécurisé que OTP car il n'y a pas de clé partagée entre Keycloak et les applications tierces (FreeOtp et GoogleAuthenticator)

1. Doit être conforme aux exigences de FIDO2

Fonctionnement général

Lors de l’enregistrement :

- L’authenticator génère une paire de clés (publique/privée).
- La clé publique est envoyée au serveur (ici Keycloak).
- La clé privée est stockée dans l’authenticator (clé matérielle, OS, etc.).

Lors de l’authentification :

- Le serveur envoie un challenge.
- Le navigateur le signe en faisant appel à l’authenticator avec la clé privée.
- Le serveur vérifie la signature avec la clé publique.

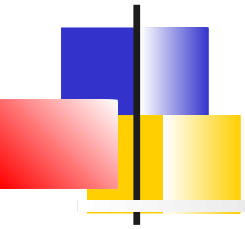
Mise en place *Webauthn*

Définir un flow avec

WebAuthn authentication ou WebAuthn Passwordless

Dans sa console, l’utilisateur enregistrer son Authenticator





Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit

Introduction

Les sessions permettent à Keycloak de déterminer si les utilisateurs et les clients sont authentifiés, pendant combien de temps ils doivent être authentifiés et quand il est temps de les ré-authentifier.

Les sessions sont conservées en mémoire et peuvent avoir un impact sur Keycloak

L’exploitant Keycloak peut alors :

- Configurer des timeouts
- Visualiser les sessions actives
- Faire de l’audit
- Forcer la fermeture de session

Types de sessions

Keycloak maintient 2 types de session lorsqu'un utilisateur s'authentifie :

- **La session Single Sign-on (SSO)**, permet de suivre l'activité de l'utilisateur quelque soit le client/application
  La configuration de ses timeouts contrôle la fréquence des ré-authentifications utilisateurs et clients
  Lorsque la session expire, toutes les sessions client associées expirent
- **Les sessions cliente** Sous-sessions associées à chaque client (application) accédé par l'utilisateur.
  Gère : les jetons (Access, Refresh), les scopes autorisés,les timeouts propres à ce client (si configurés).
  Expire si la session sso expire ou si le refresh token associé expire ou est révoqué

Timeout

La configuration des timeout pour les différentes sessions se fait dans
Realm settings → Sessions

On définit principalement :

- ***Idle timeout*** : La durée faisant expirer la session si il n'y a pas d'activité user ou de rafraîchissement de jeton (par défaut 30mn)
- ***Max time*** : La durée maximale d'une session

Au niveau d'un client :
Client → Settings → Login Settings

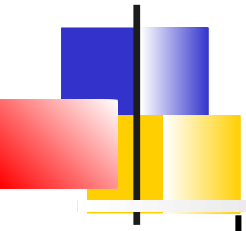
Gestion des sessions actives

- Les administrateurs ont une visibilité des sessions actives au niveau
  - Du realm
  - D’un client
  - D’un utilisateur
- Il peut forcer un logout à tous les niveaux

Cookies

- Après une authentification réussie, Keycloak positionne un cookie HttpOnly[1] ***KEYCLOAK_IDENTITY*** avec une expiration correspondant au Max Time d'une session
- Si https, il positionne également le flag secure sur le cookie empêchant un transfert en clair

1. Le cookie n'est pas accessible par le javascript du navigateur pour se prévenir des attaques XSS



Jetons

- Les jetons sont généralement liés à des sessions. Par conséquent, la validité des jetons (pas nécessairement leur durée de vie) dépend des sessions.
- Les jetons ont leur propre durée de vie et la durée pendant laquelle ils sont considérés comme valides dépend de la façon dont ils sont validés
- Avec JWT, les serveurs de ressources valident indépendamment de Keycloak un jeton
  => Pendant sa durée de vie, la session peut avoir expirée
  => Si l'on ne veut pas, il faut se rabattre sur une validation en utilisant l'endpoint d'introspection

Durée de vie des jetons

- Les jetons d’identification et d’accès partagent la même durée de vie, généralement courte car ils sont utilisés par des clients publics et fréquemment échangés
- Les jetons de rafraîchissement ont une durée plus longue mais leur validité dépend de la durée de vie définie pour les sessions utilisateur et client
  Dans Keycloak, leur durée de vie suivent la configuration des timeout idle des sessions SSO ou sessions clientes

Configuration des jetons

La configuration s'effectue dans

Realm Settings → Tokens

Les valeurs par défaut sont 5 minutes pour les jetons d'accès et d'identification

On peut configurer globalement ou par clients

Jetons de rafraîchissement

Ils sont toujours liés à une session client

Ils sont considérés comme valides si les sessions utilisateur et client auxquelles ils sont liés n'ont pas expiré.

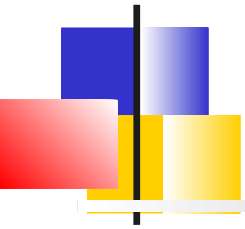
Les clients peuvent utiliser des jetons d'actualisation pour obtenir de nouveaux jetons uniquement si leurs sessions client respectives sont toujours actives.

Ils offrent une surface plus longue aux attaque

=> un client confidentiel a généralement des jetons d'actualisation plus long que les clients publics

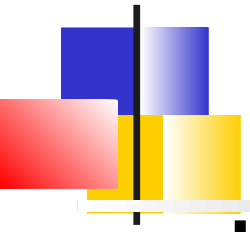
Rotation des jetons de rafraîchissement

- Stratégie pour empêcher les attaquants de réutiliser les jetons d'actualisation consistant à invalider le jeton avant d’en émettre un nouveau
  - Cela permet d'identifier rapidement le moment où le jeton d'actualisation a fuité et forcer l'attaquant ou le client légitime à se ré-authentifier
- En activant cette stratégie, l’administrateur doit définir également
  - ***Refresh Token Max Reuse*** : qui définit combien de fois un jeton de rafraîchissement peut être réutilisé. Par défaut 1



Exploitation courante

Gestion des utilisateurs
Mots de passe et MFA
Sessions et jetons
Événements et audit



Événements Keycloak

User events : actions des utilisateurs

- LOGIN, LOGIN_ERROR, LOGOUT, REGISTER, UPDATE_PASSWORD
- CODE_TO_TOKEN, REFRESH_TOKEN, CLIENT_LOGIN, …

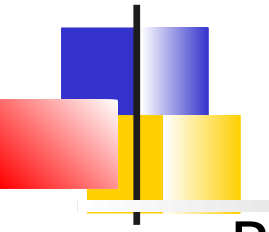
Admin events : opérations d'administration

- CREATE, UPDATE, DELETE, ACTION sur une ressource (user, client, rôle…)
- Via la console ou l'API REST

Les événements sont transmis à des event listeners :

- jboss-logging (actif par défaut), email, listeners personnalisés via SPI

Configuration des événements



Realm settings → Events

User events settings :

- Save events : stockage en base de données
- Expiration : purge automatique des événements anciens
- Choix des types d’événements enregistrés

Admin events settings : Save events

- Include representation : contenu JSON de la ressource modifiée (attention aux données sensibles)

Event listeners : ajout des listeners actifs pour le realm

Consultation et audit

Menu Events : onglets User events et Admin events

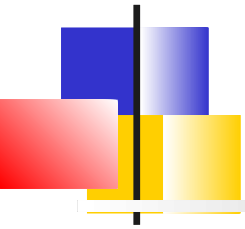
- Filtres : utilisateur, type, client, IP, période
- Admin events : realm, chemin de la ressource, opération, auteur

Accès via l'API d'administration :

```
GET /admin/realms/{realm}/events?type=LOGIN_ERROR
GET /admin/realms/{realm}/admin-events?operationTypes=DELETE
```

Usages : diagnostic des connexions, traçabilité des actions admin

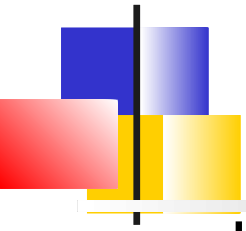
La base Keycloak n'est pas un outil d'audit à long terme : centralisation dans un SIEM (voir supervision)



Stratégies d’autorisation

RBAC, GBAC, OAuth2 scopes

Authorization service



Introduction

Les données relatives à l'autorisation (l'utilisateur peut-il accéder à une ressource) sont extraites du jeton fourni par Keycloak : les revendications

- Cela peut être n'importe quel revendication
- En général, on utilise les rôles, les groupes ou les scopes

2 stratégies pour implémenter l'autorisation :

- Implémenter le contrôle d'accès au niveau de l'application déclarativement ou programmatiquement
- Déléguer les décisions d'accès à un service externe

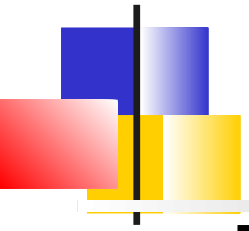
RBAC

RBAC (Role Base Access Control) vous permet de protéger les ressources en fonction de l'attribution ou non d'un rôle à l'utilisateur.

Keycloak a un support pour la gestion des rôles, ainsi que pour propager ces rôles dans les jetons

Les rôles représentent généralement un rôle qu'un utilisateur a

- dans l’organisation : Realm roles dans Keycloak
- ou dans le contexte d’une application :



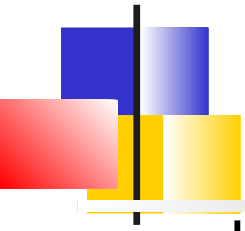
Gestion des rôles

Pour éviter l’explosion de rôles, n'utilisez pas les rôles pour une autorisation fine.

Keycloak propose 2 mécanismes pour faciliter la gestion des rôles :

- Un rôle peut être affecté à un groupe. => Tous les membres du groupe ont le rôle
- Un rôle peut être composite. Il englobe plusieurs rôles.

Groupes



Une stratégie ***GBAC (Group Base Access Control)*** définit des ACLs par rapport aux groupes

Les groupes représentent généralement le service auquel appartient un utilisateur

Les groupes sont hiérarchiques, ce sont une cartographie de l'organigramme d'une organisation

Les groupes ne sont pas automatiquement inclut dans les jetons. Cela nécessite un mapper particulier

Dans les jetons, les groupes sont représentés par un chemin

- Ex : */human resource/manager*



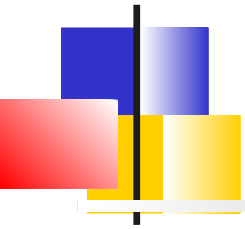
Modèle oAuth2

Dans le modèle oAuth2, les ACLs sont définis en fonction des scopes
=> l'autorisation est uniquement basée sur le consentement de l'utilisateur.
=> Les ACLs protège alors en fonction du client plutôt que de l'utilisateur
L'objectif principal étant de protéger les informations des utilisateurs plutôt que les ressources du serveur de ressources.

Par défaut, les clients de Keycloak n'utilisent pas le consentement de l'utilisateur.

Les services (APIs) peuvent donc naturellement configurer des ACLs en fonction du client qui les accède :

- Client interne / externe
- Outil de monitoring / service applicatif



Stratégies d’autorisation

RBAC, GBAC, OAuth2 scopes

Authorization service

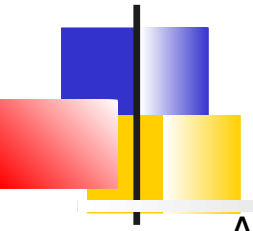
Introduction

L'autorisation centralisée permet d'externaliser la gestion des accès à l'aide d'un service d'autorisation externe.

Elle permet d'utiliser plusieurs mécanismes de contrôle d'accès sans y coupler l’application.

Keycloak peut agir comme un service d'autorisation centralisé via une fonctionnalité appelée ***Authorization Services***.

Introduction



Approche RBAC classique :

```
If (User.hasRole("manager") {
// can access the protected resource
}
```

=> Si l'on veut changer les règles d'accès (ajout d'un nouveau rôle par exemple), il faut modifier le code et redéployer

Approche Autorisation centralisée

```
If (User.canAccess("Manager Resource") {
  // can access the protected resource
}
```

=> aucune référence à un mécanisme de contrôle d'accès spécifique ; le contrôle d'accès est basé sur la ressource.

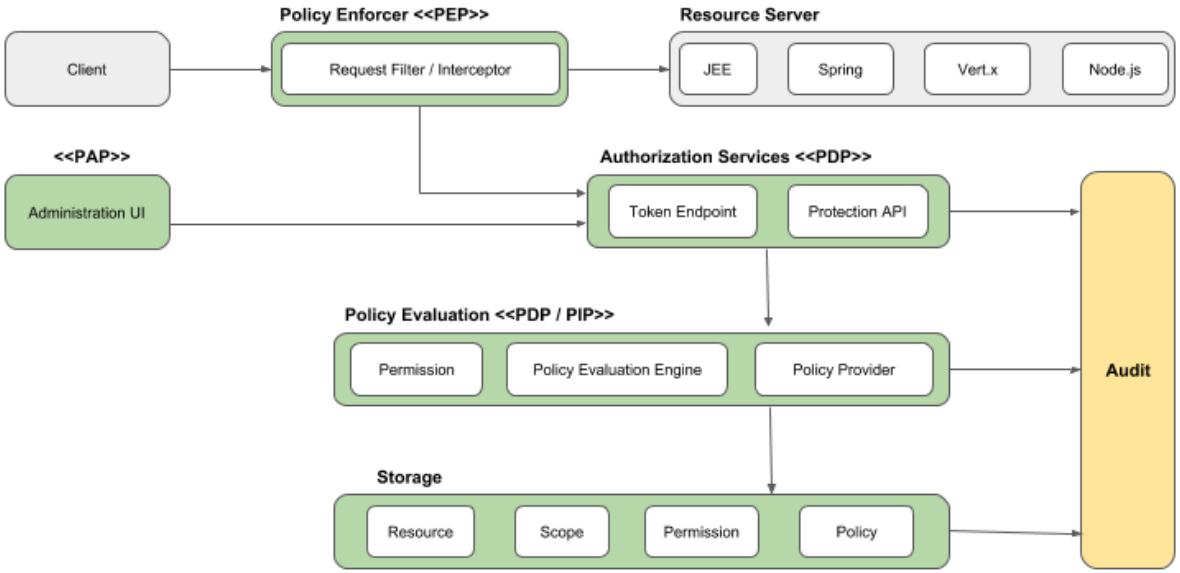
=> Les modifications apportées à l'accès à *Manager Resource* se définissent dans Keycloak et n'ont pas d'impact sur le code applicatif

Fonctionnalités

Keycloak autorisation services est capable de combiner différent mécanisme de contrôle d'accès :

- ABAC
- RBAC
- UBAC (User-based access control)
- CBAC (Context-based access control)
- Rule-based access control
- Utilisation de JavaScript pour évaluer les décisions
- Time-based access control
- Custom Access Control via un SPI

Architecture



Configuration

La configuration s’effectue en 3 étapes :

1) **Définition des ressources** via la console d’admin ou l’API:
   Resource Server <=> Client
   Ressources <=> Motif d’URI
   Scopes (Optionnel) <=> Action sur la ressource

2) **Définition des *policy* et des permissions** via la console d’admin ou l’API
   policy : conditions devant être satisfaites
   permissions : Associe une Ressource et éventuellement un scope à un ensemble de *policies*

3) **Appliquer les ACLs** sur le serveur de ressources. Ceci est réalisé en activant un point d'application (PEP) sur le serveur de ressources qui est capable de communiquer avec le serveur d'autorisation.
   Keycloak fournit des implémentations pour ses clients Adapter

Policy Enforcer Point

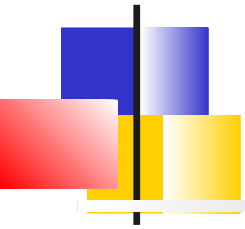
Un PEP est un filtre ou intercepteur dans l'application qui vérifie les ACLs.

Les autorisations sont appliquées en fonction du protocole que l'on utilise :

- Si UMA[1], le PEP attend un jeton particulier contenant les permissions résolues : le RPT[2]
  Le RPT s'obtient avec un jeton d'accès standard
- Sinon, le PEP envoie le jeton d'accès standard au serveur d'autorisation pour résoudre les ACLs

1. https://docs.kantarainitiative.org/uma/wg/rec-oauth-uma-grant-2.0.html
2. Requesting Party Token





Adaptation et personnalisation

Organisation

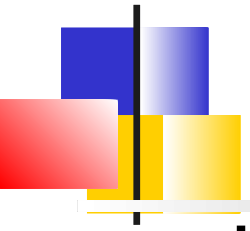
Flows d’authentification

Thèmes

Fédération d’identités

SPI

Admin API



Introduction

- Une organisation relativement classique consiste à avoir :
  - 1 ou plusieurs super-admin
  - Des administrateurs dédiés à des realms
  - Éventuellement, parmi les administrateurs dédiés des sous-rôles déterminant des permissions limitées

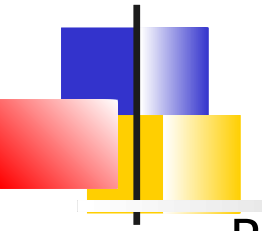


Attribution des droits d'administration

Chaque realm dispose de son propre client ***realm-management***, qui contient des rôles d'administration.

Un utilisateur devient admin d'un realm lorsqu'il reçoit un ou plusieurs rôles de ce client.

- ***realm-admin*** : Admin complet sur le realm
- ***manage-users*** :Gérer les utilisateurs
- ***view-clients*** : Lire les clients
- ***manage-roles*** : Gérer les rôles



Organiser realms et clients

Realm master réservé à l’administration du serveur

Un realm = un domaine de sécurité : utilisateurs, SSO, politiques

- Un realm par population (employés, clients, partenaires) plutôt que par application
- Pas de SSO entre realms

Environnements (dev, recette, prod) : instances séparées de préférence

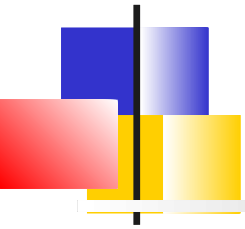
Éviter la multiplication des realms (démarrage, mémoire, console)

Clients :

- Convention de nommage des client_id
- Client scopes partagés plutôt que mappers dupliqués

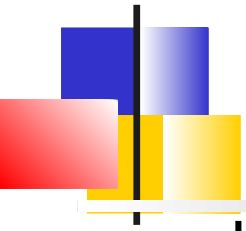
Organizations

- Fonctionnalité supportée depuis Keycloak 26 : multi-tenant B2B dans un seul realm
- Une organisation regroupe :
  - Des membres (gérés ou issus d’un IdP)
  - Des domaines de messagerie
  - Des fournisseurs d’identité associés
- Authentification identity-first : redirection vers l’IdP de l’organisation selon le domaine de l’email
- Client scope organization : claim organization dans les jetons
- Activation : Realm settings → Organizations



Adaptation et personnalisation

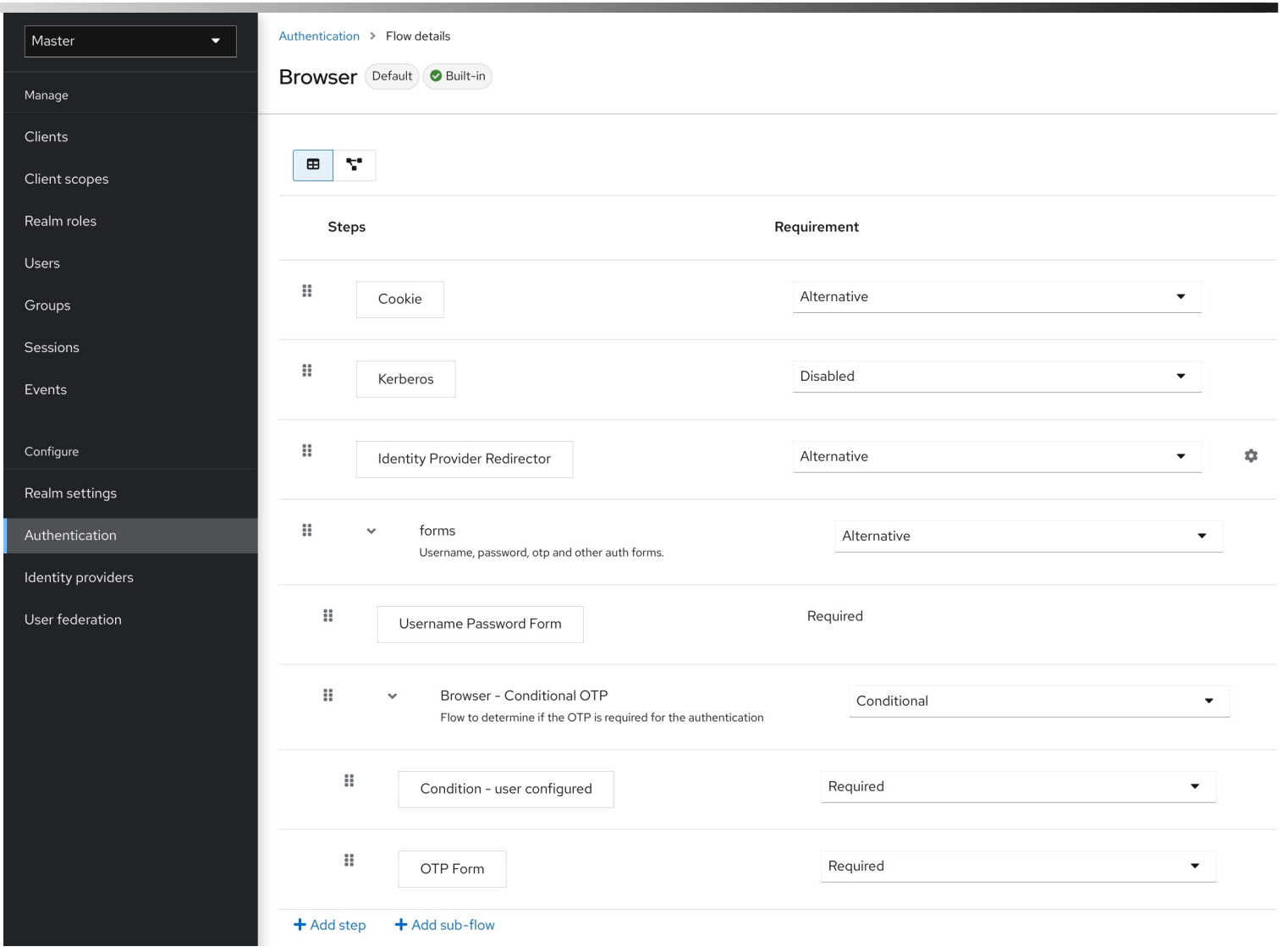
Organisation
Flows d’authentification
Thèmes
Fédération d’identités
SPI
Admin API



Flux d’authentification

- Keycloak permet de configurer toutes les étapes d’un processus d’authentification (Authentication Flow)
  - Le processus englobe des actions, des écrans de saisie,...
  - Certains flows sont prédéfinis.
    Les étapes ne peuvent pas être redéfinies, on peut juste les marquer comme obligatoire et facultative
  - On peut redéfinir un processus d’authentification complet et l’associer à un cas d’usage
- Les flows d’authentification sont définies au niveau realm, peuvent être surchargés au niveau client ou associés à des client policies.

Exemple Browser Flow



Attributs des étapes

Des radio buttons contrôlent l'exécution des étapes du flow :

- ***Required*** : Tous les éléments requis du flux doivent être exécutés séquentiellement avec succès.
  Le flux se termine si un élément requis échoue.
- ***Alternative*** : Un seul élément doit s'exécuter avec succès pour que le flux soit évalué comme réussi.
  => Tout élément alternative dans un flux contenant des éléments requis ne s'exécutera pas.
- ***Disabled*** : L'élément n'est pas pris en compte pour marquer un flux comme réussi.
- ***Conditional*** : Seulement sur des sous-flux
  - Contient des exécutions qui doivent correspondre à des instructions logiques.
  - Si toutes les exécutions sont évaluées comme vraies, le sous-flux conditionnel agit comme requis.
  - Si toutes les exécutions sont évaluées comme fausses, le sous-flux conditionnel agit comme désactivé.

Exemple Browser Flow

Cookie : La première fois qu'un utilisateur se connecte avec succès, Keycloak définit un cookie de session. Si le cookie est déjà défini, ce type d'authentification est réussi. Étant donné que le fournisseur de cookies a renvoyé un succès et que chaque exécution à ce niveau du flux est ALTERNATIVE, Keycloak n'effectue aucune autre exécution. => connexion réussie.

Kerberos : Cet authentificateur est désactivé par défaut et est ignoré pendant le flux du navigateur. On peut l’activer lors de l’authentification Kerberos

Identity Provider Redirector : Redirige vers un autre fournisseur d'identité

Form : Sous-flux alternatif qui contient un type d'authentification supplémentaire qui doit être exécuté. Keycloak charge les exécutions pour ce sous-flux et les traite.

Sous-flux Form

Username Password Form : la page de login et de mot de passe. Onligatoire (REQUIRED) => nom d'utilisateur et un mot de passe valides.

Browser - ***Conditional OTP*** : Sous-flux conditionnel en fonction du résultat de condition d'exécution configuré par l'utilisateur.

Condition - User Configured authentication : Vérifie si l'utilisateur dispose d'informations d'identification OTP configurées.

OTP form : REQUIRED mais elle ne s'exécute que lorsque l'utilisateur dispose d'un identifiant OTP configuré.

acr

acr (***Authentication Context Class Reference)*** est un claim standard qui indique le niveau ou le contexte de l'authentification utilisée pour connecter un utilisateur.

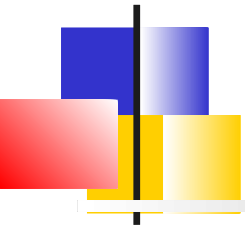
Avec Keycloak, on peut associer (au niveau realm ou au niveau client) des valeurs d’acr à des entiers : les ***LoA (Level Of Authentication)***

Il est alors possible de mettre au point un flow d’authentification avec une étape conditionnel dépendant du *LoA*



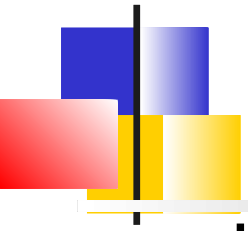
Scénario

1. Lors de la requête OIDC initiale, le client précise une valeur du paramètre acr_values. Ex :
   `acr_values=urn:myorg:mfa`
2. Keycloak mappe cette valeur à un LoA
3. Le flow d'authentification qui s'applique dépend du LoA
4. A l'issue de l'authentification, le claim ***acr*** est dans l'ID Token



Adaptation et personnalisation

Organisation
Flows d’authentification
Thèmes
Fédération d’identités
SPI
Admin API



Introduction

Keycloak permet de personnaliser les pages utilisés par les utilisateurs via des thèmes :

- Formulaires de login
- Pages de l’application account
- Email
- Pages d’accueil Keycloak
- Console d’administration

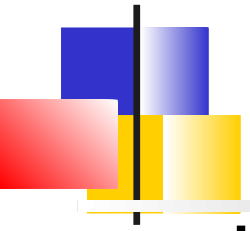
Il existe des thèmes prédéfinis et il est possible de développer son propre thème

Configuration

La configuration s’effectue via la console d’admin
Realm Settings → Theme

Un thème peut également être précisé au démarrage du serveur :

```
bin/kc.[sh|bat] start --spi-theme-welcome-theme=custom-theme
```



Eléments d’un thème

Un thème est constitué de :

- De gabarits freemarker
- D’images
- De messages localisés (bundle)
- De feuilles de style css
- De script
- De propriétés

Héritage

La création s'effectue généralement en étendant :

- Le thème ***Keycloak*** qui contient des images et les feuilles de style si l'on veut modifier les gabarits (ajout de lien par exemple)
- Ou le thème ***base*** qui contient les gabarits et les messages si l'on veut travailler le look and feel

Après avoir étendu un thème on peut surcharger ses éléments

Lors de la création de thème, il est recommandé de désactiver la fonctionnalité de cache pour éviter de redémarrer le serveur

Procédure

1) Désactivation du cache :

```
bin/kc.[sh|bat] start --spi-theme-static-max-age=-1 --spi-theme-cache-themes=false –spi-theme-cache-templates=false
```

2) Création d’un sous-répertoire dans le répertoire ***themes***

3) Puis création de répertoire pour chaque type de théme (ex. /login pour la page de login)

4) Pour chaque type créer un fichier ***theme.properties***

Propriétés de thème

parent : Le thème parent à étendre

import : Importer des ressources d'un autre thème

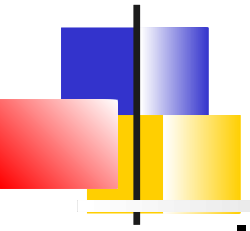
common : Surcharge le chemin vers les ressources communes. (par défaut est *common/keycloak*) Utilisé dans les gabarits *Freemarker* via ${url.resourcesCommonPath}

styles/scripts : Listes de styles/script à inclure. Les fichiers sont placés dans */resources/css* | */resources/js* du répertoire du thème

locales : Liste des langues supportés

Des propriétés sont également utilisées pour changer les classes css de certains éléments

Il est également possible de définir des propriétés custom qui seront utilisées dans les gabarits et d'utiliser des propriétés JVM ou de l'environnement



Création de gabarit

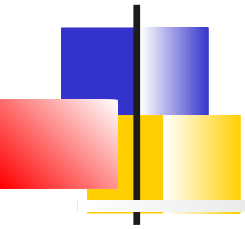
- Il est possible de surcharger un gabarit du parent en plaçant un gabarit freemarker à la racine du répertoire
- La liste des gabarits que l’on peut surcharger est visible en ouvrant le jar `lib/lib/main/org.keycloak.keycloak-themes-22.0.5.jar` présent dans la distribution

Déploiement du thème

Les thèmes peuvent être déployés

- En copiant le répertoire de thème dans le répertoire *themes* de la distribution
- En créant une archive qui contient les ressources + un fichier *META-INF/keycloak-themes.json* listant les fichiers ressources du jar et en plaçant l'archive dans le répertoire *providers* de la distribution





Adaptation et personnalisation

Organisation
Flows d’authentification
Thèmes
Fédération d’identités
SPI
Admin API

Intégration LDAP ou Active Directory

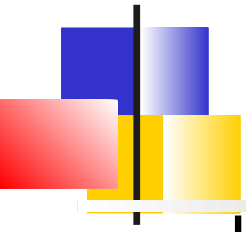
Keycloak peut s'intégrer à LDAP de 2 façons :

- Les données de l’annuaire LDAP sont importées dans la base de données Keycloak, et synchronisées 3 modes de synchronisation possible : READ_ONLY, WRITABLE et UNSYNCED
- La vérification des informations d'identification sont déléguées à LDAP.

Il est possible de configurer plusieurs annuaires LDAP au sein d'un même realm et de configurer un ordre de priorité.

Lors de l’intégration, des mappers sont configurés pour faire correspondre les champs LDAP aux attributs des user keycloak, en particulier les groupes et les rôles

Fédération LDAP / Active Directory



User federation → Add LDAP providers

- Connexion : URL, bind DN, users DN, filtre
- Edit mode : READ_ONLY, WRITABLE, UNSYNCED
- Import users : copie locale ou consultation à la volée

Synchronisation périodique : complète ou utilisateurs modifiés

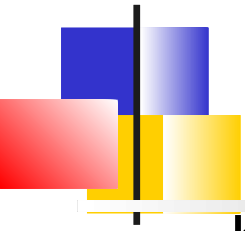
Mappers LDAP : user-attribute, full-name, group-ldap-mapper, role-ldap-mapper, msad-user-account-control (AD)

Kerberos (SPNEGO) optionnel pour le SSO poste de travail

Autres fournisseurs d’identité

Keycloak peut s'intégrer à des fournisseurs d'identité tiers en utilisant SAML v2 ou OpenID Connect v1.0 ou aux « social providers » (Google, Github, ...)

A leur 1ère authentification, les utilisateurs sont importés dans Keycloak.
Ils peuvent alors profiter de toutes les fonctionnalités fournies par Keycloak et respecter les contraintes de sécurité imposées par votre royaume



Identity brokering

- Keycloak délègue l’authentification à un fournisseur externe puis émet ses propres jetons
  - Applications inchangées : elles ne connaissent que Keycloak
- Fournisseurs : OpenID Connect, SAML 2.0, sociaux (Google, GitHub, Microsoft…)
- Identity providers → Add provider
  - Alias et redirect URI : /realms/{realm}/broker/{alias}/endpoint
  - Trust email, Hide on login page, Sync mode (Import, Legacy, Force)
- Paramètre kc_idp_hint : redirection directe vers un IdP

First broker login

Flow exécuté lors de la 1ère connexion via un IdP externe

- Review profile : complétion des informations manquantes
- Création de l'utilisateur local
- Ou liaison avec un compte existant : confirmation, vérification par email ou ré-authentification

Post login flow : étapes supplémentaires à chaque connexion (ex. OTP)

Comptes liés visibles dans la console du compte utilisateur

Flows personnalisables comme les autres flows d'authentification

Mappers des fournisseurs d’identité

- Transforment les informations reçues de l’IdP
  - Attribute importer : claim ou attribut SAML → attribut utilisateur
  - Username template importer : construction du username
  - Claim to role, Advanced claim to role, SAML attribute to role
  - Hardcoded role, Hardcoded attribute, Hardcoded group
- Sync mode par mapper : appliqué à la création ou à chaque connexion
- Les rôles importés peuvent ensuite être exposés aux applications via les mappers des clients

Intégration avec un BD spécifique

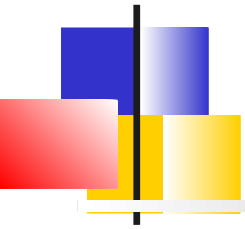
Si on veut conserver les utilisateurs dans son propre schéma relationnel !

Plusieurs alternatives :

- Implémenter un SPI en Java implémentant l'interface ***UserStorageProvider***. Déployer ce provider dans Keycloak Les utilisateurs ne sont plus présents dans keycloak[1]
- Utiliser l'API Rest de Keycloak. Et implémenter une synchronisation des users



1. https://github.com/keycloak/keycloak-quickstarts/tree/latest/extension/user-storage-jpa



Adaptation et personnalisation

Organisation
Flows d’authentification
Thèmes
Fédération d’identités
SPI
Admin API

Introduction

Les ***SPI (Service Provider Interfaces)*** sont des extensions personnalisées qui permettent aux développeurs d'étendre, de personnaliser et d'intégrer Keycloak avec d'autres systèmes ou services.

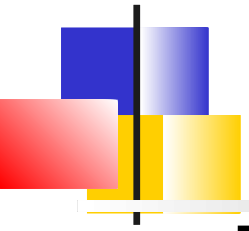
Les SPI offrent une flexibilité et une extensibilité considérables pour adapter Keycloak à des cas d'utilisation spécifiques

SPIs disponibles

Les principales SPIs sont :

- ***Authenticator SPI*** : Étendre les flows d'authentification
- ***User Storage SPI*** : Stocker les utilisateurs dans un système de persistance personnalisé
- ***Federated Identity Provider SPI*** : Pour intégrer des fournisseurs d'identité externes
- ***Protocol Mapper SPI*** : Pour ajouter des informations personnalisés dans les jetons
- ***Event Listener SPI*** : Pour intégrer des listeners des événements Keycloak
- ***Action Token Handler SPI*** : Des jetons permettant des actions côté Keycloak (Réintialiser les mots de passe, Confirmer une adresse e-mail, Executer des required action(s))
- ***Vault SPI :*** Connexion à une implémentation Vault
- ...

1. Des exemples sont disponibles à https://github.com/keycloak/keycloak-quickstarts/tree/latest/extension



Implémentation

Pour implémenter une SPI, il faut :

- Implémenter 2 interfaces ***ProviderFactory*** et ***Provider***
- Créer un fichier de configuration du service qui référence la factory

Interface factory

Le serveur *Keycloak* instancie la factory comme singleton et appelle ses méthodes de callback *init()*, *postInit()*, *close()*.

La responsabilité de la factory est de créer l'implémentation du provider lors de sa méthode *create()* et de fournir un identifiant :

- Unique
- Ou surchargeant un service existant

Exemple *ThemeSelectorProviderFactory*

```
public class MyThemeSelectorProviderFactory implements ThemeSelectorProviderFactory {

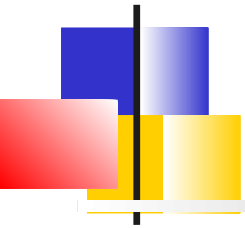
    @Override
    public ThemeSelectorProvider create(KeycloakSession session) {
        return new MyThemeSelectorProvider(session);
    }

    @Override
    public void init(Config.Scope config) {
    }

    @Override
    public void postInit(KeycloakSessionFactory factory) {
    }

    @Override
    public void close() {
    }

    @Override
    public String getId() {
        return "myThemeSelector";
    }
```



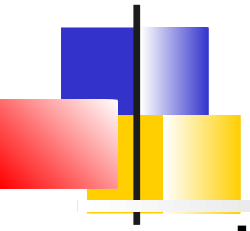
Exemple *ThemeSelectorProvider*

```
public class MyThemeSelectorProvider implements
   ThemeSelectorProvider {


     private KeycloakSession session;


     public MyThemeSelectorProvider(KeycloakSession
   session) {
          this.session = session;
     }


     @Override
     public String getThemeName(Theme.Type type) {
          return
   session.getContext().getRealm().getLoginTheme();
     }
}
```



Fichier de configuration

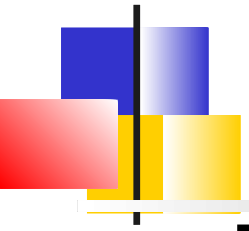
Le fichier de configuration se place ***META-INF/services/*** et se nomme en fonction de la SPI

Par ex :

```
META-INF/services/org.keycloak.theme.ThemeSelectorProviderFactory
```

Il référence l'implémentation :

```
com.acme.provider.MyThemeSelectorProviderFactory
```



Déploiement

Placer le jar dans le répertoire **providers/**

Reconstruire le serveur :

- Automatique au démarrage sans --optimized
- Explicite avec ./bin/kc.sh build pour une image optimisée

En conteneur : copier le jar dans /opt/keycloak/providers/ avant l'étape de build du Dockerfile

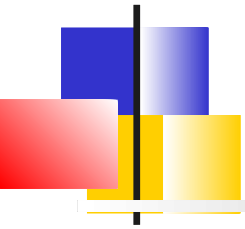
Affichage dans la console

Afin que la SPI développée s'affiche dans la page server info de la console d'administration, il faut implémenter l'interface ***ServerInfoAwareProviderFactory*** dans la classe factory

```
public class MyThemeSelectorProviderFactory implements
ThemeSelectorProviderFactory, ServerInfoAwareProviderFactory {
 ...


 @Override
 public Map<String, String> getOperationalInfo() {
     Map<String, String> ret = new LinkedHashMap<>();
     ret.put("theme-name", "my-theme");
     return ret;
 }
}
```





Adaptation et personnalisation

Organisation
Flows d’authentification
Thèmes
Fédération d’identités
SPI
Admin API

Introduction

- Keycloak propose une API Rest d'administration
- Pour pouvoir l'invoquer, il faut obtenir un jeton avec les permissions adéquates
  - En utilisant le client ***admin-cli*** et le login d'un compte administrateur
  - En créant, un client spécifique qui inclut dans son token d'accès une audience ***security-admin-console*** et un rôle ***admin***[1]

1. Onglet service account roles

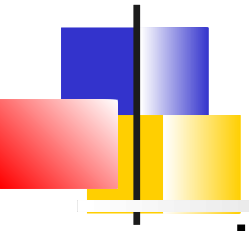
Obtention du jeton

Jeton via l'admin-cli

```
curl \
-d "client_id=admin-cli" \
-d "username=admin" \
-d "password=password" \
-d "grant_type=password" \
"http://localhost:8080/realms/master/protocol/openid-connect/token"
```

Jeton via client credentials après avoir configuré un Audience Mapper

```
curl \
-d "client_id=<YOUR_CLIENT_ID>" \
-d "client_secret=<YOUR_CLIENT_SECRET>" \
-d "grant_type=client_credentials" \
"http://localhost:8080/realms/master/protocol/openid-connect/token"
```



Admin API

L’API permet de :

- Manipuler les realms
- Manipuler les users / groupes /realm roles
- Les clients/ client scopes / rôles associés aux clients
- Les mappers
- ...
- Mais également certains endpoint pour détecter des attaques brute force

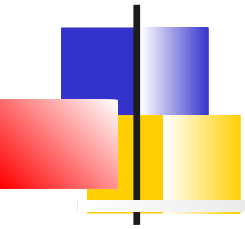
kcadm.sh

kcadm.sh est l'outil en ligne de commande officiel pour administrer Keycloak via son API REST.

Il permet de gérer les utilisateurs, rôles, clients, groupes, etc.

Utile pour automatiser les tâches d'administration dans des scripts ou pipelines CI/CD.

Livré avec chaque distribution Keycloak (présent dans /bin).



Authentification

Avant l’utilisation de l’API rest, se connecter à l’instance Keycloak

```
kcadm.sh config credentials \
  --server http://localhost:8080 \
  --realm master \
  --user admin \
  --password admin
```

Quelques commandes

```
# Lister les utilisateurs du realm 'myrealm'
kcadm.sh get users -r myrealm

# Créer un utilisateur
kcadm.sh create users -r myrealm \
  -s username=testuser -s enabled=true

# Ajouter un rôle à un utilisateur
kcadm.sh add-roles -r myrealm \
  --uusername testuser \
  --rolename user
```

Possibilité de chainer des appels et de manipuler des IDs via *jq* ou *grep*.

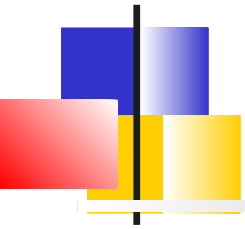
Fichiers json

```
# Importation d'un realm
kcadm.sh create realms -f realm-export.json

# Création d'un user
kcadm.sh create users -r monrealm -f user.json
kcadm.sh set-password -r monrealm --userid
  $USER_ID --new-password monpass123

# Création d'un client
kcadm.sh create clients -r monrealm -f client.json
```





Vers la production

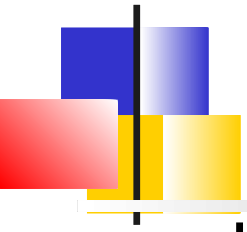
Configuration et optimisation du démarrage

Configuration de production

Clustering

Sécurisation Keycloak

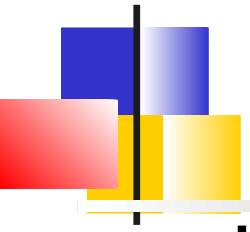
Comptes utilisateur et applications



Sources de configuration

Keycloak charge sa configuration à partir de quatre sources, par ordre de précédence :

- Paramètres de ligne de commande
  `--<key-with-dashes>=<value>`
  ex : `--db-url=<value>`
- Variables d'environnement
  `KC_<key_with_underscores>=<value>`
  ex : `KC_DB_URL=<value>`
- Fichier *.conf* créé par l'utilisateur
- Fichier *keycloak.conf* situé dans le répertoire conf.
  `<key-with-dashes>=<value>`
  ex : `db-url=<value>`



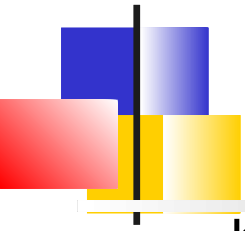
Fichiers de configuration

Les fichiers de configuration peuvent faire référence à des variables d’environnement et à des valeurs par défaut :

```
db-url-host=${MY_DB_HOST:mydb}
```

Un fichier de configuration dédié peut être fourni au démarrage :

```
bin/kc.[sh|bat] --config-file=/path/to/myconfig.conf start
```



Démarrage

Keycloak peut être démarré en 2 modes :

Démarrage en mode dev

```
bin/kc.[sh|bat] start-dev
```

HTTP est activé
La résolution stricte du nom d'hôte est désactivée
Le cache est défini sur *local*
La mise en cache des thèmes et des modèles est désactivée (utile pour le développement de thème)

Démarrage en mode production

```
bin/kc.[sh|bat] start
```

HTTP est désactivé
Configuration du Hostname requise
HTTPS/TLS requis

Utilisateurs admin

Au 1er démarrage, création d'un administrateur temporaire :

```
KC_BOOTSTRAP_ADMIN_USERNAME=admin
KC_BOOTSTRAP_ADMIN_PASSWORD=change_me
```

Les anciennes variables KEYCLOAK_ADMIN / KEYCLOAK_ADMIN_PASSWORD sont dépréciées

La console signale le caractère temporaire du compte

Récupération d'accès : ./bin/kc.sh bootstrap-admin user

Ensuite, créer des comptes nominatifs via la console ou kcadm et supprimer le compte temporaire

Optimisation du démarrage

Il est recommandé d'optimiser Keycloak pour de meilleurs temps de démarrage et une meilleure consommation de mémoire avant de déployer dans des environnements de production.

Lors de l'utilisation des commandes *start* ou *start-dev*, Keycloak[1] exécute une commande de **build** qui peut prendre du temps.
=> Pour optimiser, exécuter cette commande de build avant le démarrage en fixant la plupart des valeurs de configuration, mettre au point un fichier *keycloak.conf* contenant les clé de configuration définies au runtime puis démarrer avec l'option ***optimized***.

1. Fonctionnement du framework Quarkus : la configuration est résolue lors d'une phase de build nommé augmentation

Exemple optimisation

Phase de build

```
bin/kc.[sh|bat] build --db=postgres
```

Keycloak.conf :

```
db-url-host=keycloak-postgres
db-username=keycloak
db-password=change_me
hostname=mykeycloak.acme.com
https-certificate-file
```

Démarrage du serveur

```
bin/kc.[sh|bat] start --optimized
```

Cas d’un container

L'image du conteneur Keycloak par défaut est livrée prête à être configurée et optimisée.

On peut donc inclure la phase de build de Keycloak durant la construction du conteneur

Exemple Dockerfile

```
FROM quay.io/keycloak/keycloak:latest as builder

# Enable health and metrics support
ENV KC_HEALTH_ENABLED=true
ENV KC_METRICS_ENABLED=true

# Configure a database vendor
ENV KC_DB=postgres

WORKDIR /opt/keycloak
# for demonstration purposes only, please make sure to use proper certificates in production instead
RUN keytool -genkeypair -storepass password -storetype PKCS12 -keyalg RSA -keysize 2048 -dname "CN=server" -alias server -
     ext "SAN:c=DNS:localhost,IP:127.0.0.1" -keystore conf/server.keystore
RUN /opt/keycloak/bin/kc.sh build

FROM quay.io/keycloak/keycloak:latest
COPY --from=builder /opt/keycloak/ /opt/keycloak/

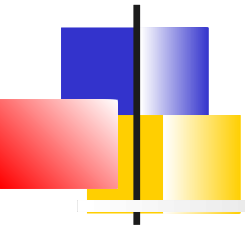
# change these values to point to a running postgres instance
ENV KC_DB_URL=<DBURL>
ENV KC_DB_USERNAME=<DBUSERNAME>
ENV KC_DB_PASSWORD=<DBPASSWORD>
ENV KC_HOSTNAME=localhost
ENTRYPOINT ["/opt/keycloak/bin/kc.sh"]
```

Build et démarrage

Après avoir mis au point le fichier *Dockerfile*

```
podman|docker build . -t mykeycloak
```

```
podman|docker run --name mykeycloak -p 8443:8443 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=change_me \
        mykeycloak \
        start --optimized
```



Vers la production

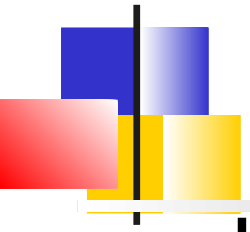
Configuration et optimisation du démarrage

Configuration de production

Clustering

Sécurisation Keycloak

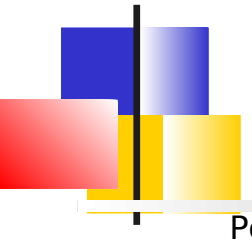
Comptes utilisateur et applications



Configuration de production

La configuration de production nécessite :

- De configurer **TLS** pour les communications http
- Configurer le **hostname**
- Mettre en place un **reverse proxy**
- Mettre en place le **clustering**
- Configurer une **base de données de production**



TLS

Pour charger les certificat, Keycloak s'appuie sur des fichiers PEM ou des keystore

PEM

```
bin/kc.[sh|bat] start --https-certificate-file=/path/to/certfile.pem
    --https-certificate-key-file=/path/to/keyfile.pem
```

Keystore

L'emplacement par défaut est conf/keystore

```
bin/kc.[sh|bat] start –https-key-store-password=<value>
```

Keycloak utilise le port 8443, cela peut être changé avec :

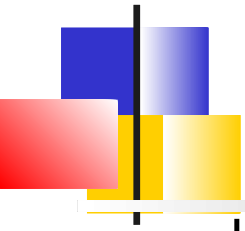
```
bin/kc.[sh|bat] start –https-port=<port>
```

L'authentification du client est également supportée :

```
bin/kc.[sh|bat] start --https-trust-store-file=/path/to/file
    --https-trust-store-password=<value>
    --https-client-auth=<none|request|required>
```

Types d’endpoints et hostname

- Keycloak expose différents type d’endpoints :
  - **Frontend** : Accessibles via un domaine public et généralement liés aux flux qui passent par le canal front.
    Par exemple, le *authorization_endpoint*
    Clé de configuration : ***hostname***
  - **Backend** : Accessible via un domaine public ou un réseau privé. Ils sont utilisés pour une communication directe entre le serveur et les clients.
    Par exemple : *Token introspection, User info, Token endpoint, JWKS*
    Clé de configuration : ***hostname-strict-backchannel*** :
    - ***true :*** Ces endpoints utiliseront l’adresse public
    - ***false :*** Ces endpoints utilisent l’adresse privé d’écoute
  - **Console d'administration** : Généralement la console d’administration n’est pas accessible publiquement.
    Clé de configuration : ***hostname-admin***



Reverse proxy

L’ancienne option proxy (edge, reencrypt, passthrough) est supprimée

--proxy-headers=xforwarded ou forwarded : prise en compte des en-têtes du proxy

- Nécessaire pour calculer correctement les URLs et l’IP cliente

--http-enabled=true si le TLS est terminé au proxy (ancien mode edge)

TLS ré-établi entre proxy et Keycloak : configurer aussi les certificats sur Keycloak

TLS passthrough : ne pas activer proxy-headers

--proxy-trusted-addresses : limiter les proxies autorisés à positionner ces en-têtes

Configuration BD

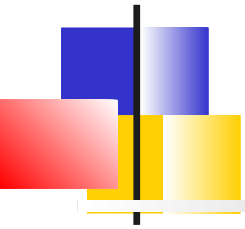
Options basiques :

```
bin/kc.[sh|bat] start
--db postgres --db-url-host mypostgres
--db-username myuser --db-password change_me
```

Le schéma par défaut est *keycloak*, peut être modifié via *db-schema*.

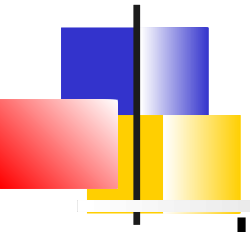
Bases supportées : *mariadb, mssql, mysql, oracle, postgres*





Vers la production

Configuration et optimisation du démarrage
Configuration de production
Clustering
Sécurisation Keycloak
Comptes utilisateur et applications

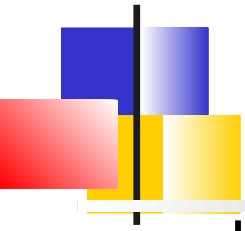


Introduction

Le clustering apporte :

- Haute disponibilité (HA)
- Scalabilité horizontale
- Réplication de sessions (SSO)
- Synchronisation du cache utilisateur, tokens, sessions

Keycloak quarkus s’appuie sur 2 sous-projet Redhat indépendant JGroups et Infinispan



JGroups

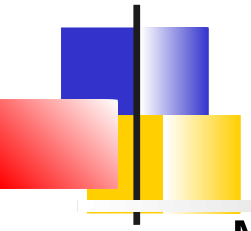
Librairie Java de communication réseau peer-to-peer

Utilisée pour la découverte des nœuds et le transport des messages

S’appuie sur différents protocoles de transport : UDP, TCP, TCPPING, KUBE_PING, etc.

Fonctions clés :

- Formation du cluster et découverte des membres
- Détection de partition réseau
- Réplication des messages de cache



Infinispan – Cache distribué

Moteur de cache mémoire hautes performances

Fournit un cache distribué et répliqué

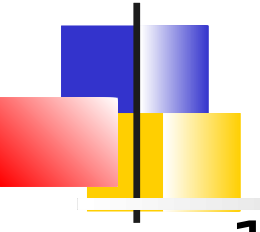
Utilisé par Keycloak pour :

- Sessions utilisateur
- Tokens
- Métadonnées clients et utilisateurs

Propose différents types de cache :

- local (non répliqué)
- repliqué (synchronisation entre tous les nœuds)
- distribué (partitionné entre nœuds)

Flux de synchronisation dans un cluster



1) L’utilisateur s’authentifie sur un nœud

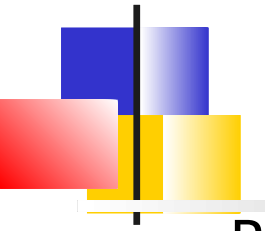
2) La session est stockée dans le cache Infinispan

3) JGroups transmet la mise à jour aux autres nœuds

4) Les autres nœuds peuvent répondre à l’utilisateur sans redirection

La Sticky session n’est pas requise si le cache est bien configuré.





Load balancer

Répartition devant les nœuds Keycloak :

- Sessions affines recommandées : la route du nœud est portée par le cookie AUTH_SESSION_ID
- Health checks sur /health/ready (port de management 9000)

Côté Keycloak :

- --proxy-headers=xforwarded ou forwarded
- --hostname avec l'URL publique
- --http-enabled=true si TLS terminé au load balancer

N'exposer publiquement que les chemins nécessaires : /realms/, /resources/

- /admin/ réservé au réseau interne

Exemple HAProxy

```
frontend fe_keycloak
  bind *:443 ssl crt /etc/haproxy/certs/auth.pem
  option forwardfor
  http-request set-header X-Forwarded-Proto https
  http-request deny if { path_beg /admin }
  default_backend be_keycloak

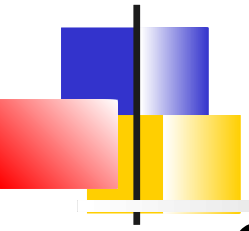
backend be_keycloak
  balance roundrobin
  cookie KC_ROUTE insert indirect nocache
  option httpchk GET /health/ready
  server kc1 10.0.0.11:8080 check port 9000 cookie kc1
  server kc2 10.0.0.12:8080 check port 9000 cookie kc2
```

Keycloak démarré avec --proxy-headers=xforwarded --http-enabled=true --health-enabled=true



Base de données externe

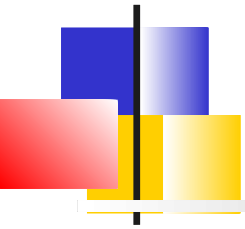
- Base partagée par tous les nœuds : point de défaillance à rendre HA
  - Réplication PostgreSQL, service managé, sauvegardes
- Sessions utilisateur persistées en base (défaut Keycloak 26) : dimensionner la BD en conséquence
- Pool de connexions :
  - --db-pool-initial-size, --db-pool-min-size, --db-pool-max-size
- Découverte des nœuds du cluster :
  - --cache=ispn --cache-stack=jdbc-ping (via la base) ou kubernetes (DNS)



Keycloak Operator

- Opérateur Kubernetes officiel : déploiement, configuration, montée de version
- CRD Keycloak et KeycloakRealmImport

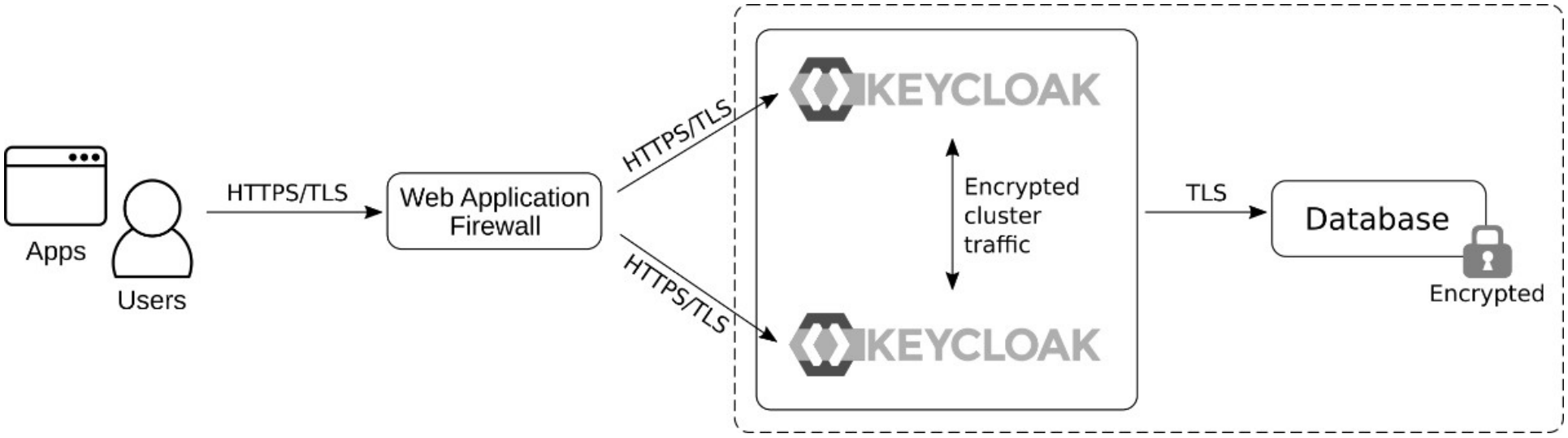
```
apiVersion: k8s.keycloak.org/v2alpha1
kind: Keycloak
metadata:
  name: keycloak
spec:
  instances: 3
  db:
    vendor: postgres
    host: postgres-rw
    usernameSecret: { name: kc-db, key: username }
    passwordSecret: { name: kc-db, key: password }
  http:
    tlsSecret: keycloak-tls
  hostname:
    hostname: auth.example.com
  proxy:
    headers: xforwarded
```



Vers la production

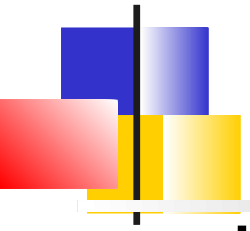
Configuration et optimisation du démarrage
Configuration de production
Clustering
Sécurisation Keycloak
Comptes utilisateur et applications

Architecture



Sécurisation Keycloak

- TLS end 2 end
  - Utiliser la dernière version de TLS (TLS 1.3) et s'assurer que les librairies d'intégration choisies le supporte
  - Configuration de Keycloak et du proxy en TLS passthrough
- Configurer le hostname,
  - sinon Keycloak détermine le hostname à partir de l'entête HTTP Host
    => Un attaquant peut facilement le tromper
- Rotation des clés de signature utilisées par Keycloak
  - Realm Settings → Keys : Par exemple 1 fois par mois
- Mettre à jour régulièrement Keycloak
- Chargement des secrets utilisés par Keycloak à partir d'un vault externe
- Protéger Keycloak avec un pare-feu et un système de prévention d'intrusion

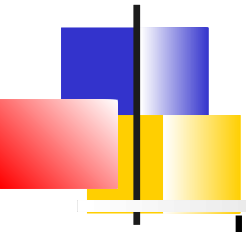


Sécuriser la BD

Keycloak stocke beaucoup de données sensibles dans sa base de données
On doit sécuriser la BD et ses backups

Recommandations minimales :

- Protéger la bd avec un pare-feu
- Activer l'authentification et le contrôle d'accès
- Crypter les backups



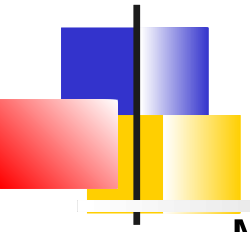
Cluster

Les informations concernant les sessions utilisateur sont conservées en mémoire.

Lors d’une architecture en cluster, les nœuds répliquent ses informations (InfiniSpan)

Même si ses informations ne sont pas aussi sensibles que celles de la base de données, il est recommandé de sécuriser ces échanges entre nœud Keycloak

- Autoriser l’authentification. Les nœuds doivent s’authentifier pour faire partie du cluster.
- Crypter les communications : Configuration JGroups



Paramètres JVM et performances

- Mémoire : par défaut dimensionnée en pourcentage de la mémoire disponible (MaxRAMPercentage)
  - Adapter via JAVA_OPTS_APPEND, fixer les limites du conteneur
- CPU : le hachage des mots de passe est le principal consommateur
  - Nombre d’itérations de la politique de mot de passe à arbitrer
- Mémoire : caches de sessions et de configuration
- Démarrage : image optimisée (kc.sh build, --optimized)
- Base de données : taille du pool, index, latence réseau
- Mesurer avant d’ajuster : métriques JVM et HTTP (voir supervision)

Configuration des logs

Destinations : --log=console,file,syslog

Niveaux par catégorie :

```
--log-level="INFO,org.keycloak.authentication:debug"
```

Format structuré pour la collecte :

```
--log-console-output=json
--log-file=/var/log/keycloak/keycloak.log --log-file-output=json
```

Rotation : fichier via logrotate ou collecte par la plateforme (conteneurs)

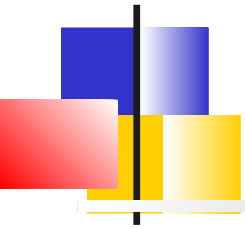
En production : INFO par défaut, DEBUG ciblé et temporaire

En-têtes et défenses

- Realm settings → Security defenses → Headers
  - Content-Security-Policy, X-Frame-Options : protection contre le clickjacking
  - X-Content-Type-Options, X-Robots-Tag
  - Strict-Transport-Security : HTTPS obligatoire
- Brute force detection (même onglet)
  - Nombre maximal d’échecs, verrouillage temporaire ou permanent
- Clients : redirect URIs exactes sans joker, web origins restreintes

Comptes administrateurs

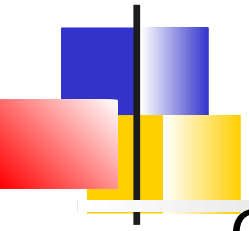
- Compte bootstrap temporaire au 1er démarrage, à remplacer par des comptes nominatifs
- Realm master limité aux super-administrateurs
- Administrateurs de realm via les rôles du client realm-management
- MFA obligatoire pour les comptes d’administration
- Console d’administration non exposée publiquement : --hostname-admin, filtrage au proxy
- Comptes de service dédiés pour l’automatisation, rôles minimaux
- Traçabilité : admin events activés



Vers la production

Configuration et optimisation du démarrage
Configuration de production
Clustering
Sécurisation Keycloak
Comptes utilisateur et applications

Sécurité des applications Web

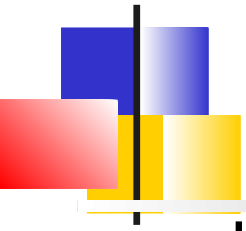


Comprendre les vulnérabilités des applications Web en consultant OWASP[1]

Appliquer ses recommandations :

- Authentification : S'assurer que les sessions sont sécurisées
- Autorisation : Donner toujours le minimum de privilèges
- Comprendre les TOP10 de OWASP
- Mettre à jour régulièrement les librairies et les frameworks utilisés
- Crypter les données sensible
- Trace et surveillance afin de détecter les attaques
- Firewall

1. Open Web Application Security Project



Recommandations oAuth2.0

Les spécifications oAuth2 et OpenID Connect sont très flexibles et certaines configuration ne sont pas sécurisée

Voir https://oauth.net/2/

- OAuth 2.0 for mobile and native apps
- OAuth 2.0 for browser-based apps
- OAuth 2.0 threat model and security considerations
- OAuth 2.0 security best current practice

OAuth 2.1 intègre plusieurs bonnes pratiques dans la spécification.

Configuration des applications clientes

Consentement requis : Activé cette option pour toute application tierce.

Type d'accès : Confidentiel si l'on peut conserver le crédentiel en toute sécurité côté serveur.

Implicit Flow Enabled : Jamais sauf pour application legacy

Direct Access Grants Flow Enabled: Jamais sauf pour application legacy

Valid Redirect URIs: Exact match

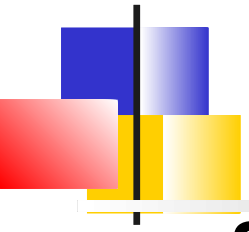
Algorithmes de signature

Keycloak supporte plusieurs algorithmes de signature :

- ***Signature Rivest–Shamir–Adleman (RSA)*** : Algorithme par défaut utilisé par Keycloak. Pas la plus sécurisée, mais la plus largement disponible.
- ***Elliptic Curve Digital Signature Algorithm (ECDSA)***: Plus sûr que RSA et est également beaucoup plus rapide. Recommandé si possible
- ***Hash-based message authentication code (HMAC)***: Algorithme de signature symétrique qui nécessite l'accès à un secret partagé.

On peut également choisir entre différentes longueurs de hachage de signature.
Avec des jetons à durée de vie relativement courte, une longueur de 256 bits est considérée correcte

Checklist de mise en production (1/2)



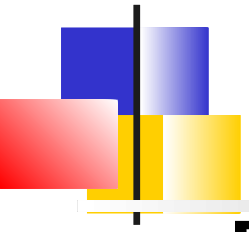
Serveur

- Mode start (jamais start-dev), image optimisée
- TLS de bout en bout, hostname configuré, proxy-headers cohérents
- Au moins 2 nœuds, health checks, sessions affines
- Base de données externe HA et sauvegardée

Administration

- Console admin non exposée, comptes nominatifs avec MFA
- Compte bootstrap supprimé

Checklist de mise en production (2/2)

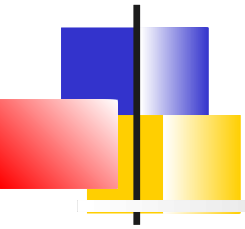


Realms et clients

- Politique de mot de passe, brute force detection, en-têtes de sécurité
- Durées de sessions et de jetons revues, rotation des refresh tokens
- Redirect URIs exactes, full scope désactivé, consentement pour les applications tierces

Exploitation

- Events activés avec expiration
- Métriques et logs centralisés
- Procédure de sauvegarde et de restauration testée



Supervision, maintenance et sauvegarde

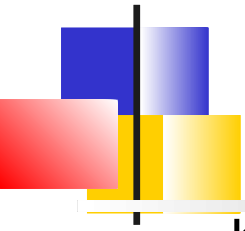
Métriques et supervision

Logs et événements

Sauvegarde et restauration

Mises à jour et migrations

Incidents courants



Métriques Keycloak

- Keycloak expose des métriques au format Prometheus
  - Activation : --metrics-enabled=true
  - Endpoint /metrics sur le port de management (9000)
- Familles de métriques :
  - JVM : mémoire, GC, threads
  - HTTP : nombre et durée des requêtes par URI et statut
  - Pool de connexions BD (agroal_*)
  - Caches Infinispan
- Métriques d’événements utilisateur : --event-metrics-user-enabled=true
  - Compteur keycloak_user_events_total par realm, client, type et erreur

Métriques utiles

Activité : logins réussis et en échec, émission et rafraîchissement de jetons

Performance : latence des endpoints token et auth (percentiles)

- Histogrammes : --http-metrics-histograms-enabled=true

Ressources : heap, pauses GC, CPU

Base de données : connexions actives, requêtes en attente de connexion

Cluster : taille du cluster, statistiques des caches

Disponibilité : état readiness des nœuds

Prometheus

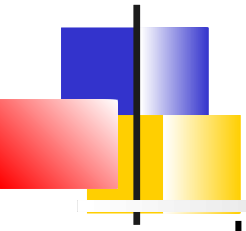
Collecte des métriques de chaque nœud :

```
scrape_configs:
  - job_name: keycloak
    metrics_path: /metrics
    scheme: https
    static_configs:
      - targets: ["kc1:9000", "kc2:9000"]
```

Sur Kubernetes : ServiceMonitor ou PodMonitor (Prometheus Operator)

Exemples d’alertes :

- Taux de LOGIN_ERROR anormal
- Latence p95 de l’endpoint token dégradée
- Requêtes en attente de connexion BD, nœud non ready



Tableaux de bord Grafana

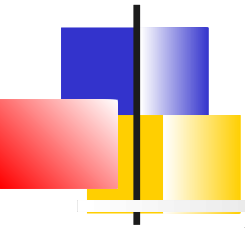
Le projet Keycloak fournit des tableaux de bord Grafana prêts à l'emploi

- Troubleshooting : erreurs, latences, état du cluster et des caches
- Capacity planning : charge, volumétrie des sessions, dimensionnement

Organisation conseillée :

- Vue synthèse : disponibilité, logins par minute, taux d'erreurs
- Vue par nœud : JVM, CPU, pool BD
- Vue par realm et client : événements utilisateur

Health checks



Activation : --health-enabled=true, exposés sur le port de management

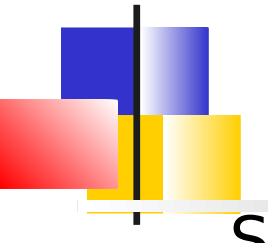
```
/health/started # démarrage terminé
/health/live # processus vivant
/health/ready # prêt à recevoir du trafic (BD accessible)
/health # agrégat
```

Usages :

- Sondes startup, liveness et readiness de Kubernetes
- Health checks du load balancer
- Supervision externe (disponibilité)

Ne pas exposer le port de management publiquement

Supervision de l’infrastructure

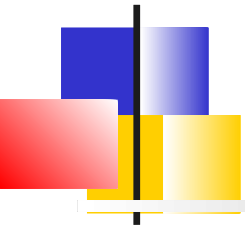


Supervision applicative :

- Sonde synthétique : parcours de login complet et obtention d’un jeton
- Validité du endpoint discovery et des clés JWKS

Supervision infrastructure :

- Nœuds : CPU, mémoire, disque, redémarrages de pods
- Base de données : réplication, espace, verrous, sauvegardes
- Réseau : load balancer, certificats TLS et leur expiration
- Dépendances : LDAP, IdP externes, SMTP



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Mises à jour et migrations
Incidents courants

Centralisation des logs

- Sortie JSON vers la console, collectée par la plateforme

```
kc.sh start --log-console-output=json
```

- Chaînes classiques :
  - Fluent Bit ou Vector → Loki ou Elasticsearch → Grafana ou Kibana
  - Syslog : --log=console,syslog --log-syslog-endpoint=siem:514
- Rétention définie selon les exigences d’audit et le RGPD

Événements vers un SIEM

- Le listener jboss-logging écrit les événements dans les logs
  - Erreurs en WARN, succès en DEBUG par défaut
  - Tracer aussi les succès en INFO :

```
--spi-events-listener-jboss-logging-success-level=info
```

- Listener personnalisé (SPI EventListenerProvider) :
  - Envoi vers Kafka, un webhook HTTP ou directement le SIEM
  - Découple l’audit de la base Keycloak
- Garder une expiration courte des événements stockés en base

Audit d’authentification

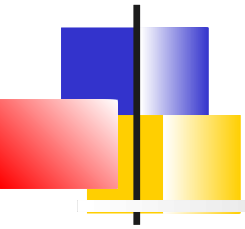
Indicateurs à surveiller :

- Échecs répétés par utilisateur ou par IP (brute force, credential stuffing)
- Connexions depuis des IP ou horaires inhabituels
- Comptes verrouillés, réinitialisations de mot de passe

Admin events sensibles :

- Attribution de rôles d’administration
- Régénération de secrets clients, modification des clés
- Création de clients ou de fournisseurs d’identité

Revue périodique et alertes sur les événements critiques



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Mises à jour et migrations
Incidents courants

Que sauvegarder ?

La base de données : source de vérité

- Realms, clients, utilisateurs, credentials, clés de signature, sessions persistées

Les fichiers du serveur :

- conf/keycloak.conf, providers/, themes/
- Certificats TLS et keystores
- Dockerfile ou image, manifests Kubernetes

Les secrets : mots de passe BD, secrets des comptes de service

Les exports de realms : complément utile, pas un substitut à la sauvegarde BD

Sauvegarde de la base

Exemple PostgreSQL :

```
pg_dump -Fc -h db -U keycloak keycloak > kc-$(date +%F).dump
```

Sauvegarde cohérente à chaud (transactionnelle)

Pour un RPO faible : archivage continu des WAL et restauration à un instant donné (PITR)

Service managé : sauvegardes automatiques du fournisseur

Chiffrer les sauvegardes : elles contiennent hachés, secrets et clés privées

Copie hors site et durée de rétention définie

Export et import de realms

En ligne de commande, sur une instance arrêtée ou dédiée :

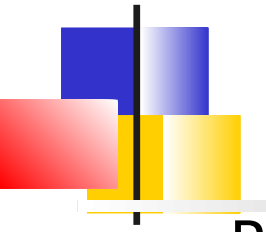
```
kc.sh export --dir /backup --realm myrealm --users realm_file
kc.sh import --dir /backup --override false
```

Au démarrage : start --import-realm (fichiers de data/import, realm existant non écrasé)

Depuis la console : Realm settings → Action → Partial export

- Sans utilisateurs, secrets masqués

Usages : migration entre environnements, initialisation, archive de configuration



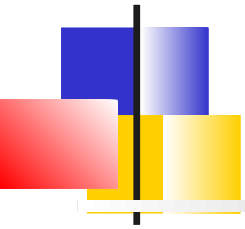
Restauration

Procédure type :

- Arrêter tous les nœuds
- Restaurer la base : pg_restore --clean -d keycloak kc.dump
- Redéployer la même version de Keycloak, providers et thèmes
- Démarrer un seul nœud, contrôler logs et /health/ready
- Tester un login et l’obtention d’un jeton, puis démarrer les autres nœuds

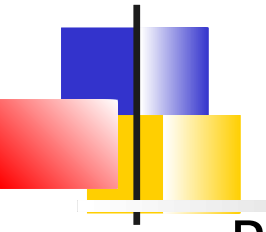
Les caches embarqués sont reconstruits au redémarrage

Tester la restauration régulièrement, sur un environnement isolé



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Mises à jour et migrations
Incidents courants



Politique de versions

- Plusieurs versions majeures par an
  - Seule la dernière version reçoit les correctifs de la communauté
  - Support long terme : Red Hat build of Keycloak
- Suivre les versions correctives : failles de sécurité (CVE)
- Avant chaque montée de version, lire :
  - Les release notes
  - Le guide Upgrading : changements incompatibles, options supprimées
- Monter régulièrement : plusieurs versions d’écart multiplient les changements à absorber

Procédure de montée de version



Préparation :

- Sauvegarde de la base
- Recompilation des providers sur la nouvelle version, vérification des thèmes
- Test en recette sur une copie de la base de production

Exécution :

- Arrêt de tous les nœuds pour une version majeure
- Nouvelle image, démarrage d'un nœud : migration automatique du schéma
- Contrôles puis démarrage des autres nœuds

Versions correctives : mise à jour progressive possible (Operator, stratégie Auto)

Migration de la base

Au démarrage, Keycloak met à jour le schéma (Liquibase)

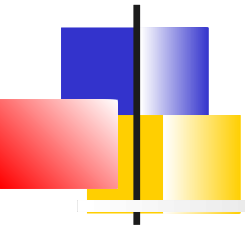
Mode manuel pour faire valider le script par un DBA :

```
kc.sh start --spi-connections-jpa-quarkus-migration-strategy=manual
```

Génère keycloak-database-update.sql sans l'appliquer

Points d'attention :

- Pas de retour arrière hors restauration de la base
- Durée de migration sur les grosses bases (sessions, événements) : purger avant
- Compatibilité des applications et des librairies clientes



Supervision, maintenance et sauvegarde

Métriques et supervision
Logs et événements
Sauvegarde et restauration
Mises à jour et migrations
Incidents courants

Démarche de diagnostic

Qualifier : tous les utilisateurs, un realm, un client, un nœud ?

Vérifier l’état : /health/ready de chaque nœud, métriques

Consulter :

- Les logs Keycloak et du proxy, sur la période concernée
- Les events LOGIN_ERROR et leur code d’erreur

Isoler la couche : proxy, Keycloak, base, cluster, fournisseurs externes

Reproduire avec curl et un jeton décodé

Augmenter le niveau de log de façon ciblée et temporaire

Incidents courants (1/2)

Redirections vers une mauvaise URL, erreurs d'issuer

- hostname ou proxy-headers mal configurés

Boucle de login, « Cookie not found »

- Sessions affines absentes, cookies SameSite, horloges désynchronisées

Démarrage bloqué

- Base inaccessible, migration en cours, verrou Liquibase (table DATABASECHANGELOGLOCK)

Les nœuds ne forment pas de cluster

- Découverte JGroups (jdbc-ping, DNS), port 7800 filtré

Incidents courants (2/2)

Lenteurs lors des pics de connexion

- CPU saturé par le hachage, pool BD épuisé, mémoire (OOMKilled)

Utilisateurs LDAP rejetés

- Bind credential expiré, certificat LDAPS, annuaire injoignable

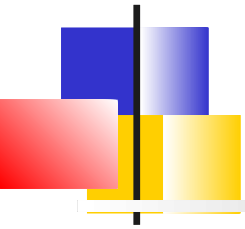
401 généralisés côté API

- Rotation de clés trop rapide, cache JWKS des applications

Emails non reçus

- Configuration SMTP, test depuis Realm settings → Email

Perte d'accès administrateur : kc.sh bootstrap-admin



Automatisation et exploitation quotidienne

Scripts d’administration
Sauvegardes planifiées
CI/CD et GitOps
Bonnes pratiques et checklist d’audit

Compte de service d’automatisation

- Client confidentiel dédié, Service accounts enabled
- Rôles minimaux du client realm-management :
  - manage-users, view-realm, manage-clients… selon les besoins
- Authentification kcadm par client credentials :

```
kcadm.sh config credentials --server https://auth.example.com \
  --realm master --client automation --secret "$KC_SECRET"
```

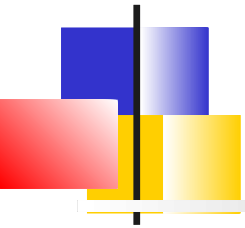
- Secret stocké dans un coffre (Vault, secrets CI/CD), rotation périodique

Scripts kcadm

- Exemple : désactiver les comptes d’une liste de départs

```
#!/bin/bash
set -euo pipefail
kcadm.sh config credentials --server "$KC_URL" --realm master \
  --client automation --secret "$KC_SECRET"
while read -r u; do
  id=$(kcadm.sh get users -r myrealm -q username="$u" -q exact=true \
       --fields id --format csv --noquotes)
  [ -n "$id" ] && kcadm.sh update users/$id -r myrealm -s enabled=false
done < departs.txt
```

- Autres usages : création de clients, affectation de rôles, purge, rapports



API REST depuis un script

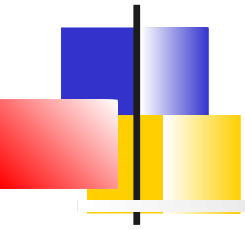
Obtention du jeton puis appel de l'API d'administration :

```
TOKEN=$(curl -s -d grant_type=client_credentials \
  -d client_id=automation -d client_secret="$KC_SECRET" \
  "$KC_URL/realms/master/protocol/openid-connect/token" | jq -r .access_token)

curl -s -H "Authorization: Bearer $TOKEN" \
  "$KC_URL/admin/realms/myrealm/users?briefRepresentation=true&max=100" \
  | jq -r ".[].username"
```

Export partiel automatisé :

```
POST /admin/realms/myrealm/partial-export?exportClients=true&exportGroupsAndRoles=true
```



Automatisation et exploitation quotidienne

Scripts d’administration

Sauvegardes planifiées

CI/CD et GitOps

Bonnes pratiques et checklist d’audit

Planification des sauvegardes

Exemple : CronJob Kubernetes quotidien

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: kc-db-backup }
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: pgdump
              image: postgres:16
              envFrom: [ { secretRef: { name: kc-db } } ]
              command: ["sh", "-c", "pg_dump -Fc -h postgres-rw \
                -U keycloak keycloak > /backup/kc-$(date +%F).dump"]
              volumeMounts: [ { name: backup, mountPath: /backup } ]
          volumes: [ { name: backup, persistentVolumeClaim: { claimName: kc-backup } } ]
```

Sauvegardes : organisation

- Sauvegarde BD quotidienne, plus WAL si RPO faible
- Export de configuration des realms versionné (partial export)
- Rotation des sauvegardes, chiffrement et copie hors site
- Supervision : alerte si une sauvegarde échoue ou manque
- Test de restauration planifié (mensuel par exemple)

Fenêtres de maintenance

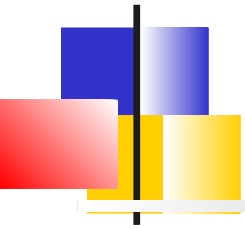
Opérations à planifier :

- Montées de version, changements de configuration serveur
- Rotation des clés de signature des realms
- Purge des événements, renouvellement des certificats

Rotation de clé sans coupure :

- Ajouter une nouvelle clé active de priorité supérieure
- Garder l’ancienne en passive le temps de validité des jetons
- Puis la désactiver et la supprimer

Retirer un nœud du load balancer avant intervention, communiquer auprès des équipes applicatives



Automatisation et exploitation quotidienne

Scripts d’administration
Sauvegardes planifiées
CI/CD et GitOps
Bonnes pratiques et checklist d’audit

Configuration as code

Realms, clients, rôles, scopes, flows décrits dans Git

- Revue par merge request, historique, retour arrière
- Application idempotente par un pipeline

Séparer :

- La configuration (Git) des données (utilisateurs en base)
- Les valeurs par environnement (variables) du modèle commun
- Les secrets (coffre) des fichiers versionnés

Outils : keycloak-config-cli, Terraform, Ansible, Keycloak Operator

keycloak-config-cli

- Outil open source d’import idempotent de fichiers de realm JSON ou YAML
  - Crée, met à jour et peut supprimer les éléments
  - Substitution de variables : $(env:CLIENT_SECRET)

```
docker run --rm \
  -e KEYCLOAK_URL=https://auth.example.com \
  -e KEYCLOAK_USER=admin -e KEYCLOAK_PASSWORD="$PWD_ADMIN" \
  -e IMPORT_FILES_LOCATIONS="/config/*" \
  -e IMPORT_VARSUBSTITUTION_ENABLED=true \
  -v ./config:/config adorsys/keycloak-config-cli:<version>
```

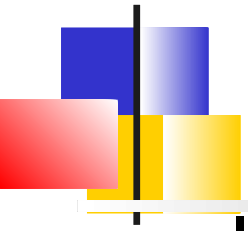
- Image à choisir selon la version de Keycloak ciblée

Terraform

- Provider keycloak/keycloak : realms, clients, rôles, IdP, flows...

```
provider "keycloak" {
  url = "https://auth.example.com"
  client_id = "terraform"
  client_secret = var.kc_secret
}
resource "keycloak_realm" "app" {
  realm = "app"
}
resource "keycloak_openid_client" "web" {
  realm_id = keycloak_realm.app.id
  client_id = "web-app"
  access_type = "CONFIDENTIAL"
  standard_flow_enabled = true
  valid_redirect_uris = ["https://app.example.com/callback"]
}
```

- plan en revue, apply dans le pipeline, état stocké à distance



Ansible

- Installation et configuration des serveurs : rôles et collections dédiés
- Configuration des realms : modules community.general.keycloak_*

```
- name: Client web-app
  community.general.keycloak_client:
    auth_keycloak_url: https://auth.example.com
    auth_realm: master
    auth_username: "{{ kc_admin }}"
    auth_password: "{{ kc_admin_password }}"
    realm: app
    client_id: web-app
    redirect_uris: ["https://app.example.com/callback"]
    state: present
```

- Autres modules : keycloak_realm, keycloak_role, keycloak_group, keycloak_user...

GitOps sur Kubernetes

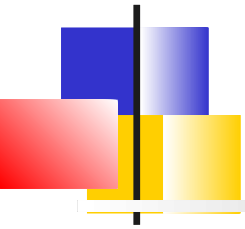
Ressources Keycloak et KeycloakRealmImport versionnées, synchronisées par Argo CD ou Flux

Limite : KeycloakRealmImport crée un realm mais ne met pas à jour un realm existant

- Mises à jour via un Job keycloak-config-cli ou Terraform

Pipeline type :

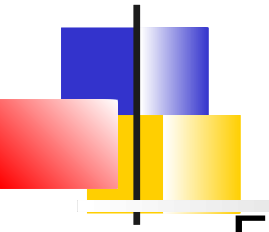
- Merge request → validation (plan, dry-run)
- Application en recette → test de login automatisé
- Promotion en production



Automatisation et exploitation quotidienne

Scripts d’administration
Sauvegardes planifiées
CI/CD et GitOps
Bonnes pratiques et checklist d’audit

Bonnes pratiques d’exploitation



- Environnements homogènes, configuration versionnée
- Modifications en production via le pipeline, pas depuis la console
- Runbooks pour les opérations et incidents courants
- Alertes sur la disponibilité, les erreurs et les sauvegardes
- Versions correctives appliquées rapidement
- Rotation des secrets et des clés
- Revue périodique des comptes d’administration et des clients

Checklist d’audit (1/2)

Comptes et accès

- Administrateurs nominatifs, MFA, droits minimaux
- Comptes de service : rôles justifiés, âge des secrets
- Comptes inactifs ou orphelins

Clients

- Clients inutilisés à supprimer
- Redirect URIs et web origins exactes
- Type public ou confidentiel adapté, PKCE, full scope désactivé

Checklist d’audit (2/2)

Configuration des realms

- Politique de mot de passe, brute force detection, en-têtes
- Durées de sessions et de jetons
- Clés : algorithmes, ancienneté

Plateforme

- Version supportée, CVE corrigées
- TLS, console admin non exposée, port de management protégé
- Sauvegardes testées, rétention des événements, alertes actives

Annexes

FAPI

Authentification mTLS et CIBA

Maximiser la sécurité des standards

Financial Grade API

Financial-grade API (FAPI) est une spécification technique qui durcit oAuth2 et OpenID

- Publiée par l'OpenID Foundation, avec une adoption croissante dans les environnements réglementés (notamment PSD2[1] en Europe).

Objectifs principaux :

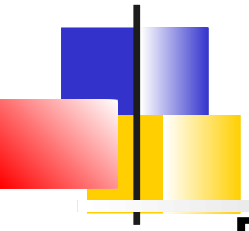
- Protéger contre les attaques de type interception de jeton, redirection malveillante, injection de code, etc.
- Renforcer l'authentification et la signature des requêtes.
- Garantir l'intégrité des messages et l'identité du client (preuve de possession).

1. Payment Services Directive 2

Exigence FAPI

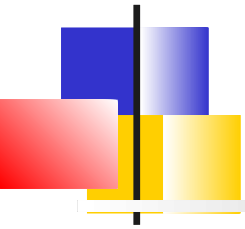
- Flow OpenID : **Authorization Code + PKCE** obligatoire
- Authentification client : **mTLS** ou **Private Key JWT**[1]
- Jetons : Doivent être **signés et chiffrés**
- ID Token : Doit inclure des **claims supplémentaires** (`acr`, `auth_time`, etc.)
- Délai de validité : Jetons courts, rotation imposée
- Preuve de possession : Recommandation : **DPoP ou mTLS**

1. Dans ce cas, client génère un JWT signé avec sa clé privé et l'envoie via le paramètre *client_assertion*



Financial-Grade API (FAPI)

- FAPI est un groupe de travail qui essaie de spécifier des profiles de configuration OIDC pour le domaine bancaire. 2 profils sont définis :
  - FAPI 1.0 – Part 1: Baseline API Security Profile
    https://openid.net/specs/openid-financial-api-part-1-1_0.html
  - FAPI 1.0 – Part 2: Advanced Security Profile
    https://openid.net/specs/openid-financial-api-part-2-1_0.html
- Keycloak 15.0.2 a été certifié FAPI OpenID Provider en Janvier 2022



Annexes

FAPI

Authentification mTLS et CIBA

Maximiser la sécurité des standards

Authentification par certificat client (mTLS)

Le client s’authentifie avec un certificat TLS client, via HTTPS mutuel (mTLS).

Keycloak utilise ce certificat pour identifier et authentifier l’utilisateur sans saisie de login/mot de passe.

Avantages de mTLS

- Sécurité forte : authentification à deux facteurs implicite (possession du certificat)
- Sans mot de passe : pas de credential à stocker, pas de phishing
- Adapté aux machines ou environnements contrôlés : terminaux d'entreprise, API inter-applications
- Compatible avec un flow OAuth2 / OIDC standard si bien configuré

Inconvénients / limites de mTLS

Complexité de déploiement :

- Génération et gestion des certificats client (PKI interne)
- Configuration des reverse proxies pour le mTLS

Expérience utilisateur moins fluide :

- Nécessite parfois installation manuelle des certificats sur les terminaux

Pas adapté au grand public (ex. applications mobiles ou utilisateurs anonymes)

Pas natif dans tous les navigateurs/appareils mobiles

Authorization code Flow avec mTLS

Accès initial :

- Le client accède à Keycloak via HTTPS avec authentification mutuelle (mTLS).
- Le certificat client est présenté dès la connexion TLS.

Configuration côté Keycloak :

- Le certificat client est requis ou demandé (HTTPS_CLIENT_AUTH=request|required)
- La validation du certificat est assurée par Keycloak ou par un reverse proxy TLS (ex : NGINX en mTLS).

Authentification de l'utilisateur :

- Keycloak utilise un authenticator SPI X.509 pour mapper le certificat à un utilisateur : mapping via : subjectDN, issuerDN, serialNumber, SAN, etc.
- L'utilisateur est automatiquement connecté, sans saisie de login/mot de passe.

Mise en œuvre :

- Ajouter l'authenticator X.509 dans le flow "browser".
- Configurer un proxy TLS si besoin (pour extraire le certificat).
- Définir les règles de mapping certificat → user dans Keycloak.

CIBA Flow

Client-Initiated Backchannel Authentication (CIBA) est une extension d'OpenID Connect pour l'authentification asynchrone sans redirection navigateur
Idéale pour les appareils sans interface utilisateur (IoT, banques, assistants vocaux…).

Principe :

- Le client n'ouvre pas de navigateur.
- Il initie l'authentification côté serveur (via un backchannel).
- Le fournisseur d'identité notifie l'utilisateur sur un autre canal (ex : mobile, push).
- L'utilisateur accepte/refuse, et le jeton est délivré ensuite.

Terminologie CIBA

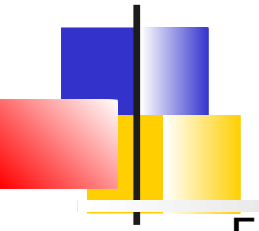
Backchannel Authentication Endpoint : Point d'entrée CIBA du serveur d'authentification

login_hint / id_token_hint : Identifie le user à authentifier

Binding Message : Message à afficher à l'utilisateur

Notification Endpoint (optionnel) : endpoint de callback du client pour recevoir le résultat de l'authentification

Poll / Ping / Push : Méthodes de récupération du token après autorisation



Flow CIBA

Flow pour l’obtention du jeton

1) Le client envoie une requête d’authentification backchannel au serveur, avec un *login_hint* ou *id_token_hint*, et optionnellement un *notification_endpoint*.
2) Le serveur notifie le client via le notification_endpoint, quand l’utilisateur a terminé l’authentification, que ce soit un succès ou un échec.
3) Le client utilise cette notification et l’*auth_req_id* qu’il a reçu à l’étape 1 pour récupérer un token auprès du token endpoint.

Flow pour l’authentification

1) Lors d’une requête d’authentication, le serveur d’autorisation notifie un service
2) Le service acquitte puis demande à authentifier l’utilisateur
3) Il informe le serveur du résultat de l’authentification. Si elle réussie, le jeton est disponible pendant x temps

Support dans Keycloak

Activation par client (Enable CIBA)

Configuration générale *Authentication → CIBA Policy*, peut être surchargé par client (pas supporté par l'UI par contre) :

- Mode de livraison du jeton : Seulement ping
- Expiration de la demande authentification (120s par défaut)
- Intervalle de polling
- Comment identifier l'utilisateur : Seulement login_hint

Canal d'authentification implémenté sous forme de SPI

- Soit on implémente son propre canal
- Soit on utilise le SPI prédéfini : ***HTTP Authentication Channel Provider***

Pas de support pour le notification endpoint dynamique

HTTP Authentication Chanel Manager

Doit être activé en temps que SPI :

```
kc.[sh|bat] start --spi-ciba-auth-channel-ciba-http-auth-channel-http-authentication-channel-uri=https://backend.internal.example.com
```

Après une demande d'authentification

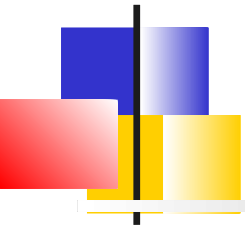
POST Keycloak avec un jeton

Le backend répond avec un 201 et conserve le jeton

Lorsque l'utilisateur est authentifié, le backend ou le client renvoie une requête POST vers :

/realms/[realm]/protocol/openid-connect/ext/ciba/auth/callback avec :

- Le jeton initial
- Le résultat de l'authentification



Annexes

FAPI
Authentification mTLS et CIBA
Maximiser la sécurité des standards

Maximiser la sécurité des standards

Modèle de menaces – RFC 9700

Preuve de possession

FIPS

FAPI

Contexte de RFC 9700

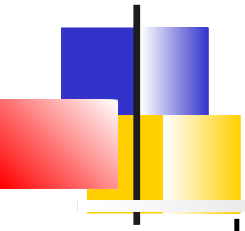
Titre : "OAuth 2.0 Security Best Current Practice"

Publiée en janvier 2025

Actualise les recommandations de sécurité pour OAuth 2.0, en tenant compte des retours d’expérience accumulés depuis la publication initiale d’OAuth

Le RFC 9700 :

- Actualise les modèles de menaces (au-delà des RFC 6749, 6750, 6819),
- Propose de nouvelles contre-mesures face aux attaques connues et émergentes,
- Déprécie certains flux jugés peu sûrs (notamment le Implicit Grant),
- Introduit des exigences de sécurité renforcées, en anticipation de OAuth 2.1.



Menaces

Le RFC identifie 5 types d'attaquants (A1–A5), allant de l’attaquant web basique (exécutant un client malveillant) au network attacker capable d’intercepter ou modifier des communications. Il inclut :

- Les attaques via Referer ou historique du navigateur
  Contre mesures : CSP, Referrer-Policy: strict-origin, éviter le stockage des token dans l’url
- Les attaques par mix-up, le code/jeton est renvoyé à un serveur d’autorisation malveillant
  Contre mesure : Reditect URI, claim iss
- L’injection de code d’autorisation ou de tokens d’accès,
- Les attaques sur les flux de communication via navigateur (postMessage...), Exploitation des communications inter-origines dans le navigateur.
- Le PKCE downgrade attack, où l’attaquant tente de forcer une négociation sans PKCE.

Protection des flux basés sur la redirection

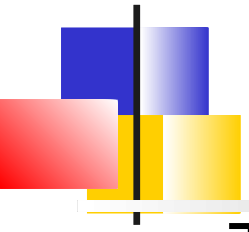
Validation stricte des URI de redirection : utilisation de correspondance exacte, pas de motifs avec jokers, sauf pour les ports dans les apps natives.

Éviter les redirections ouvertes : clients et serveurs doivent empêcher les redirections vers des URI non vérifiés.

Protection contre les attaques CSRF :

- Via state + token CSRF à usage unique,
- Ou via PKCE (Proof Key for Code Exchange),
- Ou via nonce en OpenID Connect (Implémenté nativement dans Keycloak)

Injection et Rejeu de Code d’autorisation

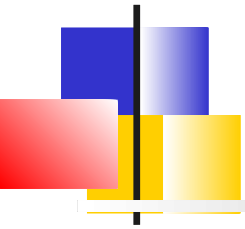


Tous les clients (même confidentiels) doivent implémenter PKCE.

L’usage de méthodes de challenge sûres (S256) est obligatoire.

Les serveurs doivent refuser toute tentative d’échange de code si *code_challenge* est attendu mais absent.

Prévention du rejeu des tokens



Access Tokens : contraindre leur usage à l'émetteur via une preuve de possession :

- mutual TLS (RFC8705),
- ou DPoP (RFC9449).

Refresh Tokens :

- Doivent être tournants (rotation) ou contraints à un émetteur unique.
- Interdiction d'utiliser les mêmes tokens entre clients.

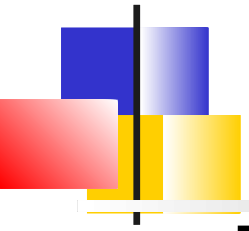
Restrictions des privilèges des tokens

Restreindre les tokens à un périmètre minimum (scopes)

Imposer une audience (aud) claire et vérifiée côté ressource

Utiliser *authorization_details*[1] (RFC9396) si nécessaire.

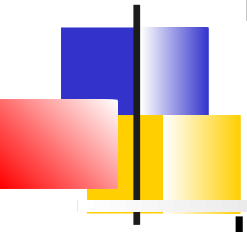
1. Claim utilisé pour spécifier des détails sur les ressources auxquelles le client souhaite accéder, les actions qu'il souhaite effectuer, et d'autres métadonnées pertinentes.



Authentification des clients

- Fortement recommandé d’utiliser une authentification asymétrique :
  - TLS mutuel, ou
  - JWT signé (private_key_jwt).

Métadonnées et configuration automatique



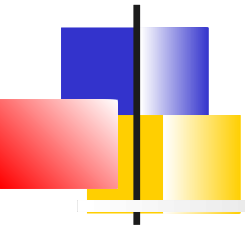
Utiliser Authorization Server Metadata (RFC8414).

Accès des méta-données de configuration du serveur via :
`/.well-known/openid-configuration`
et
`/.well-known/oauth-authorization-server`

Avantages :

- Réduction des erreurs de configuration,
- Détection automatique des capacités de sécurité (PKCE, algorithmes supportés...).

Maximiser la sécurité des standards



Modèle de menaces – RFC 9700

Preuve de possession

FIPS

FAPI

Introduction - DPoP

DPoP – Demonstration of Proof of Possession est définie via la RFC 9449

C’est un mécanisme léger pour démontrer que le client possède une clé privée liée au token

Recommandé par la RFC 9700

Alternative légère à mTLS, adaptée aux apps mobiles, SPA, etc.

Objectif : éviter le rejeu (replay) d’un jeton d’accès intercepté par un attaquant. Le jeton devient inutile sans la clé privée du client.

Mécanismes

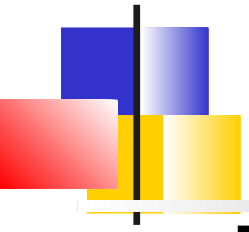
1) Le client génère une paire de clés asymétriques (EC ou RSA)

2) Lors de la requête d'authentification :
   Il envoie sa clé publique encodée dans un header DPoP (JWT signé)
   Le serveur d'autorisation enregistre le thumbprint de cette clé et l'insère dans l'access token sous le claim ***cnf.jkt***

3) Lors de chaque appel API avec le jeton d'accès :
   Le client signe un nouveau JWT DpoP qui contient la clé publique
   Ce JWT est ajouté dans l'en-tête de la requête

4) Le serveur de ressources :
   Vérifie que le JWT DPoP est valide
   Vérifie que le token d'accès a été émis pour cette clé en recalculant le thumbprint avec le même algorithme

Résultat : l'appel ne peut être effectué que par le détenteur de la clé privée.



DpoP et Keycloak

Depuis la v22+, Keycloak supporte DpoP en mode Preview

Ce mécanisme est désactivé par défaut. Pour l'activer :

```
kc.sh start --features=dpop
```

Ensuite pour un client, activer DpoP dans la configuration

Implémentation côté client

Générer une clé privée/publique (ex : EC P-256)

- Construire un JWT DPoP signé avec la clé privée
- Inclure les claims : htu, htm, jti, iat
- Ajouter l'en-tête DPoP à chaque requête vers l'API
- Stocker et réutiliser la clé

Librairies utiles :

- JavaScript jsonwebtoken (npm)
- Java jjwt (Java JWT), Nimbus JOSE+JWT
- Python PyJWT
- C#/.NET System.IdentityModel.Tokens.Jwt

L'en-tête DPoP doit être généré pour chaque appel, car le JWT est horodaté (iat) et lié à l'URL (htu).

Exemple avec *jjwt*

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
 keyGen.initialize(2048);
 KeyPair keyPair = keyGen.generateKeyPair();

 // Construction du JWK (simplifié pour l'exemple)
 Map<String, Object> jwk = new HashMap<>();
 jwk.put("kty", "RSA");
 jwk.put("n", Base64.getUrlEncoder().withoutPadding() .encodeToString(keyPair.getPublic().getEncoded()));
 jwk.put("e", "AQAB");

 // Header personnalisé DPoP
 Map<String, Object> header = new HashMap<>();
 header.put("typ", "dpop+jwt");
 header.put("alg", "RS256");
 header.put("jwk", jwk);

 // Claims DPoP requis
 Map<String, Object> claims = new HashMap<>();
 claims.put("jti", UUID.randomUUID().toString());
 claims.put("htm", "POST");
 claims.put("htu", "https://exemple.com/oauth2/token");
 claims.put("iat", System.currentTimeMillis() / 1000);

 // Génération du JWT signé
 String jwt = Jwts.builder()
     .setHeader(header)
     .setClaims(claims)
     .signWith(keyPair.getPrivate(), SignatureAlgorithm.RS256)
     .compact();
```

Avantages et limites

Avantages :

- Légèreté → pas besoin de certificat client ou infrastructure PKI
- Facilement intégrable dans des applications SPA ou mobiles
- Réduit considérablement le risque de rejeu de jeton

Limites :

- Ne sécurise que les clients honnêtes (pas d'attestation du matériel comme mTLS)
- Nécessite une gestion locale de clés (durée de vie, persistance)
- Dépend du support correct côté ressource server

Pour les environnements critiques (FAPI Advanced), DPoP peut être insuffisant → préférer mTLS.

TP 7.1

Preuve de possession avec mTLS

- Il est également possible d'effectuer la preuve de possession avec mTLS
  - Il faut configurer Keycloak afin qu'il associe le certificat client au jeton.
    Advanced – Advanced Settings - OAuth 2.0 Mutual TLS Certificate Bound Access Tokens Enabled
  - Keyclak renvoie alors un jeton avec le claim cnt contenant l'empreinte du certificat (thumbprint)
  - Lorsque le serveur d'api reçoit le jeton un peut vérifier le certificat client et le thumbprint

Maximiser la sécurité des standards

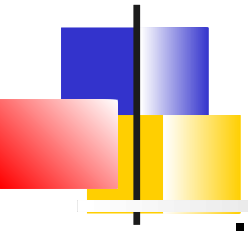
Modèle de menaces – RFC 9700
Preuve de possession
FIPS
FAPI

Introduction

FIPS (Federal Information Processing Standards) est un ensemble de standards de sécurité informatique définis par le gouvernement des États-Unis.

Le plus connu est FIPS 140-2 / 140-3, qui spécifie les exigences pour les modules cryptographiques (logiciels ou matériels).

Objectif : Assurer que les algorithmes cryptographiques utilisés sont robustes, éprouvés et conformes



Implications

- Interdiction des algorithmes faibles ou obsolètes (ex : MD5, SHA-1, RSA 1024)
- Imposition de l’usage de :
  - RSA ≥ 2048 bits
  - AES-256 / GCM
  - SHA-256, SHA-384, SHA-512
  - Signatures PSS (RSA-PSS)
- Nécessité d’utiliser une JVM et des fournisseurs cryptographiques compatibles FIPS

Conditions pour keycloak

Le serveur doit tourner sur un système FIPS-activé (ex. RHEL/Fedora, ou Ubuntu FIPS via Ubuntu Pro)

Keycloak utilise un Java FIPS-compliant via OpenJDK.

BouncyCastle FIPS : la version par défaut de BouncyCastle utilisé pour la cryptography n'est pas FIPS compliant.

=> Il faut installer les librairies adéquates dans ***KEYCLOAK_HOME/providers*** :

- *bc-fips-2.0.0.jar*
- *bctls-fips-2.0.19.jar*
- *bcpkix-fips-2.0.7.jar*

Le keystore doit être compatible FIPS :

- PKCS12 : fonctionne en mode non-approved
- BCFKS : requis pour le mode FIPS strict

Démarrage

Une fois les conditions remplies, on peut démarrer keycloak en mode ***fips*** ou en ***strict fips***.

Modifier le fichier *java.security* afin que la JVM ne tente pas un algorithme interdit. Ex :

```
security.provider.1=org.bouncycastle.jcajce.provider.BouncyCastleFipsProvider
security.provider.2=org.bouncycastle.jsse.provider.BouncyCastleJsseProvider
security.overridePropertiesFile=true

export JAVA_OPTS_APPEND="-Djava.security.properties=/opt/keycloak/conf/fips-java.security"
```

Activer la features fips et éventuellement positionner le mode script

```
bin/kc.[sh|bat] start --features=fips --fips-mode=strict --hostname=localhost \
     --https-key-store-password=passwordpassword \
     --log-level=INFO,org.keycloak.common.crypto:TRACE,org.keycloak.crypto:TRACE
```

Impacts de FIPS sur Keycloak et les applications

Avantages

- Conformité réglementaire (USA, institutions financières ou gouvernementales)
- Usage exclusif d'algorithmes robustes
- Accroît la sécurité globale de votre chaîne d'authentification
- Souvent exigé pour FAPI Advanced, OpenBanking, ou projets certifiables

Limitations / Effets de bord

- Algorithmes : Algos faibles désactivés (ex. SHA-1, RSA-1024)
- Signatures : Certaines signatures interdites (ex. none)
- Tokens JWT : JWT doivent être signés avec algo FIPS-compatibles
- Clients existants : Incompatibles si ils utilisent un algo non autorisé
- Intégrations tierces Doivent être compatibles FIPS

Maximiser la sécurité des standards

Modèle de menaces – RFC 9700
Preuve de possession
FIPS
FAPI

Introduction

FAPI (Financial-grade API) est un ensemble de spécifications de sécurité renforcées, conçues pour protéger les échanges OAuth2 / OIDC dans des contextes hautement sensibles, en particulier dans :

- la finance (Open Banking UK, Berlin Group, PSD2)
- la santé
- les données personnelles critiques

Apports :

- Des profils renforcés d'OAuth2 / OIDC
- Des exigences sur les clients, tokens, signatures, preuves de possession
- Une standardisation du niveau de sécurité pour des appels API critiques

Profils FAPI

FAPI 2.0 combine plusieurs recommandations (RFC 9207, RFC 8705, RFC 8707, DPoP...) et impose un ensemble de contraintes, regroupées dans deux profils principaux :

- ***Baseline***
  Renforce OAuth/OIDC standard
  Clients web ou natifs sécurisés
- ***Advanced***
  Sécurité maximale (certification)
  Services financiers, eIDAS, etc.

Principales exigences FAPI

PKCE obligatoire : Même pour clients confidentiels

Preuve de possession (DPoP ou mTLS) : Obligatoire pour les tokens

Signed Authorization Response (JARM) : Réponse d'autorisation encapsulée dans un jeton JWT signée (*response_mode=jwt* dans la requête initiale)

Claims précises (*aud, nonce, acr, client_id*) : Présence vérifiée et horodatée

Flows interdits : *implicit, password, client_credentials* (selon profil)

ID Token signé obligatoirement : Pas de token non signé ou avec alg: none

Refresh tokens bound (DpoP/mTLS) : Refresh token lié à une clé, même pour clients publics

FAPI et Keycloak

Keycloak est conforme FAPI1 depuis la version 15.0.2, il n'est pas officiellement certifié FAPI-2.0

Via les ***client policies***, Keycloak permet d'imposer des exigences FAPI (Baseline, Advanced, etc.) sans avoir à tout configurer manuellement pour chaque client.

Ces politiques contrôlent dynamiquement les conditions d'enregistrement, d'authentification et de traitement des clients en fonction de règles (policies) et de profils (profiles).



Mise en œuvre

Admin → Realm Settings → Client Policies

Créer une politique et définir les clients qui y seront associés (tous, par rôle, par client_id, par client_scope)

Appliquer un profil client

- En utilisant les profils définis
- Ou un profil custom qui spécifie chaque feature requises (*pkce-enforcer, dpop-enforcer, mtls-holder-of-key-enforcer*, ...)

Tout client qui correspond à la policy définie sera soumis automatiquement aux règles FAPI imposées.